

La ventana de escritorio

AlgoProject — manual de uso. Generado el 2026-09-28 con `tools/manual.py`.

1. 61. Proyectos — la galería de todos los proyectos, y el camino hasta una estrategia
2. 65. Proyecto — el raíl del workflow: mapa, botones de correr y estado
3. 69. Proyecto — filtros sobre un databank, y «Continuar workflow»
4. 68. Estrategia — la ficha de una estrategia: curvas, estadísticas, metadatos y «Archivar»
5. 58. Estrategia — cómo se lee el resultado de un estudio
6. 70. OPERACIÓN — «En marcha», «Registro de búsquedas» y la franja de trabajos
7. 71. Portfolios — las estrategias archivadas, e «Importar»
8. 35. La ventana — cómo se abre, el mapa de sus zonas, y las plantillas
9. 38. Activos — costes, tramos, MC Retest y Cross Market de cada instrumento
10. 66. Configuración SQX — los ajustes con que nacen los proyectos nuevos
11. 67. Datos — AlgoData, velas, spread real y calidad del feed
12. 60. Cómo se lee la ventana — etiquetas, números, identidad y modo lectura

61. Proyectos — la galería de todos los proyectos, y el camino hasta una estrategia

Qué pregunta responde

- **¿Qué proyectos tengo, dónde están y cómo van?** La zona **Proyectos** (grupo PROYECTO de la barra lateral) enseña una tarjeta por proyecto de SQX, esté en el maestro, en el conductor o en el custodio, **tenga informes o no**. Antes, un proyecto recién construido no aparecía en ningún sitio hasta que un estudio le escribía un informe.
- Cada tarjeta dice **símbolo y timeframe**, **cuántas estrategias** tiene en total entre todas sus databanks, **qué plantilla** lo generó y **en qué estado** está: corriendo, parado, terminado o construido.
- **¿Cómo llego de un proyecto a una estrategia?** Clic en la tarjeta → se abre **Proyecto** con ese proyecto cargado → doble clic en una fila del panel de databanks → se abre **Estrategia** con su ficha. La barra de arriba (Proyectos > proyecto > estrategia) y Ctrl+K llevan a las tres.

Cuándo lo usas, y cuándo no

Lo usas para empezar a mirar cualquier proyecto: es la puerta de entrada al espacio de trabajo.

No sirve para lanzar nada en SQX. La galería lee ficheros; al único worker que está corriendo un proyecto le pregunta su estado con `-project action=status`, que no sincroniza nada. Al maestro no le pregunta nunca: de él solo lee ficheros. Nunca se cuenta una databank con el verbo `count` de SQX, que sincroniza desde disco y borraría lo que el worker solo tiene en memoria.

No enseña los cinco proyectos de fábrica de SQX (Builder, Retester, Optimizer, PortfolioComposer, PortfolioMaster): no son proyectos de nadie (regla 10 de `CLAUDE.md`).

Antes de empezar

Nada. La ventana se abre con `bin/algoui` (o `algoui` desde cualquier terminal, o el icono **AlgoProject** del escritorio). Qué hacer si no abre, y el mapa de todas las zonas de la barra lateral con el capítulo de cada una: capítulo 35.

La primera vez que el demonio responde a la galería tarda **unos 12 segundos**: calcula la identidad de cada estrategia de los tres installs (7 181 el 27-09-2026) para contar las distintas. La ventana no se congela: la galería dice «Leyendo los proyectos de los installs...» y se rellena sola al llegar la respuesta. Las siguientes veces tarda unas centésimas.

Cómo se ejecuta

En la ventana: barra lateral → PROYECTO → **Proyectos**. O Ctrl+K y escribir el nombre del proyecto: la fila dice símbolo, timeframe, estrategias y estado, y Enter abre el proyecto igual que un clic en su tarjeta.

La misma lista, sin ventana:

```
curl -s 127.0.0.1:8765/api/projects/all | python3 -m json.tool
```

Dónde vive una estrategia de un proyecto, por su identidad (lo que usa la ventana al abrir la ficha; la identidad es el SHA-256 que se ve en el tooltip de la estrategia):

```
curl -s "127.0.0.1:8765/api/projects/find?project=USDJPY&identity=3a34db2c3a4eedb545466b83fb5c190027165e94119f776af00cccd8df0d8d2a"
```

parámetro	obligatorio	qué hace
project	sí	el proyecto
identity	sí	la identidad de la estrategia
databank	no	la databank donde mirar primero; sin él, se miran todas

Nada de esto toca SQX.

Qué produce

La galería no escribe ningún fichero. Lo que sí hace el **clic en una tarjeta**:

1. **Pregunta antes de cargar.** Una caja lista las databanks con estrategias («SPP OOS (219 estrategias)», ...) y recuerda que las operaciones y la cosecha se exportan con `orderstocsv` en el conductor, un trabajo detrás de otro.
2. **Sí:** pide la carga de cada una de esas databanks (`POST /api/load` , lo mismo que elegir una databank a mano; las métricas se leen de los `.sqx` sin SQX), elige el proyecto y, como databank, **la primera que tiene estrategias**, y abre **Proyecto**. La línea de estado dice cuántas se pidieron («USDJPY: carga pedida para 3 de 3 databanks») y la franja de trabajos de abajo va mostrando los exports.
3. **No:** elige solo el proyecto, sin databank (elegir una ya la cargaría), y abre Proyecto sin cargar nada. La línea de estado dice «AUDJPY: elegido sin cargar».

Ctrl+K lista los proyectos que la galería ya ha leído; si la abres justo al arrancar la ventana, antes de que lleguen, dice «Los proyectos aún se están leyendo» en vez de congelarse. Los proyectos recientes se recuerdan solo por su nombre.

El doble clic en una estrategia del panel la elige (proyecto, databank y estrategia) y abre **Estrategia**.

Cómo se lee el resultado

PROYECTOS		
Una tarjeta por proyecto de cualquier install, tenga informes o no. Las estrategias son el total entre todos sus databanks. Clic en una tarjeta para abrir el proyecto entero.		
AUDJPY · H1 AUDJPY Estrategias 2 341 Plantilla sin plantilla registrada ● TERMINADO en el maestro	EURJPY · H4 EURJPY_H1 Estrategias 3 Plantilla sin plantilla registrada ● TERMINADO en el maestro	EURUSD · M30 EURUSD Estrategias 1 030 Plantilla sin plantilla registrada ● TERMINADO en el maestro
GBPJPY · H4 GBPJPY_H1 Estrategias 1 558 Plantilla sin plantilla registrada ● TERMINADO en el maestro	SP500ft · H4 Infinox_SP500ft_H4_HighPrecision Estrategias 145 Plantilla sin plantilla registrada ● TERMINADO en el maestro	USA500 · H1 SP500_H1 Estrategias 27 Plantilla sin plantilla registrada ● TERMINADO en el maestro
USDCHF · H1 USDCHF Estrategias 2 Plantilla sin plantilla registrada ● TERMINADO en el maestro	USDJPY · H1 USDJPY Estrategias 396 Plantilla sin plantilla registrada ● TERMINADO en el maestro	XAUUSD · H1 XAUUSD Estrategias 1 679 Plantilla sin plantilla registrada ● TERMINADO en el maestro
CADJPY · M30 CADJPY_H1 Estrategias 0 Plantilla sin plantilla registrada ● CONSTRUIDO en el maestro	USDJPY · M30 Test_USDJPY_donchianUpperCrossUp_M30 Estrategias 0 Plantilla donchianUpperCrossUp ● CONSTRUIDO en el custodio	XAUUSD · H1 XAUUSD_Breakout_H1 Estrategias 0 Plantilla sin plantilla registrada ● CONSTRUIDO en el maestro

La galería pone tantas tarjetas por fila como caben (de una a cuatro): en una ventana de 1440 de ancho son tres, en una de 1680, cuatro. Nunca aparece barra de scroll horizontal.

Cada tarjeta, de arriba abajo:

- **Símbolo · timeframe**, en grande. Salen de `registry.csv` cuando el proyecto lo creó el builder; si no, del gráfico principal de la primera tarea del `project.cfx`. Por eso `EURJPY_H1` sale **H4**: su primera tarea está configurada en H4, diga lo que diga el nombre.
- **El nombre del proyecto**, en gris.
- **Estrategias**: estrategias **distintas** (por identidad) sumando todas sus databanks. Puede ser menos que los ficheros `.sqx` si hay copias idénticas (XAUUSD: 1 805 ficheros, 1 679 estrategias).
- **Plantilla**: la de `registry.csv`; «sin plantilla registrada» en los proyectos anteriores al registro.
- **El estado**, con su color:

estado	color	qué significa	de dónde sale
CORRIENDO	violeta	SQX lo está corriendo ahora; la tarjeta añade cuántas estrategias lleva procesadas	el log de hoy lo arrancó y no lo ha terminado, y el worker está vivo (su <code>action=status</code>)
PARADO	rojo	se paró antes de acabar	el log de hoy dice «Project stopped», o lo dejó a medias y el worker ya no está
TERMINADO	verde	la última corrida acabó	el log de hoy dice «Project finished»; o, si hoy no corrió, sus databanks tienen estrategias
CONSTRUIDO	ámbar	existe, pero ninguna tarea ha dejado estrategias	ninguna databank tiene <code>.sqx</code>
AUSENTE	gris	ya no está en ningún install, pero quedan sus informes	solo existe <code>AlgoData/reports/<proyecto>/</code>

- **En el maestro / conductor / custodio:** el install donde vive. Si está en un worker y también en el maestro (una copia que el dueño hizo para mirarla), manda la del worker.

Pasa el ratón por una tarjeta: el tooltip dice cómo se supo el estado («El log de hoy dice «Project finished».») y para qué se creó el proyecto (la columna `purpose` del registro).

En **Estrategia**, la barra de arriba dice `Proyectos > USDJPY > Strategy 1.12.28` : cada miga abre su zona. La identidad no se escribe nunca en pantalla: está en el tooltip de la miga de la estrategia. La ficha que se abre ahí (curvas, estadísticas IS/OOS1/OOS2, metadatos y los estudios) tiene su propio capítulo, el 68, con sus capturas.

Un ejemplo completo

El 27-09-2026, con el custodio y el maestro con proyectos:

```
$ curl -s 127.0.0.1:8765/api/projects/all | python3 -m json.tool | head -40
{
  "projects": [
    {
      "name": "AUDJPY",
      "kind": "legacy",
      "purpose": "",
      "template": null,
      "reports": false,
      "symbol": "AUDJPY",
      "timeframe": "H1",
      "strategies": 2341,
      "databanks": {
        "Existing portfolio": 0,
        ...
        "OOS": 726,
        "Results": 1595,
        "RetestMarkets - Structural": 20,
        ...
      },
      "state": "terminado",
      "why": "No corre hoy y sus databanks guardan estrategias.",
      "live": null,
      "install": "master",
      "install_label": "maestro"
    },
  ],
}
```

AUDJPY no tiene carpeta en AlgoData/reports/ ("reports": false) y aparece igual: antes no salía en ninguna lista. En la ventana, clic en la tarjeta **USDJPY · H1** → la caja pregunta por SPP OOS, WFM y WFM Stability (las tres con estrategias) → «Sí» → la línea de estado dice «USDJPY: carga pedida para 3 de 3 databanks», la databank elegida es SPP OOS y se abre Proyecto. Doble clic en una fila → Estrategia con Strategy 1.12.28 de la databank WFM.

Qué NO te dice

- **«Terminado» sin log de hoy es una deducción.** SQX no deja un log por proyecto; si el proyecto no corrió hoy, la galería solo sabe que sus databanks tienen estrategias. Pudo pararse otro día.
- **El total no dice nada de calidad:** son estrategias guardadas, no supervivientes. El embudo de Proyecto dice cuántas pasan cada criba.
- **Un proyecto corriendo se cuenta por ficheros,** no por identidad: mientras SQX escribe, un .sqx puede estar a medias y no se abre.

Si algo falla

- **La galería se queda en «Leyendo los proyectos de los installs...» más de medio minuto:** el demonio no responde. Ciérrala y ábrela, o arranca el demonio a mano (python3 -m ui.daemon.serve) para ver su error.
- **«El demonio no respondió: ...» bajo el título:** el demonio se cayó o no está levantado.
- **La línea de estado dice «el demonio no aceptó ...» tras un clic:** esas databanks no se pudieron pedir; elígelas a mano y usa el ↻ de la barra de carga.

- **«ninguna databank de guarda esa estrategia»:** la estrategia se borró o se movió de databank desde que se pintó el panel; recarga el proyecto.

65. Proyecto — el raíl del workflow: mapa, botones de correr y estado

Qué pregunta responde

- **¿En qué paso está este proyecto?** Arriba de la zona Proyecto hay una tarjeta por cada fila de `WORKFLOW.md` (los 25 pasos más los tres medios: 10.5, 16.5 y 18.5, **28 tarjetas**). Cada una dice su estado con una palabra y un color: **hecho** (verde), **en marcha** (violeta), **pendiente** (gris), **bloqueado** (rojo), **sellado** (ámbar: 17, 18 o 19 terminados cuyo resultado no se enseña hasta que el paso 20 se abra) y **sin dato**.
- **¿Dónde miro su resultado?** Cada tarjeta dice a qué pestaña del panel de databanks va («→ Puerta IS/OOS», «→ Cierre > Stop ATR»). Al hacer clic en la tarjeta, el panel de abajo abre esa pestaña y, debajo del raíl, se despliega el **cajón** con las pruebas de ese paso.
- **¿Qué puedo correr desde aquí, y con qué configuración?** En el cajón, cada prueba (cada estudio de Python del paso) tiene su casilla, su estado, su configuración en una línea, el botón «configuración» que despliega todos sus ajustes con su explicación, y su ►.
- **¿Cuándo puedo leer el paso 20?** Una línea ámbar arriba lo dice: «SELLADO — el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19] ... (en el estudio USDJPY_M30_donchianUpperCrossUp)». La puerta mira **un solo estudio del ledger**: el de símbolo, timeframe y plantilla del proyecto, el mismo que lee el análisis del paso 20. Filas apuntadas bajo otro nombre (por ejemplo, pruebas con el nombre del proyecto como familia) no la abren. Mientras falte uno, la tarjeta 20 está **bloqueada** y su prueba no se puede lanzar.

Cuándo lo usas, y cuándo no

Lo usas para ver de un vistazo qué le falta a un proyecto y para lanzar sus análisis de Python sin escribir comandos: los estudios se reparten solos en paralelo (un trabajo por carril y, dentro, una estrategia por núcleo físico).

No sirve para lanzar nada en StrategyQuant X. Los pasos de SQX (build, retests, MC Retest, SPP, variantes del 16.5, WFM del 19...) salen con su estado, leído del disco, pero **no tienen botón en el raíl**. La tarea siguiente de SQX se lanza con «Continuar workflow», en la cabecera del panel de databanks, después de filtrar y tras una pantalla de confirmación (capítulo 69). Tampoco se lanza desde el raíl el análisis que cuelga de un paso de SQX (la nube de parámetros del 16.5, el análisis del WFM del 19): se lee después de la tarea y se corre desde el panel de databanks. Si alguien lo pide al demonio, se niega con esa frase.

No sirve para los pasos 21 a 25 desde el raíl: leen las supervivientes y el raíl no sabe en qué databank están. Se corren desde el panel de databanks, con el databank que tengas abierto (su botón «correr los marcados de este panel»).

Antes de empezar

- La ventana abierta (`bin/algoui`) y el proyecto elegido en la galería.
- Lo que cada estudio lee tiene que existir en `AlgoData`: la cosecha, el export de operaciones, el lote de variantes... Si falta, el botón no falla: dice qué falta y con qué skill se consigue.
- Para los estudios que apuntan su mirada en el ledger (SPA/StepM del paso 8, WFC, CSCV, superficies por mercado y el paso 20), el proyecto tiene que tener **plantilla** en `AlgoData/projects/registry.csv`: la familia del ledger es el nombre de esa plantilla. Sin plantilla, esas pruebas salen **bloqueadas** con la frase «el proyecto ... no tiene plantilla en projects/registry.csv».

Cómo se ejecuta

No hay comando: son botones.

botón	dónde	qué corre
casilla de la tarjeta	cada tarjeta	marca o desmarca todas las pruebas del paso que se pueden correr
► de la tarjeta	pasos de Python	las pruebas marcadas de ese paso
casilla y ► de una prueba	cajón	solo esa prueba
► correr los marcados de «pestaña»	cajón, y la cabecera del panel de databanks	las pruebas marcadas de todos los pasos cuyo resultado vive en esa pestaña
► correr marcados (N)	cabecera del raíl	todas las pruebas marcadas del proyecto
►► correr todo	cabecera del raíl	toda prueba gratuita de Python que aún no tiene resultado, marcada o no (nunca las que gastan oos2 o escriben en el ledger, ni 21-25)
Rehacer las filas de 17-19	cabecera, solo cuando hace falta	<code>python3 -m ledger.backfill --blind <proyecto> ... --write</code>

Por defecto salen marcadas solo las pruebas **gratuitas** pendientes de los pasos pendientes. **Nunca salen marcadas, ni entran en «correr todo»**: las que gastan una mirada al oos2 o escriben una fila en el ledger (WFC, CSCV, superficies por mercado, el análisis del WFM, el SPA/StepM del paso 8 y el paso 20), ni las de los pasos 21-25 (el raíl no sabe en qué databank están sus supervivientes). Esas se marcan a mano o se lanzan con su ►, y antes de correr la ventana **pregunta** con una ventanita que dice qué gasta cada una («gasta una mirada al oos2 y escribe su fila en el ledger»); si dices que no, no se lanza nada. En el cajón llevan delante el aviso «△ ...» en mayúsculas.

Cada prueba decide sola en qué databank corre: el de su último resultado; si no tiene, el databank que produjo el paso de SQX anterior (para el 8, el del retest OOS y luego el del build). Un estudio de población corre una vez; uno que lee una sola estrategia corre una vez por estrategia del databank, todas en cola a la vez. WFC, CSCV y superficies corren una vez por madre con lote de variantes.

La cola deja arrancar trabajos mientras quedan núcleos y **al menos 20 GB de RAM libre**, la reserva de Python del reparto del dueño; lo demás espera su turno en el orden pulsado.

Ningún botón del raíl toca SQX. Tarda lo que tarde cada estudio (un `profitShape` de una estrategia, 1-2 s).

Qué produce

Lo mismo que el estudio lanzado a mano: su carpeta en

`AlgoData/reports/<proyecto>/<databank>/<día>/<estudio>/` (o en `strategyPermutations/<proyecto>/<madre>/estudios/` para WFC, CSCV y superficies) y el log del trabajo en `AlgoData/logs/ui/`. «Rehacer las filas de 17-19» **añade** filas al ledger del estudio (símbolo × timeframe × familia); no reescribe ninguna, y si ya existen se niega.

Cómo se lee el resultado

El raíl del proyecto `Test_USDJPY_donchianUpperCrossUp_M30`, con el paso 8 abierto en el cajón:

WORKFLOW 12 hecho 13 pendiente 1 bloqueado 2 sellado ▶ correr marcados (0) ▶▶ correr todo

0052 de USDJPY: 3 miradas(s) de este proyecto (paso 17; 2, paso 18: 1); 12 en 3 estudio(s) de USDJPY. Reservado para los pasos 17, 18, 18.5, 19, 24. La política no fija un número de miradas: cada una lo gasta SELLADO – El paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

paso 20 bloqueado: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

1 Idea en el chat → Sin panel HECHO	2 Vocabulario → Sin panel HECHO	3 Plantilla → Sin panel HECHO	4 Preflight → Sin panel HECHO	5 Custom project → Sin panel HECHO	6 Build → Puerta IS/OOS PENDIENTE	7 Retest OOS → Puerta IS/OOS PENDIENTE	8 Análisis IS/OOS → Puerta IS/OOS 14 pruebas HECHO	9 Retest crossmarkets → Cross Market PENDIENTE	10 Análisis crossmarkets → Cross Market nulls.draws=25000 HECHO
10.5 Preparación crossTF → Cross Timeframe PENDIENTE	11 Retest crossTF → Cross Timeframe PENDIENTE	12 Análisis crossTF → Cross Timeframe run.blocks=None u HECHO	13 MC Retest → MC Retest PENDIENTE	14 Análisis MC Retest → MC Retest ingest.capital=1000 HECHO	15 SPP → SPP PENDIENTE	16 Análisis SPP → SPP read_metric=ReturnD HECHO	16.5 Variantes para el WFC → WFC run.symbol=XAUUSD · spl HECHO	17 Walk Forward Correlation → WFC min.trades=30 · spl SELLADO	18 CSGV → CSGV min.trades=30 · spl SELLADO
18.5 Superficies por mercado → Market Surfaces min.trades=30 · seg PENDIENTE	19 Walk Forward Matrix → WFM read_metric=ReturnD PENDIENTE	20 Análisis conjunto (ciego) → Cierre · Análisis benchmark.sizing=req BLOQUEADO	21 Exposición vs buy & hold → Cierre · Exposici study.sample=OOS1 PENDIENTE	22 Mapa condicional → Cierre · Mapa con run.symbol=USDJPY · PENDIENTE	23 Estructura → Cierre · Estructu subjects.draws=10000 HECHO	24 Stop loss ATR → Cierre · Stop ATR stop_atr_period=20 PENDIENTE	25 Edge por coste, por estrategia → Cierre · Edge por verdict_min_edge_sp PENDIENTE		

PASO 8 · Análisis IS/OOS – HECHO

Gate sobre Results del 2026-09-27. También: edgeCost, spread.

```

Puerta IS/OOS      hecho presencia.kind=hard · sanidad.kind=hard · sanidad.min_trades=20 (+19) · Último resultado del 2026-09-27 sobre Results
Edge por coste     hecho verdict.min_edge_spreads=2.0 · verdict.action=mark · Último resultado del 2026-09-28 sobre Results
SPA y StepM contra buy & hold pendiente Δ ESCRIBE UNA FILA EN EL LEDGER (UNA BÚSQUEDA MAS DE ESTE ESTUDIO) · study.segment=oos1 · benchmark.sizing=equal risk · stepm.fwer=0.05 (+2)
Calidad del feed   pendiente scale.window_weeks=52 · scale.min_weeks=13 · scale.floor_ticks=3 (+53) · Sin resultado para este proyecto
Spread real de Darwinex hecho assets.XAUUSD.ticks=XAUUSD_DarwTick_Infinox · assets.XAUUSD.bars=XAUUSD_DukasM1_Infinox · assets.USDJPY.ticks=USDJPY_DarwTick_theSers (+85)
Panel IS/OOS        hecho sin configuración · Último resultado del 2026-09-27 sobre OOS
Filtros candidatos pendiente sin configuración · Sin resultado para este proyecto
Replicación         bloqueado sin configuración · Compara varios databanks contra uno de referencia y la ventana corre uno: desde la terminal con --reference y --databa
Decaimiento IS→OOS pendiente sin configuración · Sin resultado para este proyecto
Exceso sobre el mono pendiente sin configuración · Sin resultado para este proyecto
Falsos positivos    bloqueado sin configuración · Aún no existe: sólo su README
Test del mono        pendiente nulls.draws=2500 · nulls.seed=20260922 · nulls.chunk_trades=200000 (+8) · Sin resultado para este proyecto
Forma del beneficio hecho run.sample=OOS1 · run.trim=[0, 5, 10, 20] · run.best_months=3 (+9) · Último resultado del 2026-09-27 sobre OOS
Calidad de la entrada pendiente run.sample=OOS1 · run.feed=XAUUSD_DukasM1_Infinox · run.timeframe=M30 (+8) · Sin resultado para este proyecto

```

▶ correr los marcados de «Puerta IS/OOS»

- La primera línea es el presupuesto de **OOS2**: cuántas veces lo ha mirado este proyecto y qué pasos lo tienen reservado. Cada mirada lo gasta.
- La segunda es la **puerta del paso 20**: ámbar mientras esté sellada, verde cuando 17, 18 y 19 estén los tres en el ledger.
- La tercera, en rojo, el motivo de cada paso bloqueado (un botón desactivado no enseña su ayuda, por eso va escrito).
- En cada tarjeta: número, SQX/PY/TÚ, título, la pestaña donde se lee su resultado, su configuración en una línea (o cuántas pruebas tiene) y el estado.

Esta vez no sale nada marcado: lo que queda pendiente o gasta oos2 o es del 21-25. Se marcan a mano dos pruebas del paso 8, «Edge por coste» y «Forma del beneficio», y se pulsa «►» correr los marcados de «Puerta IS/OOS»»:

WORKFLOW 12 hecho 13 pendiente 1 bloqueado 2 sellado ▶ correr marcados (2) ▶▶ correr todo

0052 de USDJPY: 3 miradas(s) de este proyecto (paso 17: 2, paso 18: 1); 12 en 3 estudio(s) de USDJPY. Reservado para los pasos 17, 18, 18.5, 19, 24. La política no fija un número de miradas: cada una lo gasta SELLAOO – El paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY M30 donchianUpperCrossUp)

paso 20 bloqueado: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY M30 donchianUpperCrossUp)

4 terminados, 0 en marcha

1	2	3	4	5	6	7	8	9	10
Idea en el chat – Sin panel HECHO	Vocabulario – Sin panel HECHO	Plantilla – Sin panel HECHO	Preflight – Sin panel HECHO	Custom project – Sin panel HECHO	Build – Puerta IS/OOS PENDIENTE	Retest OOS – Puerta IS/OOS PENDIENTE	Análisis IS/OOS – Puerta IS/OOS 14 pruebas HECHO	Retest crossmarkets – Cross Market PENDIENTE	Análisis crossmarkets – Cross Market nulls.draws=25000 HECHO
10.5	11	12	13	14	15	16	16.5	17	18
Preparacion crossTF – Cross Timeframe PENDIENTE	Retest crossTF – Cross Timeframe PENDIENTE	Análisis crossTF – Cross Timeframe run.blocks=None · u HECHO	MC Retest – MC Retest PENDIENTE	Análisis MC Retest – MC Retest ingest.capital=1000 HECHO	SPP – SPP PENDIENTE	Análisis SPP – SPP read_metric=ReturnD HECHO	Variantes para el WFC – WFC run.symbol=XAUUSD · HECHO	Walk Forward Correlation – WFC min.trades=30 · spl SELLADO	CSCV – CSCV min.trades=30 · spl SELLADO
18.5	19	20	21	22	23	24	25		
Superficies por mercado – Market Surfaces min.trades=30 · seg PENDIENTE	Walk Forward Matrix – WFM read_metric=ReturnD PENDIENTE	Análisis conjunto (ciego) – Cierre · Análisis benchmark.sizing=req BLOQUEADO	Exposición vs buy & hold – Cierre · Exposici study.sample=0051 PENDIENTE	Mapa condicional – Cierre · Mapa con run.symbol=USDJPY · PENDIENTE	Estructura – Cierre · Estructu subsets.draws=10000 HECHO	Stop loss ATR – Cierre · Stop ATR stop_atr_period=20 PENDIENTE	Edge por coste, por estrategia – Cierre · Edge por verdict_min_edge_sp PENDIENTE		

PASO 8 - Análisis IS/OOS – HECHO

Gate sobre Results del 2026-09-27. También: edgeCost, spread.

Puerta IS/OOS **hecho** presencia.kind=hard · sanidad.kind=hard · sanidad.min_trades=20 (+19) · Último resultado del 2026-09-27 sobre Results

✓ **Edge por coste** **hecho** verdict.min_edge_spreads=2.0 · verdict.action=mark · Último resultado del 2026-09-28 sobre Results

SPA y StepM contra buy & hold **pendiente** Δ ESCRIBE UNA FILA EN EL LEDGER (UNA BÚSQUEDA MAS DE ESTE ESTUDIO) · study.segment=oos1 · benchmark.sizing=equal risk · stepm.fwer=0.05 (-4)

Calidad del feed **pendiente** scale.window.weeks=52 · scale.min.weeks=13 · scale.floor.ticks=3 (+53) · Sin resultado para este proyecto

Spread real de Darwinex **hecho** assets.XAUUSD.ticks=XAUUSD_DarwinTick_Infinox · assets.XAUUSD.bars=XAUUSD_DukasM1_Infinox · assets.USDJPY.ticks=USDJPY_DarwinTick_theSers (+85)

Panal IS/OOS **hecho** sin configuración · Último resultado del 2026-09-27 sobre OOS

Filtros candidatos **pendiente** sin configuración · Sin resultado para este proyecto

Replicación **bloqueado** sin configuración · Compara varios databanks contra uno de referencia y la ventana corre uno: desde la terminal con --reference y --databar

Decaimiento IS-OOS **pendiente** sin configuración · Sin resultado para este proyecto

Falso sobre el mono **pendiente** sin configuración · Sin resultado para este proyecto

Falsos positivos **bloqueado** sin configuración · Aún no existe; sólo su READNE

Test del mono **pendiente** nulls.draws=2500 · nulls.seed=20260922 · nulls.chunk_trades=200000 (+8) · Sin resultado para este proyecto

✓ **Forma del beneficio** **hecho** run.sample=0051 · run.trim=[0, 5, 10, 20] · run.best_months=3 (+9) · Último resultado del 2026-09-27 sobre OOS

Calidad de la entrada **pendiente** run.sample=0051 · run.feed=XAUUSD_DukasM1_Infinox · run.timeframe=M30 (+8) · Sin resultado para este proyecto

WORKFLOW 12 hecho 13 pendiente 1 bloqueado 2 sellado

▶ correr marcados (2) ▶▶ correr todo

0052 de USDJPY: 3 mirada(s) de este proyecto (paso 17: 2, paso 18: 1); 12 en 3 estudio(s) de USDJPY. Reservado para los pasos 17, 18, 18.5, 19, 24. La política no fija un número de miradas: cada una lo gasta SELLADO – El paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

paso 20 bloqueado: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

4 terminados, 0 en marcha

1 Idea en el chat → Sin panel HECHO	2 Vocabulario → Sin panel HECHO	3 Plantilla → Sin panel HECHO	4 Preflight → Sin panel HECHO	5 Custom project → Sin panel HECHO	6 Build → Puerta IS/OOS PENDIENTE	7 Retest OOS → Puerta IS/OOS PENDIENTE	8 ▶ Análisis IS/OOS → Puerta IS/OOS 14 pruebas HECHO	9 Retest crossmarkets → Cross Market PENDIENTE	10 ▶ Análisis crossmarkets → Cross Market mults.draws=25000 HECHO
10.5 Preparación crossTF → Cross Timeframe PENDIENTE	11 Retest crossTF → Cross Timeframe PENDIENTE	12 ▶ Análisis crossTF → Cross Timeframe run.blocks=None · u HECHO	13 MC Retest → MC Retest PENDIENTE	14 ▶ Análisis MC Retest → MC Retest ingest.capital=1000 HECHO	15 SPP → SPP PENDIENTE	16 ▶ Análisis SPP → SPP read.metric=ReturnD HECHO	16.5 Variantes para el WFC → WFC run.symbol=XAUUSD · HECHO	17 ▶ Walk Forward Correlation → WFC min.trades=30 · spl SELLADO	18 ▶ CSCV → CSCV min.trades=30 · spl SELLADO
18.5 ▶ Superficies por mercado → Market Surfaces min.trades=30 · seg PENDIENTE	19 Walk Forward Matrix → WFM read.metric=ReturnD PENDIENTE	20 Análisis conjunto (ciego) → Cierre · Análisis benchmark.sizing=eq BLOQUEADO	21 ▶ Exposición vs buy & hold → Cierre · Exposici study.sample=0051 · PENDIENTE	22 ▶ Mapa condicional → Cierre · Mapa con run.symbol=USDJPY · PENDIENTE	23 Estructura → Cierre · Estructu subsets.draws=10000 HECHO	24 ▶ Stop loss ATR → Cierre · Stop ATR stop.atr.period=20 PENDIENTE	25 ▶ Edge por coste, por estrategia → Cierre · Edge por verdict.min_edge_sp PENDIENTE		

PASO 20 · Análisis conjunto (ciego) – BLOQUEADO

El paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

Lectura conjunta ciega bloqueado ▶ ABRE EL 0052 DEL PASO 20 Y ESCRIBE SU FILA EN EL LEDGER · benchmark.sizing=equal_risk · stepm.fwr=0.05 · joint.pieces=None (+3) · Bloqueado: configuración ▶

Con el proyecto USDJPY elegido, el mismo recorrido que las capturas. Lo que el demonio devolvió en `/api/jobs` medio segundo después de pulsar (los cuatro trabajos arrancados en el mismo segundo, sin cola):

```
20260928-070450-0 edgeCost 8 Results - rc= None queued= None
20260928-070450-1 profitShape 8 OOS Strategy 10.11.79 rc= None queued= None
20260928-070450-2 profitShape 8 OOS Strategy 9.25.72 rc= None queued= None
20260928-070450-3 profitShape 8 OOS Strategy 9.27.83 rc= None queued= None
```

Y dos segundos después, los cuatro con `rc= 0` . La línea del raíl: «4 terminados, 0 en marcha».

Qué NO te dice

- **«Hecho» no es «bueno».** Dice que hay un resultado en disco, no qué dice. El veredicto está en el panel de databanks.
- **El estado de un paso de SQX sale del disco**, no de preguntarle a SQX: el log del install y los `.sqx` de sus databanks. Si `/curate` ya quitó estrategias, las cuentas son las de después.
- **«Sellado» no dice nada del resultado a propósito.** Mirar el 17 y el 18 antes de correr el 19 contamina la decisión de correrlo.
- La configuración de una línea es la del `config.yaml` del estudio tal y como está hoy; si un resultado viejo se hizo con otra, el panel lo marca como desfasado, no el raíl.

Si algo falla

- **«no se lanzó: ... necesita la cosecha de este databank»:** falta la entrada; la frase dice con qué skill se hace (`/oos-gate` , `/export` , `/variants` ...).
- **«elige en el panel el databank que lee»:** es un paso del 21 al 25; lánzalo desde el panel.
- **«el proyecto ... no tiene plantilla en projects/registry.csv»:** el estudio firmaría en el ledger bajo una familia inventada; se corre a mano con `--family` si sabes cuál es.
- **«paso de SQX: la ventana no lo lanza desde el raíl...»:** el análisis cuelga de un paso de SQX; lánzalo desde el panel de databanks.
- **«No se lanzó nada.»:** respondiste «no» a la pregunta de una prueba que gasta oos2 o ledger.
- **«El demonio no respondió»:** la ventana perdió su demonio; ciérrala y vuelve a abrirla.

65b. Proyecto — el embudo y el panel de databanks

Qué pregunta responde

- **¿Cuántas estrategias quedan y dónde se cayeron las demás?** El **embudo**, en medio de la zona Proyecto, pinta una barra por criba de la Puerta IS/OOS (verde lo que pasa, rojo lo que cae, gris las cribas blandas que solo marcan) y, debajo, una por cada paso de Python posterior con su entrada y su salida. Al lado de cada barra, escrito, **por qué** caen.
- **¿Qué dijo cada prueba de cada estrategia?** El **panel de databanks**, abajo, al estilo de SQX: una fila por estrategia, sus métricas de SQX (IS y OOS) y una columna por prueba con su veredicto en color. Dos filas de pestañas: arriba la prueba (Puerta IS/OOS, Cross Market, Cross Timeframe, MC Retest, SPP, WFM, WFC, CSCV, Market Surfaces, Cierre), debajo sus sub-paneles (cada tarea del MC Retest, cada timeframe del cross-TF, cada composición del WFC...).
- **¿Cuánto gana el databank entero, con el spread de verdad?** A la derecha, la **equity agregada**: la suma diaria de todas las estrategias según SQX (violeta) y la misma con el spread y el slippage reales de Darwinex (ámbar), con la frontera IS/OOS.

Cuándo lo usas, y cuándo no

Lo usas para leer un proyecto entero de un vistazo: qué pasó cada criba, qué estrategias sobreviven y qué dijo de cada una cada prueba, sin abrir informes sueltos.

No sirve para mirar el WFC, el CSCV, el WFM ni el paso 20 antes de tiempo: mientras 17, 18 y 19 no estén los tres hechos, sus sub-paneles salen **BLOQUEADOS** con la frase del ledger, y sus columnas no aparecen en ninguna tabla. Es a propósito.

No hay pestaña de Monte Carlo de operaciones: irá con las carteras, en la zona **Portfolios** (decisión del dueño del 2026-09-23).

Antes de empezar

- La ventana abierta y el proyecto elegido en la galería.
- Las métricas salen de la cosecha del databank (paso 8, /oos-gate) o, si no la tiene, de su export de métricas, que la ventana hace sola al elegir el proyecto.
- La curva ámbar necesita el informe del estudio spread sobre ese databank (paso 8).

Cómo se ejecuta

No hay comando: son pestañas y botones.

control	qué hace
pestaña de arriba	elige la prueba; debajo aparecen sus sub-paneles
sub-pestaña	elige qué columnas se ven (y, en SPP, qué databank: IS u OOS)
clic en una cabecera	ordena por esa columna (los números como números)
doble clic en una fila	abre la ficha de esa estrategia en «Estrategia»
▼ Plegar / ► Desplegar	deja el panel en su cabecera y da la altura al embudo
arrastrar el borde de arriba del panel	cambia su altura
🔄 Recargar databank	vuelve a leer del disco qué estrategias hay en el databank y pide cargar lo que le falte (métricas; operaciones y cosecha van al conductor, de una en una)
► Correr marcados de este panel	corre las pruebas marcadas en el raíl cuyo resultado vive en esta pestaña, sobre este databank (y solo sobre las filas seleccionadas, si hay)
casillas SQX y spread y slippage reales	enseñan u ocultan cada curva
tira FILTROS y ►► Continuar workflow	filtran el databank y lanzan la tarea siguiente de SQX: capítulo 69

Qué produce

Mirar no escribe nada. «Recargar databank» encarga el mismo trabajo que la ventana encarga al elegir un proyecto. Lo que sí escribe —los filtros, en AlgoData/filters/ y en el Ledger, y «Continuar workflow», en SQX— está en el capítulo 69.

Cómo se lee el resultado

El embudo del proyecto Test_USDJPY_donchianUpperCrossUp_M30 :



La pestaña Puerta IS/OOS, sub-panel «Build + OOS1», ordenada por Beneficio neto OOS:

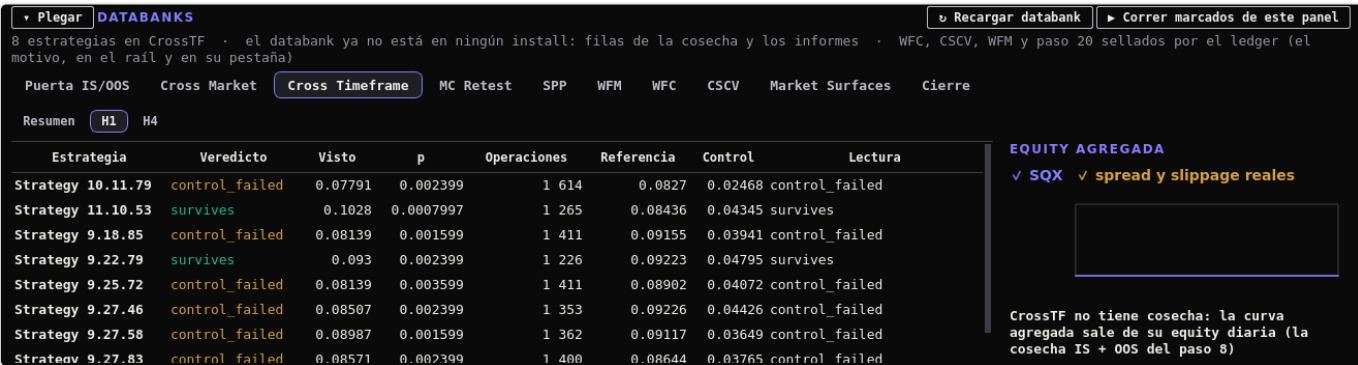
- Una fila por estrategia; las métricas de SQX en IS y luego en OOS; después una columna por prueba con su veredicto (verde pasa, rojo cae, ámbar dudoso). Los negativos, en rojo.
- Arriba, en gris: cuántas filas hay, de dónde salen y qué está sellado.
- A la derecha, la equity agregada: 17,6 M según SQX y 13,1 M con el spread y el slippage reales; el DD máximo pasa de -2.1 M a -3.2 M.



El MC Retest, sub-panel «Spread»: lo que hizo esa tarea en cada estrategia.



El cross-timeframe en H1: la lectura de cada estrategia en ese timeframe.



El WFC con 19 sin hacer: una sub-pestaña por composición que el ledger permite leer, todas bloqueadas con su motivo.



El paso 20 (Cierre > «Análisis conjunto (paso 20)») con 17, 18 y 19 sin hacer: la pestaña existe desde el principio, pero solo enseña el motivo, con la misma frase que el raíl, porque sale de la misma puerta del ledger. Un clic en la tarjeta 20 del raíl abre exactamente este sub-panel.

⌵ Plegar

DATABANKS

⌵ Recargar databank

▶ Correr marcados de este panel

Puerta IS/OOS

Cross Market

Cross Timeframe

MC Retest

SPP

WFM

WFC

CSCV

Market Surfaces

Cierre

Análisis conjunto (paso 20)

Exposición

Mapa condicional

Estructura

Stop ATR

Edge por coste

BLOQUEADO – el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

Este panel no se lee ni se pinta hasta entonces: ni tabla, ni curva, ni cifras.

La nube de parámetros del lote de variantes: solo sale la madre que tiene lote.

⌵ Plegar

DATABANKS

⌵ Recargar databank

▶ Correr marcados de este panel

1 estrategias en Results · el databank ya no está en ningún install: filas de la cosecha y los informes · WFC, CSCV, WFM y paso 20 sellados por el ledger (el motivo, en el raíl y en su pestaña)

Puerta IS/OOS

Cross Market

Cross Timeframe

MC Retest

SPP

WFM

WFC

CSCV

Market Surfaces

Cierre

IS Build → OOS OOS1

IS Build → OOS OOS2

IS Build → OOS OOS1 + OOS2

IS Build + OOS1 → OOS OOS2

Nube de parámetros

Estrategia	Beneficio neto IS	Operaciones IS	Factor de beneficio IS	Sharpe IS	Retorno / DD IS	DD máximo %
Strategy 9.27.83	106.5 K	1 280	1.32	1.07	11.07	

EQUITY AGREGADA · de todas las filas

✓ SQX ✓ spread y slippage reales



Neto SQX IS: 12.2 M Neto SQX OOS: 5.4 M
Neto SQX: 17.6 M DD máximo SQX: -2.1 M
Neto real: 13.1 M DD máximo real: -3.2 M
cosecha del 2026-09-27 · spread real del

Un ejemplo completo

Lo que el demonio devolvió para el databank Results de ese proyecto (200 filas, una por estrategia; 7 pruebas con columnas; los veredictos de la puerta y del MC Retest en la misma fila):

```
filas: 200 | origen filas: informes | métricas: cosecha
estudios: ['gate', 'isOos', 'crossTF', 'mcRetest', 'cloud', 'edgeCost', 'spread'] | sellado: ['wfc', 'cscv', 'wfm', 'blindJoint']
Strategy 17.28.58 67235.3 MANTENER None
con mcRetest: 8
sqx 4033 real 4033 split 2694
```

Pulsar ⌵ Recargar databank en Results , que ya no está en ningún install: la línea gris lo dice y las filas siguen siendo las de la cosecha. En un databank que un install sí tiene, dice cuántas estrategias hay ahora en disco y qué se ha puesto a cargar.

⌵ Plegar

DATABANKS

⌵ Recargar databank

▶ Correr marcados de este panel

Releído: Results no está en ningún install: las filas siguen siendo las de la cosecha y los informes

Puerta IS/OOS

Cross Market

Cross Timeframe

MC Retest

SPP

WFM

WFC

CSCV

Market Surfaces

Cierre

Build + OOS1

Cribas

Spread real

La misma prueba sobre una copia de cuatro estrategias: se borró una y se tocó la carpeta.

```
filas antes: 4 | recarga 1: 4 estrategias en disco
recarga 2 tras touch: 3 estrategias en disco | filas después: 3
```

Qué NO te dice

- **Un veredicto en la fila no es de este databank si la prueba corrió en otro:** el MC Retest y el cross-TF se emparejan con la puerta porque firman la misma identidad de la estrategia; el cross-market y el SPP no la firman y solo salen en su propio databank.
- **Las pruebas del lote de variantes** (nube, WFC, CSCV) se emparejan por el **nombre** de la madre, porque el lote no guarda su identidad.
- **«Origen filas: informes»** quiere decir que el databank ya no está en ningún install: las filas son las que recuerdan la cosecha y los informes. El doble clic abre una ficha que puede no encontrar la estrategia.
- **La curva agregada es la de la cosecha**, no la del databank de hoy: si `/curate` quitó estrategias después, la curva aún las cuenta.

Si algo falla

- **«X no tiene cosecha: la curva agregada sale de su equity diaria»:** ese databank no es el del build o no se cosechó; la tabla sí sale.
- **«ningún resultado de este estudio todavía»:** la prueba no ha corrido sobre ese databank; córrala desde el raíl.
- **«No se pudo leer ... de ...»:** un fichero que SQX o un estudio estaba escribiendo; pulsa  **Recargar databank**.
- **«paso de SQX: la ventana no lo lanza desde el raíl...»:** el análisis cuelga de un paso de SQX; lánzalo desde el panel de databanks.
- **«No se lanzó nada.»:** respondiste «no» a la pregunta de una prueba que gasta oos2 o ledger.
- **«El demonio no respondió»:** la ventana perdió su demonio; ciérrala y vuelve a abrirla.

69. Proyecto — filtros sobre un databank, y «Continuar workflow»

Qué pregunta responde

- **¿Qué queda si me quedo solo con las estrategias que cumplen esto?** Encima de la tabla del panel de databanks hay una tira **FILTROS**. Cada fila es una condición — métrica, operador y valor — y todas se combinan con **Y (AND)**: una estrategia se queda solo si cumple todas. Al pulsar ► **Aplicar**, la tabla esconde las que no pasan, la equity agregada de la derecha se recalcula **solo con las visibles** («183 visibles») y el embudo del medio gana una fila nueva («Filtro · Results 200 → 183»).
- **¿Y si quiero quitar a mano las que no me convencen?** Selecciona filas en la tabla y pulsa **Descartar seleccionadas**. Cuenta igual que un filtro.
- **¿Cuántas búsquedas llevo en este estudio?** Cada filtro aplicado y cada borrado a mano se apunta **en el Ledger** como una búsqueda más (qué criterio, de cuántas a cuántas, en qué paso y tramo), y se ve en **OPERACIÓN > Registro de búsquedas** (capítulo 70). Mirar los datos y quedarte con una parte es buscar, y el Ledger existe para contarlo.

Cuándo lo usas, y cuándo no

Lo usas para cribar una población a mano mientras la miras: de 200 a 183 con una regla, luego a 170 con otra, luego borrar tres que no te gustan. Los filtros **se apilan**: cada uno se aplica sobre lo que dejaron visible los anteriores.

No borra nada en StrategyQuant X. Filtrar es una vista en Python: el databank de SQX sigue con todas sus estrategias (sus `.sqx` no se tocan) hasta que pulses «Continuar workflow», que es lo que de verdad las quita (segunda parte de este capítulo).

No se puede filtrar por métricas de **OOS2**: no aparecen en el desplegable. Tampoco los estudios que pueden leerlo: WFC, CSCV, superficies por mercado, WFM, paso 20, stop ATR y exposición (la lista sale de `assets/_policy.yaml`, a quién reserva el oos2 del activo, más esos hijos). Los databanks del WFC y del WFM se niegan enteros.

Las filas **sin identidad** (estrategias que solo nombra un informe antiguo) se ven en la tabla, pero ningún filtro las puede esconder: la tira y el embudo las cuentan aparte, «N sin identidad, no filtrables».

Antes de empezar

- La ventana abierta, en **Proyecto**, con un databank en pantalla que tenga tabla (no un panel bloqueado).
- El proyecto tiene que tener **plantilla** en `AlgoData/projects/registry.csv`: el Ledger apunta cada búsqueda en el estudio `<símbolo>_<timeframe>_<plantilla>`. Sin plantilla no hay dónde apuntar, y el filtro **no se aplica** (la tira lo dice).
- Nada de SQX: ni hace falta el install arrancado ni se le envía ningún comando.

Cómo se ejecuta

No hay comando: es un botón. Paso a paso:

1. En la fila vacía, elige la **métrica**. El desplegable lista las columnas que ese databank tiene de verdad (las de SQX con su tramo, «Net profit (OOS)», y las de los estudios ya corridos, «Spread.operaciones [OOS]»), y al final las **distribuciones** guardadas («Distribución · Panel IS/OOS · Retorno por operación [USD/lote]»). Al pasar el ratón por una métrica dice cuántas estrategias tienen valor.

2. Elige el **operador**: `>`, `<`, `≥`, `≤`, `=`, `entre` (dos valores, «desde» y «hasta»). Una columna de texto (un veredicto) solo admite `=`. Una distribución admite «mediana dentro del intervalo» y «mediana fuera del intervalo».
3. Escribe el **valor**. Para una distribución, el valor es el intervalo en %: **50, 80, 90 o 98**.
4. **+ condición** añade otra fila; **×** la quita.
5. ► **Aplicar**.

botón	qué hace
► Aplicar	aplica todas las filas a la vez (Y) a lo que se ve, apunta una fila en el Ledger y esconde lo que no pasa
Quitar filtros	vuelve a enseñar todo. Lo apuntado en el Ledger se queda : lo que se miró, se miró
Descartar seleccionadas	esconde las filas seleccionadas y lo apunta en el Ledger como borrado a mano
Guardar con nombre...	guarda las filas de la pantalla con un nombre, para reutilizarlas
Cargar	pone en las filas el filtro guardado elegido en el desplegable; no lo aplica

Tarda medio segundo en un databank de 200 estrategias (la primera vez que se abre un proyecto, unos 2 s en leer sus distribuciones).

El modo distribución, en concreto:

- Si la distribución tiene **dos muestras** (el panel IS/OOS: la de IS y la de OOS superpuestas), «mediana dentro del intervalo 90» significa: **la mediana de OOS cae dentro del 90 % central de IS** (entre su percentil 5 y su 95).
- Si tiene **una** (las re-ejecuciones del MC Retest), significa: **el valor real** (el backtest que ocurrió, la línea vertical) cae dentro del 90 % central de las re-ejecuciones.

Qué produce

ruta	qué es
AlgoData/filters/<proyecto>/<databank>/discards.jsonl	la lista de descartes: una línea por filtro o borrado (criterio, de cuántas a cuántas, hora) y una por estrategia escondida (su identidad, su nombre, si fue filtro o a mano). Solo se añade: «Quitar filtros» escribe una línea de corte y lo anterior queda como historia. Sobrevive a reiniciar la ventana
AlgoData/filters/saved.yaml	los filtros guardados con nombre
AlgoData/ledger/<símbolo>_<timeframe>_<plantilla>.jsonl	una línea más por filtro o borrado: paso, tramo, criterio, <code>n_in</code> , <code>n_out</code> , lanzado por «ventana»

El paso es el de Python que lee ese databank (el databank `Results`, del build, es el **paso 8**, «Análisis IS/OOS»). El tramo es `build` si el filtro solo mira columnas IS, y `oos1` si mira algo fuera de muestra (una columna OOS, un estudio, una distribución). Un borrado a mano es `oos1`.

Cómo se lee el resultado

Test_USDJPY_donchianUpperCrossUp_M30 · plantilla · 0 estrategias · sin ficha de la galería todavía

WORKFLOW

12 hecho 13 pendiente 1 bloqueado 2 sellado

0052 de USDJPY: 3 mirada(s) de este proyecto (paso 17: 2, paso 18: 1); 12 en 3 estudio(s) de USDJPY. Reservado para los pasos 17, 18, 18.5, 19, 24. La política no fija un número de miradas: cada una lo gasta

SELLADO — El paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

paso 20 bloqueado: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala (en el estudio USDJPY_M30_donchianUpperCrossUp)

1

Idea en el chat

→ Sin panel

HECHO

2

Vocabulario

→ Sin panel

HECHO

3

Plantilla

→ Sin panel

HECHO

4

Preflight

→ Sin panel

HECHO

5

Custom project

→ Sin panel

HECHO

6

Build

→ Puerta IS/OOS

PENDIENTE

7

Retest OOS

→ Puerta IS/OOS

PENDIENTE

8

Análisis IS/OOS

→ Puerta IS/OOS

14 pruebas

HECHO

9

Retest crossmarkets

→ Cross Market

PENDIENTE

10

Análisis crossmarkets

→ Cross Market

nulls.draws=25000

HECHO

10.5

Preparación crossTF

→ Cross Timeframe

PENDIENTE

11

Retest crossTF

→ Cross Timeframe

PENDIENTE

12

Análisis crossTF

→ Cross Timeframe

run.blocks=None · u

HECHO

13

MC Retest

→ MC Retest

PENDIENTE

14

Análisis MC Retest

→ MC Retest

ingest.capital=1000

HECHO

15

SPP

→ SPP

PENDIENTE

16

Análisis SPP

→ SPP

read.metric=ReturnD

HECHO

16.5

Variantes para el WFC

→ WFC

run.symbol=XAUUSD ·

HECHO

17

Walk Forward

→ WFC

min.trades=30 · spl

SELLADO

18

CSCV

→ CSCV

min.trades=30 · spl

SELLADO

18.5

Superficies por

→ Market Surfaces

min.trades=30 · seg

PENDIENTE

19

Walk Forward Matrix

→ WFM

read.metric=ReturnD

PENDIENTE

20

Análisis conjunto

→ Cierre · Análisis

benchmark.sizing=eq

BLOQUEADO

21

Exposición vs buy &

→ Cierre · Exposici

study.sample=0051

PENDIENTE

22

Mapa condicional

→ Cierre · Mapa con

run.symbol=USDJPY ·

PENDIENTE

23

Estructura

→ Cierre · Estructu

subsets.draws=10000

HECHO

24

Stop loss ATR

→ Cierre · Stop ATR

stop.atr.period=20

PENDIENTE

25

Edge por coste, por

→ Cierre · Edge por

verdict.min_edge_sp

PENDIENTE

EMBUDO DE LA POBLACION

Cuántas estrategias entran en cada criba, cuántas pasan y por qué caen las que caen. Las barras van a escala de la población del build; cada paso posterior lee su propio databank. Puerta: Results del 2026-09-27.

Build

Presencia

Sanidad

Fetátrac

200 → 200

200 → 200

200 → 200

200 → 183

la población del build que el gate juzgó

si SOX la tiro del databank de retesteo —demasiados trades ambiguos, o cualquier otra de sus ban...

una estrategia con cuatro trades fuera de muestra no tiene error estandar que leer, y una que repi...

el cual de métricas estáticas sobre el periodo fuera de muestra. I avn a nnnnnnn: hnn solo nido n

Plegar

DATABANKS

FILTROS

+ condición

► Aplicar

Quitar filtros

Descartar seleccionadas

Net profit (00S)

>

0

×

Spread.operaciones [00S]

≥

30

×

Results: 200 → 183 visibles · 17 descartadas · 1 filtro(s) o borrado(s)

200 estrategias en Results el databank ya no está en ningún install: filas de la cosecha y los informes · WFC, CSCV, WFM y paso 20 sellados por el ledger (el motivo, en el rail y en su pestaña)

Puerta IS/OOS

Cross Market

Cross Timeframe

MC Retest

SPP

WFM

WFC

CSCV

Market Surfaces

Cierre

Build + 0051

Cribas

Spread real

Estrategia	Beneficio neto IS	Operaciones IS	Factor de beneficio IS	Sharpe IS	Retorno / DD IS	DD máximo % IS	% ganadoras IS	Beneficio neto 00S
Strategy 1.1.85	50 791	1 066	1.18	0.6	3.94	9.07	50.66	38 90
Strategy 1.15.54	40 048	610	1.22	0.55	3.98	7.05	54.02	46 11
Strategy 1.19.50	46 622	1 089	1.14	0.49	2.86	12.32	51.98	45 49
Strategy 1.2.86	65 009	958	1.24	0.75	4.19	8.65	51.93	40 50
Strategy 1.4.63	39 728	1 501	1.1	0.4	2.62	11.45	50.2	57 05
Strategy 1.6.73	65 857	1 781	1.13	0.56	3.53	11.63	51.07	47 55
Strategy 1.8.60	51 277	689	1.25	0.66	5.54	7.26	52.69	9 61
Strategy 10.10.49	73 770	621	1.52	1.17	12.51	3.83	56.04	159.
Strategy 10.10.78	71 968	616	1.52	1.15	12.2	3.81	56.49	1 44
Strategy 10.11.49	70 567	611	1.51	1.12	11.97	3.81	56.14	2 34
Strategy 10.11.79	110.4 K	1 357	1.32	1.1	9.83	7.65	54.35	25 36

EQUITY AGREGADA · 183 visibles

✓ SOX ✓ spread y slippage reales

17.8 M

-677.6 K

IS < → OOS1

Neto SOX IS: 11.1 M Neto SOX OOS: 5.5 M

Neto SOX: 16.7 M DD máximo SOX: -1.9 M

Neto real: 12.4 M DD máximo real: -2.9 M

cosecha del 2026-09-27 · spread real del 2026-09-27

La línea bajo las filas dice cómo queda el databank: «Results: 200 → 183 visibles · 17 descartadas». Al aplicar dice además dónde se apuntó: «200 → 183 · apuntado en el Ledger: USDJPY_M30_donchianUpperCrossUp, paso 8, oos1». Una estrategia **sin valor** en una métrica (una que el MC Retest no probó, por ejemplo) **no se esconde**: esa condición no la juzga y sigue visible. Se cuenta aparte, entre paréntesis y en el embudo: «(191 sin valor, siguen visibles)». Ejemplo real: «MC Retest · Parámetros fuera del intervalo 90 Y Factor de beneficio IS entre 1,1 y 3» sobre las 200 de Results → 200 → 198, 2 escondidas y 191 sin valor (solo 8 pasaron por el MC Retest).

FILTROS

+ condición

► Aplicar

Quitar filtros

Descartar seleccionadas

Net profit (00S)

>

0

×

Spread.operaciones [00S]

≥

30

×

Results: 200 → 183 visibles · 17 descartadas · 1 filtro(s) o borrado(s)

En el embudo, cada filtro es una fila más al final, con su criterio como explicación:



Un ejemplo completo

Proyecto `Test_USDJPY_donchianUpperCrossUp_M30`, databank `Results` (200 estrategias). Filtro: **Net profit (OOS) > 0 Y Spread.operaciones [OOS] ≥ 30** (la cosecha no trae la columna «# of trades (OOS)» de `SQX`, solo la de `IS`; hasta que la traiga, las operaciones fuera de muestra las cuenta el estudio de spread, `Spread.operaciones [OOS]`).

Respuesta del demonio al aplicar (2026-09-28):

```
n_in 200 · n_out 183 · blank 0
expression: Net profit (OOS) > 0.0 AND spread.operaciones [OOS] ≥ 30.0
ledger: USDJPY_M30_donchianUpperCrossUp · paso 8 · oos1
```

Y en el Ledger (`python3 -m ledger.report --study USDJPY_M30_donchianUpperCrossUp`):

```
USDJPY_M30_donchianUpperCrossUp · 6 búsquedas
-- el embudo, paso a paso

          ts  step  criterion                                     segment
n_in  n_out  kept
2026-09-27T20:30:36+00:00  18.5  marketSurfaces: 10 mercados (...) x 150 variantes, NetProfit...  build
150      150  1.000
...
2026-09-28T07:18:12+00:00   8.0  Net profit (OOS) > 0.0 AND spread.operaciones [OOS] ≥ 30.0  oos1
200      183  0.915
2026-09-28T07:24:02+00:00   8.0  snoopingScreen/stepm                                           oos1
200      200  1.000
de 150 entraron a 200 supervivientes
```

(La fila del filtro es la de las 07:18:12. La de las 07:24 es otra búsqueda del mismo paso 8, el estudio `snoopingScreen`, lanzada desde el raíl.) Ojo con la última línea: el informe resume «de la primera fila a la última» por hora, y las filas del paso 8 se apuntaron después que las del 18.5. Lee la tabla, no ese resumen. Desde el 2026-09-28 los valores de las condiciones llegan al Ledger como números (`30` , no `"30"`), aunque en la ventana se escriban como texto.

La tabla pasa de 200 a 183 filas visibles; tras cerrar y abrir la ventana (o reiniciar el demonio) siguen escondidas las mismas 17.

Qué NO te dice

- **Que las 183 sean buenas.** Un filtro elegido mirando el OOS **gasta el OOS**: cuantos más pruebas, más fácil es que las supervivientes lo sean por suerte. Por eso cada uno se apunta en el Ledger, y el informe del Ledger descuenta el Sharpe por todas las búsquedas.
- **Que SQX las haya borrado.** Siguen todas en el databank de SQX hasta «Continuar workflow».
- **Nada con O (OR).** Solo Y; para un O, aplica un filtro, quítalo y aplica el otro — y los dos quedan apuntados.

Si algo falla

- «el proyecto ... no tiene plantilla en projects/registry.csv...»: sin plantilla no hay estudio del Ledger; no se aplica.
- «ninguna tarea de ... escribe ...»: el `project.cfx` del install ya no dice qué tarea escribe ese databank (o ningún install tiene el proyecto): no se sabe su paso, no se apunta, no se aplica.
- ««X» no es una columna de este databank (o es de OOS2...)»: la métrica no existe ahí o está prohibida. Mira el desplegable: solo lista las que hay.
- «WFC_OOS2 es de la etapa wfc, que lee OOS2: no se filtra»: ese databank no se filtra.

«Continuar workflow» — borrar de verdad los descartes y lanzar la tarea siguiente

Qué pregunta responde

Ninguna: **ejecuta lo que ya decidiste** con los filtros. Los filtros solo esconden; este botón es el que **borra en StrategyQuant X** las estrategias descartadas de ese databank y, a continuación, **lanza la tarea siguiente** del proyecto, para que esa tarea lea solo las supervivientes. Es el único botón de la ventana que cambia lo que hay dentro de SQX.

Cuándo lo usas, y cuándo no

Lo usas cuando has terminado de filtrar un databank y quieres seguir el workflow: por ejemplo, cribaste `005` en el paso 8 y quieres que el retest de cross markets (paso 9) corra solo con lo que queda.

No lo usas para probar un filtro: para eso están los filtros, que no tocan nada. Y **no** sirve para un proyecto del maestro: la ventana nunca toca tu install (`SQX`), solo los workers (conductor `SQX_w1` , custodio `SQX_w2`).

Antes de empezar

El botón está **apagado** y dice al lado por qué, hasta que se cumplen todas estas condiciones (se comprueban en este orden):

1. El proyecto está en `AlgoData/projects/registry.csv` y vive en un **worker**. Si vive en el maestro se niega siempre, antes de mirar nada más.
2. **Nadie usa ese worker**: su puerto no responde y no hay ningún proceso de SQX trabajando desde su carpeta. Si el worker está arrancado, **se niega y no lo para** (parar un worker mata la ejecución de quien lo esté usando): páralo tú si sabes que está ocioso.
3. Ningún **otro** proyecto de ese worker se ha modificado en las últimas **24 horas** (puede ser de otra sesión).
4. El log de SQX de hoy de ese worker lleva **15 minutos** sin escribirse y no dice que un proyecto haya empezado sin terminar.
5. Hay **descartes** en ese databank (un filtro aplicado o estrategias descartadas a mano).
6. El databank tiene sus `.sqx` **en disco** en ese worker.

7. Existe una **tarea siguiente**: la primera tarea, en el orden del workflow después del paso que escribe ese databank, que lo **lee**. Para `Results` es `005`; para `005`, `Retest Markets - Family`; para `SPP 005`, `WFM`. Se activa **solo esa tarea**.

DATABANKS · Test_USDJPY_donchianUpperCrossUp_M30 · Results

No se puede continuar: «Results» no tiene ningún .sqx en SQX_w2: no hay nada que borrar (¿Auto-sync never, o el databank no es de este install?)

▶▶ Continuar workflow

Cómo se ejecuta

No hay que teclear nada: es un botón en la cabecera del panel de databanks.

1. Filtra el databank (primera parte de este capítulo).
2. Pulsa ▶▶ **Continuar workflow**. Aparece la pantalla de confirmación con esta frase literal: «Se van a borrar **N** estrategias de en sobre , y después se lanzará ». Si SQX guarda alguna estrategia repetida con otro nombre, la frase lo añade al final («(19 ficheros: SQX guarda algunas repetidas con otro nombre, y se borran todas las copias)»).
3. **No** cancela sin tocar nada. **Sí** encola un trabajo, que sigues en la franja de trabajos y en OPERACIÓN > «En marcha» (capítulo 70).

El trabajo es un comando que también se puede lanzar desde la terminal (hace las mismas comprobaciones y se niega igual):

```
python3 -m ui.daemon.advance.run --project Test_UiContinuar_USDJPY --databank Results
```

flag	obligatorio	qué hace
--project	sí	el proyecto, tal y como aparece en <code>registry.csv</code>
--databank	sí	el databank cuyos descartes se borran; vale con espacios o con guiones bajos

Lo que hace, en este orden:

1. Vuelve a comprobarlo todo (dos clics no pueden pisarse: solo corre uno a la vez).
2. **Copia** los `.sqx` que va a borrar a `AlgoData/projects/discards/<proyecto>/<databank>/<fecha-hora>/`, con un `verdict.csv` (estrategia, `DESCARTAR`, identidad, el filtro que la descartó).
3. Los **borra** del databank con el worker parado, con el mismo código que `/curate (sqx.curate.apply_verdict)`, que antes comprueba que cada fichero es la estrategia que juzgaste.
4. Apunta una fila en el **Ledger** (el borrado, de cuántas a cuántas) y marca los descartes como gastados en su lista (una línea `cut`).
5. Deja **activa solo la tarea siguiente** en el `project.cfx`.
6. **Arranca** el worker, espera a que responda y lanza el proyecto.
7. Mientras corre, solo le pregunta «¿cómo vas?» (`-project action=status`): ninguna otra orden.
8. Cuando el log de SQX dice «Project finished», **para** el worker. Lee también el log de ayer, así que una tarea que cruza la medianoche se sigue igual.

Tiene dos topes, para que el trabajo nunca se quede colgado con el worker arrancado: si SQX no escribe «Starting project» en los **2 minutos** siguientes a lanzarlo, o si la tarea pasa de **48 horas** sin terminar, para el worker y falla diciendo cuál de los dos fue. Si el arranque del worker falla a medias, también lo para.

Cuánto tarda: lo que tarde la tarea siguiente en SQX, más ~1 minuto de arrancar y parar.

Qué produce

ruta	qué es
AlgoData/projects/discards/<proyecto>/<databank>/<fecha-hora>/*.sqx	copia de cada estrategia borrada, por si hay que recuperarla
.../<fecha-hora>/verdict.csv	qué se borró y por qué
AlgoData/reports/<proyecto>/<databank>/<día>/curate/before-HHMMSS.csv , rejected-HHMMSS.csv	lo que había antes del borrado y las métricas de lo borrado (como /curate)
el databank en el worker	con N estrategias menos — esto no se deshace solo : para recuperar, copia los .sqx de vuelta con el worker parado
AlgoData/ledger/<símbolo>_<timeframe>_<plantilla>.jsonl	una fila más: el borrado
las salidas de la tarea siguiente	su databank en el worker, como si la hubieras lanzado tú

Cómo se lee el resultado

En «En marcha» el trabajo `advance` va diciendo por dónde va: «copiando 17 descartes», «borrando 17 de Results (SQX_w1)», «activando solo OOS», «arrancando el conductor», «OOS lanzada», el porcentaje de la tarea, y al final «OOS: done; Results 200 → 183». Si se niega, la última línea dice por qué y no ha tocado nada.

Un ejemplo completo

Aún no se ha corrido contra un install de verdad: la prueba en vivo (en el conductor, con el proyecto `Test_UiContinuar_USDJPY`) espera tu autorización. Lo que sí es real es la negativa del 2026-09-28 sobre el proyecto del custodio:

```
GET /api/advance/preflight?project=Test_USDJPY_donchianUpperCrossUp_M30&databank=Results
{"ok": false,
 "reasons": ["«Results» no tiene ningún .sqx en SQX_w2: no hay nada que borrar (¿Auto-sync never,
             o el databank no es de este install?)"],
 "install": "SQX_w2", "task": "OOS", "n": 17}
```

Había 17 descartes y la tarea siguiente era `OOS`, pero el databank `Results` de ese proyecto no tiene sus `.sqx` en el disco del custodio: no hay nada que borrar, y el botón se queda apagado.

Qué NO te dice

- **Que el corte sea bueno.** Borra lo que tú descartaste; el juicio es el de los filtros.
- **Que la tarea siguiente haya ido bien.** Solo que terminó: mira su databank y sus estudios.
- **No lanza varias tareas.** Si el databank lo leen varias (a `Results` lo leen `OOS`, los ocho MC Retest y `SPP IS`), solo lanza la primera del workflow.

Si algo falla

- **«install ocupado: el puerto ... responde»** — el worker está arrancado. Mira quién lo usa (`ls -lt ~/Desktop/SQX_w1/user/projects | head`); si es tuyo y está ocioso, páralo con `bin/sqx-worker.sh --role conductor stop` y vuelve a pulsar.

- «... se modificó en SQX_w1 hace 2.0 h: puede ser de otra sesión» — otro proyecto de ese worker se tocó hace menos de 24 h. Espera, o confirma con esa sesión.
- «N de M descartes ya no están en «Results»» — el databank cambió desde que filtraste; el botón se queda apagado. Quita los filtros y vuelve a filtrar. (Tras un «Continuar» que sale bien no pasa: el corte deja una línea en la lista de descartes y los da por gastados; el siguiente filtro empieza de lo que queda.)
- «SQX no escribió «Starting project ...» en 120 s» o «SQX rechazó `action=start` » — SQX no arrancó la tarea. El worker ya está parado y el databank ya está cortado (la copia sigue en `projects/discards/`): lánzala a mano o vuelve a filtrar.
- **Cancelaste el trabajo a mitad** — el worker se queda arrancado: páralo con `bin/sqx-worker.sh --role <conductor|custodian> stop`.

68. Estrategia — la ficha de una estrategia: curvas, estadísticas, metadatos y «Archivar»

Qué pregunta responde

- **¿Cómo le fue a esta estrategia, con los costes de SQX y con los de verdad?** Arriba, la curva de equity de SQX (violeta) y la misma con el **spread y el slippage reales** de Darwinex (ámbar), cada una con su interruptor. La línea discontinua separa IS de OOS1.
- **¿Cómo se reparten sus operaciones?** Las estadísticas básicas de cada tramo — IS, OOS1 y OOS2 —: operaciones, beneficio neto (de SQX y con costes reales), factor de beneficio, % ganadoras, DD máximo, Sharpe y, del **retorno por operación**, media, mediana, desviación típica, asimetría, **curtosis**, percentiles 5, 25, 75 y 95, con su histograma al lado.
- **¿De qué está hecha?** La columna de la derecha, **Metadatos**: dirección, la señal de entrada leída como frase, cada condición con sus parámetros, las órdenes (SL, PT, salida por barras), el money management, el cierre de los viernes, el activo, el último test que guarda el `.sqx` y **sus costes**, con un aviso ámbar si una tarea del proyecto hoy carga otros.
- **¿Qué dijeron de ella los estudios?** Debajo, las pestañas de siempre (Ficha, Cribado, Transferencia, Rotura, Optimización, Cierre, Lecturas, Datos), que se abren en la familia del panel desde el que llegaste: desde «Puerta IS/OOS», en **Cribado**; desde «MC Retest», en **Rotura**; desde «SPP» o «WFC», en **Optimización**.

Cuándo lo usas, y cuándo no

Lo usas cada vez que una estrategia te llama la atención en el panel de databanks: doble clic en su fila y se abre su ficha. Y cuando decides que merece guardarse, **Archivar**.

No sirve para juzgar la estrategia: la ficha describe, no filtra. Las cribas y sus veredictos viven en las pestañas de abajo, cada una con su estudio.

No enseña OOS2 antes de tiempo: el botón OOS2 está apagado y al lado dice «OOS2 reservado: se abre tras los pasos 17, 18 y 19» hasta que los tres estén en el ledger del estudio del proyecto. Pasar el ratón por esa frase da la del ledger («faltan [17, 18, 19] ... en el estudio USDJPY_M30_donchianUpperCrossUp»). Abierto, OOS2 se lee de una cosecha que traiga la muestra oos2; si no hay ninguna, la ficha dice «OOS2 abierto, sin export: exporta el retest oos2» y no se inventa otra fuente.

Antes de empezar

- La ventana abierta (`bin/algoui`), el proyecto elegido en la galería y un databank con **cosecha** (paso 8, `/oos-gate`): las curvas y las estadísticas salen de ella. Sin cosecha la ficha lo dice en rojo con el comando que la hace.
- La curva real necesita el informe del estudio **spread** de ese databank (paso 8). Si no existe, en su sitio aparece «spread y slippage reales: no calculado» con dos botones.
- Los metadatos se leen del `.sqx`: primero del install que tiene el databank; si el databank está vacío en el install (lo normal cuando el proyecto terminó), de la copia que dejó un export en `AlgoData/raw/.../strategies/`; y si tampoco, de la última versión archivada. Sin ninguna, la columna dice «sin .sqx en ningún install».
- Para «Archivar», el proyecto tiene que tener **plantilla** en `AlgoData/projects/registry.csv`: con ella se apunta la cuenta del ledger en el archivo.

Cómo se ejecuta

No hay comando: se abre con doble clic en una fila del panel de databanks (zona Proyecto), o desde la paleta (Ctrl+K) eligiendo una estrategia.

botón	dónde	qué hace
SQX / spread y slippage reales	encima de la curva	enciende o apaga cada curva; el eje se ajusta a las que quedan
IS / OOS1 / OOS2	estadísticas	el tramo que se enseña; OOS2 apagado mientras esté reservado
USD por lote / USD por operación	estadísticas	en qué se cuenta el retorno de cada operación; por lote quita el tamaño de la posición
calcular	junto a un «no calculado»	encola el estudio que falta para esta estrategia (el spread real: <code>studies.data.spread.report --strategy ...</code>)
todo el databank	junto a un «no calculado»	lo encola para todas las estrategias del databank, en paralelo
Archivar	arriba a la derecha	pide el paso y una nota y congela la estrategia en <code>AlgoData/archive/</code>
▼ plegar panel básico	arriba a la derecha	esconde curvas y estadísticas para dar la altura a las pestañas

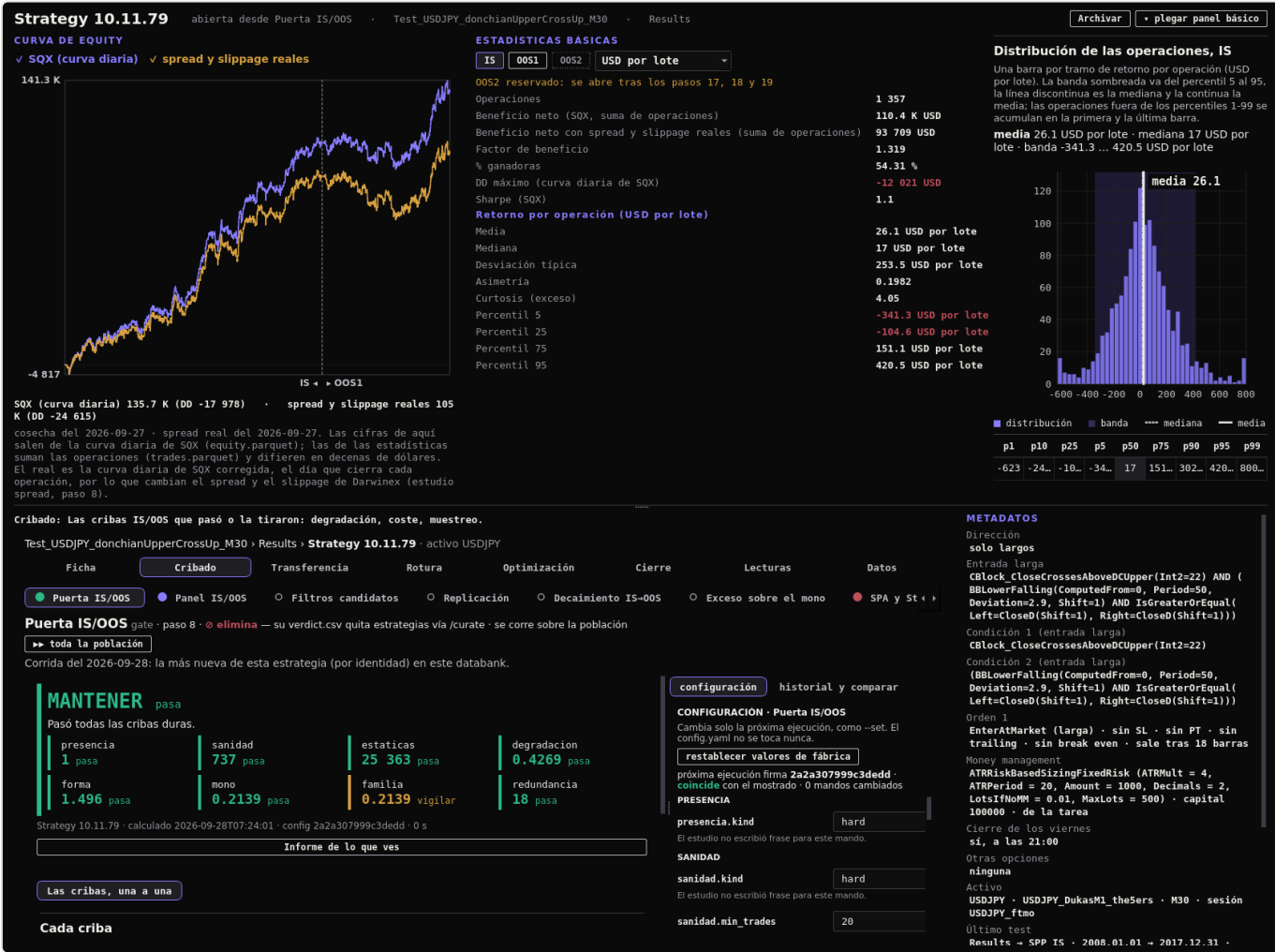
Los botones «calcular» no tocan SQX: son estudios de Python en la cola de la ventana. La ficha se redibuja sola cuando terminan. «Archivar» tampoco toca SQX: lee ficheros y tarda un par de segundos.

Qué produce

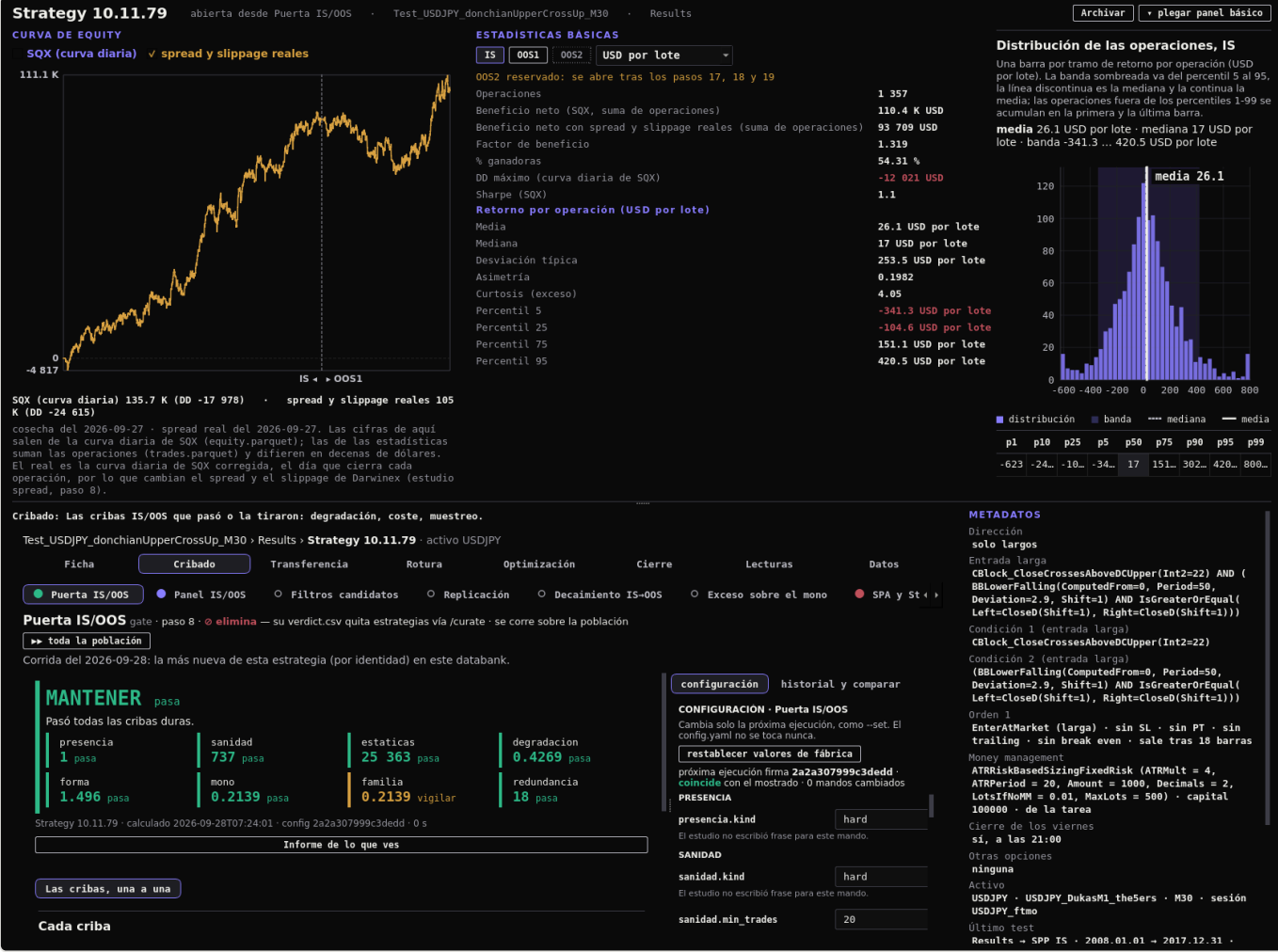
Mirar no escribe nada. Dos botones sí:

- **calcular / todo el databank** — el informe del estudio en `AlgoData/reports/<proyecto>/<databank>/<día>/spread/` (`estrategias/<nombre>.json` y `.trades.parquet` para una; `spread.json` , `trades.parquet` y `verdict.csv` para todo el databank).
- **Archivar** — una versión nueva en `AlgoData/archive/<identidad>/<AAAA-MM-DDTHMM>/` con `manifest.json` (paso, nota, de dónde salió el `.sqx` , cada estudio archivado y lo que no se pudo archivar), `strategy.sqx` , `view.json` , las filas de la cosecha, los informes, la ficha de costes del activo y `meta.json` . Se sella de solo lectura; **nunca sobrescribe**: el mismo minuto dos veces se niega.

Cómo se lee el resultado



Strategy 10.11.79 de Test_USDJPY_donchianUpperCrossUp_M30 , databank Results. A la izquierda, las dos curvas: SQX acaba en 135.7 K con un DD de -17 978; con el spread y el slippage reales de Darwinex, en 105 K con un DD de -24 615. La distancia entre las dos es lo que SQX no cobró. Las cifras bajo la curva salen de la curva diaria de SQX y las de las estadísticas suman las operaciones: difieren en decenas de dólares (OOS1: 25 258 contra 25 363), y cada etiqueta dice cuál es. En el centro, IS: 1 357 operaciones, 110.4 K de SQX (suma de operaciones) y 93 709 con costes reales, factor de beneficio 1.319, 54.31 % ganadoras. El retorno por operación tiene media 26.1 USD por lote, mediana 17, asimetría 0.198 y **curtosis (exceso) 4.05**: colas más gordas que una normal (que da 0). A la derecha, el histograma: la banda es del percentil 5 al 95, la discontinua la mediana y la línea blanca la media. Abajo, las pestañas abiertas en **Cribado** porque se llegó desde Puerta IS/OOS.



Cribado: Las cribas IS/005 que pasó o la tiraron: degradación, coste, muestreo.

Test_USDJPY_donchianUpperCrossUp_M30 · Results · Strategy 10.11.79 · activo USDJPY

Ficha Cribado Transferencia Rotura Optimización Cierre Lecturas Datos

Puerta IS/005 Panel IS/005 Filtros candidatos Replicación Decaimiento IS-005 Exceso sobre el mono SPA y St +

Puerta IS/005 gate · paso 8 · elimina — su verdict.csv quita estrategias vía /curate · se corre sobre la población

toda la población

Corrida del 2026-09-28: la más nueva de esta estrategia (por identidad) en este databank.

MANTENER pasa

Pasó todas las cribas duras.

presencia 1 pasa

sanidad 737 pasa

estaticas 25 363 pasa

degradacion 0.4269 pasa

forma 1.496 pasa

mono 0.2139 pasa

familia 0.2139 vigilar

redundancia 18 pasa

Strategy 10.11.79 · calculado 2026-09-28T07:24:01 · config 2a2a307999c3dedd · 0 s

Informe de lo que ves

Las cribas, una a una

Cada criba

configuración historial y comparar

CONFIGURACIÓN · Puerta IS/005

Cambia solo la próxima ejecución, como --set. El config.yaml no se toca nunca.

restablecer valores de fabrica

próxima ejecución firma 2a2a307999c3dedd · coincide con el mostrado · 0 mandos cambiados

PRESENCIA

presencia.kind hard

El estudio no escribió frase para este mando.

SANIDAD

sanidad.kind hard

El estudio no escribió frase para este mando.

sanidad.min_trades 20

METADATOS

Dirección solo largos

Entrada larga

CBLOCK_closeCrossesAboveDCupper(Int2=22) AND (BBLowerFalling(ComputedFrom=0, Period=50, Deviation=2.9, Shift=1) AND IsGreaterOrEqual(Left=CloseD(Shift=1), Right=CloseD(Shift=1)))

Condición 1 (entrada larga)

CBLOCK_closeCrossesAboveDCupper(Int2=22)

Condición 2 (entrada larga)

(BBLowerFalling(ComputedFrom=0, Period=50, Deviation=2.9, Shift=1) AND IsGreaterOrEqual(Left=CloseD(Shift=1), Right=CloseD(Shift=1)))

Orden 1

EnterATMarket (larga) · sin SL · sin PT · sin trailing · sin break even · sale tras 18 barras

Money management

ATRRiskBasedSizingFixedRisk (ATRMult = 4, ATRPeriod = 20, Amount = 1000, Decimals = 2, LotsIfNoMM = 0.01, MaxLots = 500) · capital 100000 · de la tarea

Cierre de los viernes

si, a las 21:00

Otras opciones

ninguna

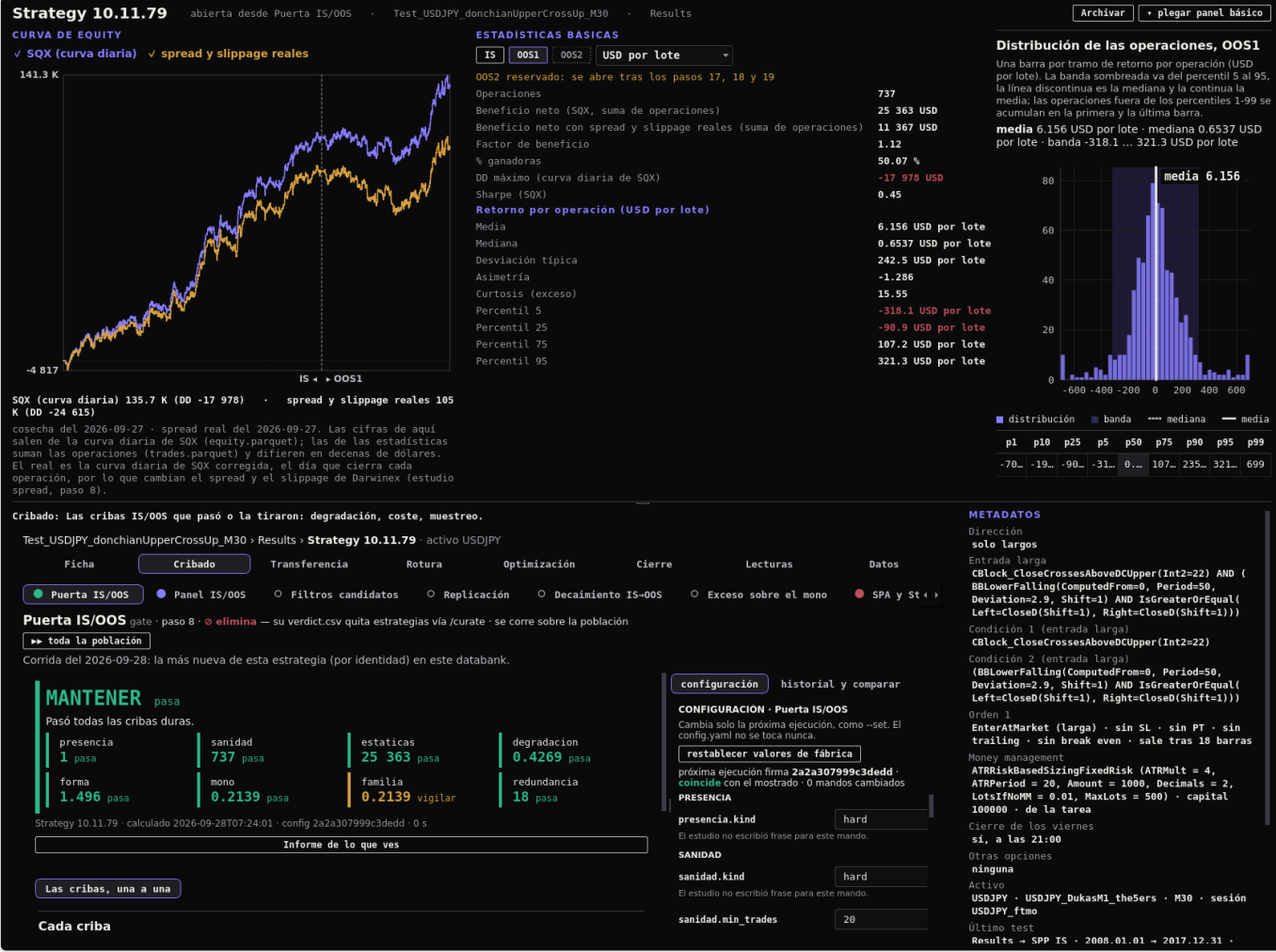
Activo

USDJPY · USDJPY_DukasM1_the5ers · M30 · sesión USDJPY_ftmo

Último test

Results → SPP IS · 2008.01.01 → 2017.12.31 ·

Con SQX apagada queda solo la curva real y el eje se reajusta a ella (111.1 K arriba).



```

(BBLowerPatching(ComputedFrom=0, Period=50,
Deviation=2.9, Shift=1) AND IsGreaterOrEqual(
Left=CloseD(Shift=1), Right=CloseD(Shift=1)))
Orden 1
EnterAtMarket (larga) · sin SL · sin PT · sin
trailing · sin break even · sale tras 18 barras
Money management
ATRRiskBasedSizingFixedRisk (ATRMult = 4,
ATRPeriod = 20, Amount = 1000, Decimals = 2,
LotsIfNoMM = 0.01, MaxLots = 500) · capital
100000 · de la tarea
Cierre de los viernes
sí, a las 21:00
Otras opciones
ninguna
Activo
USDJPY · USDJPY_DukasM1_the5ers · M30 · sesión
USDJPY_ftmo
Último test
Results → SPP IS · 2008.01.01 → 2017.12.31 ·
MetaTrader5 (hedged) · precisión 1
Costes del último test
spread 0.1 · slippage 0.05 · min distance 10 ·
comisión SizeBased 0.0 · swap largo 0.0, corto
-10.4 (points, triple el miércoles)
Nota
asset_card_sha256: ficha de costes actual, no la
del backtest (no existe la histórica)
Fichero
Strategy 10.11.79.sqx · copia exportada de
SPP_IS del 2026-09-27

```

La columna de metadatos, bajando: la orden (a mercado, sin SL ni PT, sale tras 18 barras), el money management (riesgo fijo por ATR, 1 000 por operación sobre 100 000), cierre de los viernes a las 21:00, el activo y la sesión, el último test (Results → SPP IS, 2008–2017) y sus costes: spread 0.1, slippage 0.05, swap corto -10.4 puntos con triple el miércoles. La última línea dice de dónde salió el `.sqx` : aquí, la copia que dejó el export del SPP, porque el databank del install está vacío. Por eso el «último test» es el del SPP y no el del build.

Congela la estrategia en AlgoData/archive/: su .sqx, sus filas de la cosecha, cada resultado de estudio que la nombra, la ficha de costes del activo y la cuenta del ledger de hoy. Una versión nueva cada vez; nunca se sobrescribe.

Paso

Nota

Sobrevive a la puerta y al coste real; la guardo antes del WFC.

«Archivar» pide el **paso** del WORKFLOW en que está la estrategia (sin valor por defecto: el último paso «hecho» del raíl puede ser una lectura corrida con 17-19 sellados) y una nota, que se guarda tal cual.

Un ejemplo completo

1. En Proyecto, pestaña Puerta IS/OOS de Results, doble clic en «Strategy 10.11.79».
2. Se abre la ficha: curvas, estadísticas IS, metadatos, y abajo Cribado.
3. Clic en **OOS1**: el neto real fuera de muestra es 11 367 frente a 25 363 de SQX.
4. Clic en **OOS2**: no se puede; al lado dice «OOS2 reservado: se abre tras los pasos 17, 18 y 19».
5. Clic en **Archivar**, paso 16.5, nota «Sobrevive a la puerta y al coste real; la guardo antes del WFC.», **Archivar**. Arriba aparece «archivada: versión 2026-09-28T0724 (copia exportada de SPP_IS del 2026-09-27)», y la carpeta AlgoData/archive/4d679e0c.../2026-09-28T0724/ tiene su manifest.json.

Lo mismo por el demonio, para comprobar que OOS2 está cerrado:

```
curl -s "127.0.0.1:8765/api/tearsheet?
project=Test_USDJPY_donchianUpperCrossUp_M30&databank=Results&identity=4d679e0c2ce2a63ee53bc6cb305830b
caf5a74997e6e6e2c1deeb4c30f307048&sample=00S2"
{"blocked":"reservado: se abre tras los pasos 17, 18 y 19","why":"el paso 20 es ciego hasta que 17, 18
y 19 estén los tres hechos; faltan [17, 18, 19]. ... (en el estudio USDJPY_M30_donchianUpperCrossUp)"}

```

Y una estrategia archivada se lee sin calcular nada añadiendo `source=archive` a cualquier ruta de la ficha (`/api/result`, `/api/tearsheet`, `/api/strategy/stats`, ...): responde lo mismo que la viva el día que se archivó.

Qué NO te dice

- **La curva real no es un retest en Darwinex.** Cambia solo el spread (y el slippage, la mitad del spread en cada ejecución) de cada operación, al minuto en que la paga; el precio sigue siendo el de Dukascopy. Antes de octubre de 2017 el spread es de modelo, no medido (capítulo 59).

- **Las estadísticas no son un veredicto.** Una curtosis alta dice que hay colas, no que la estrategia sea mala; la decisión la toman las cribas de abajo.
- **El «último test» es el que guarda el .sqx que se encontró.** Si el .sqx es la copia de un export (SPP, WFC...), sus costes son los de esa tarea; la línea «Fichero» lo dice.
- **Los metadatos del archivo son los del día de archivar.** La ficha de costes del activo puede haber cambiado después.

Si algo falla

- **«El databank Results de ... no tiene cosecha»** — falta el paso 8: /oos-gate .
- **«sin .sqx en ningún install»** en metadatos — ni el install, ni un export, ni el archivo guardan esa estrategia. Exporta el databank (/export) o archívala mientras siga en el install.
- **«Falta el paso del WORKFLOW...» / «el proyecto ... no tiene plantilla en projects/registry.csv»** al archivar — escribe el paso; y sin plantilla no hay familia con la que firmar la cuenta del ledger, así que no se archiva.
- **«El demonio no respondió...»** en rojo en cada panel — el demonio está caído o arrancando; «Recargar» cuando vuelva.

58. Estrategia — cómo se lee el resultado de un estudio

Todos los estudios entregan su resultado con la misma forma (pestañas, selectores, ocho tipos de gráfico, veredicto, avisos y glosario) y la ventana la pinta siempre igual. Este capítulo explica esa página: los histogramas IS/OOS superpuestos, las superficies de varios mercados lado a lado, la re-ejecución de una sola subprueba sin pisar lo guardado y el informe que copia lo que ves.

Qué pregunta responde

- **¿Se degradaron las operaciones fuera de muestra, y cuánto?** → las densidades IS/OOS con el desplazamiento de la mediana y el p del KS al lado.
- **¿La región buena de parámetros es la misma en otros mercados?** → dos o tres superficies lado a lado con la misma escala, θ_0 marcado, y el mapa de consenso debajo.
- **¿Qué pasa si repito solo esta subprueba (un mercado, una prueba de Monte Carlo)?** → «↺ solo ...»: el resultado nuevo **al lado** del guardado, o fusionado en pantalla, sin tocar el fichero.
- **¿Puedo guardar exactamente lo que estoy viendo?** → «Informe de lo que ves».

Cuándo lo usas, y cuándo no

Lo usas para leer cualquier resultado sin abrir un HTML ni un CSV, para comparar la forma de las operaciones dentro y fuera de muestra, para ver si una meseta viaja entre mercados y para repetir una subprueba con otro mando sin perder la corrida entera.

No lo usas para decidir con una re-ejecución parcial. Una re-ejecución **nunca cambia el veredicto**: un veredicto sale de un análisis entero o de ninguno. Tampoco para comparar Net Profit o drawdown entre IS y OOS: crecen con la longitud del tramo y por eso no salen en las densidades.

Antes de empezar

- El estudio ya corrido: su resultado vive en
`~/Desktop/AlgoData/reports/<proyecto>/<databank>/<día>/<estudio>/estrategias/<estrategia>.json`.
- La app abierta con `bin/algoui`. Si estaba abierta antes de este cambio, ciérrala y ábrela: el demonio viejo no conoce `/api/study/screen` ni guarda las re-ejecuciones al lado.
- Para «↺ solo ...» de Monte Carlo hace falta una corrida entera de Monte Carlo en ese databank: las pruebas que ofrece salen de ella.

Cómo se ejecuta

```
bin/algoui
```

En **Proyectos** elige el proyecto; en **Proyecto**, doble clic en la fila de la estrategia en el panel de databanks; en **Estrategia**, abre la pestaña de la familia (Cribado, Transferencia, Rotura...) y, dentro, el estudio. Esa página es la que describe este capítulo. La re-ejecución de una subprueba la lanza la ventana; por debajo corre este comando, que también puedes lanzar a mano:

```
python3 -m ui.daemon.results.rerun --study crossmarket \  
--project Test_USDJPY_donchianUpperCrossUp_M30 --databank Retest_Markets_-_Family \  
--asset USDJPY --day 2026-09-27 --strategy "Strategy 9.18.85(1)" \  
--only EURUSD_DukasM1_the5ers --set nulls.draws=2000
```

flag	obligatorio	qué hace
--study	sí	crossmarket (un mercado) o monteCarlo (una prueba)
--project , --databank	sí	dónde está el export
--asset	sí	el activo base, p. ej. USDJPY
--day	sí	la fecha del export raw/ que se lee
--strategy	sí	la estrategia, tal y como la escribe SQX
--only	sí	el mercado (EURUSD_DukasM1_the5ers) o el nombre de la prueba tal y como sale en el selector «Prueba» de Monte Carlo (Costes hasta el doble)
--set	no	cambia un mando solo para esta corrida, como el cajón de configuración

Tarda lo que tarda esa subprueba sola: un mercado de cross-market con 2.000 sorteos, **11 s.** No toca SQX.

Qué produce

- La re-ejecución: reports/<P>/<D>/<día>/<estudio>/parciales/<fecha-hora>_<subprueba>/estrategias/<estrategia>.json y su .html , **al lado** del resultado entero del día más nuevo que lo tenga. El resultado entero **no se sobrescribe** (antes sí: OPEN.md #54).
- «Informe de lo que ves»: reports/<P>/<D>/<día>/<estudio>/pantalla/<estrategia>-<fecha-hora>.html , y la ventana lo abre en el navegador.

Cómo se lee el resultado

Densidades IS y OOS

Informe de lo que ves

IS contra OOS, operación por operación

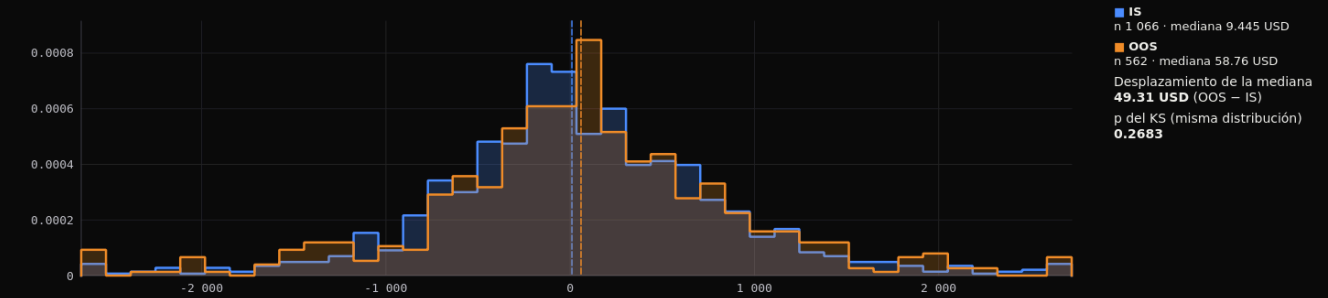
Las mismas cuatro medidas por operación dentro y fuera de muestra. Sólo medidas por operación: el beneficio neto y el drawdown crecen con la longitud de la ventana y no se comparan así. El múltiplo R espera a que la estrategia lleve stop (paso 24).

Unidad del dinero **USD**

Retorno por operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista **✓ IS (n 1 066)** **✓ OOS (n 562)**



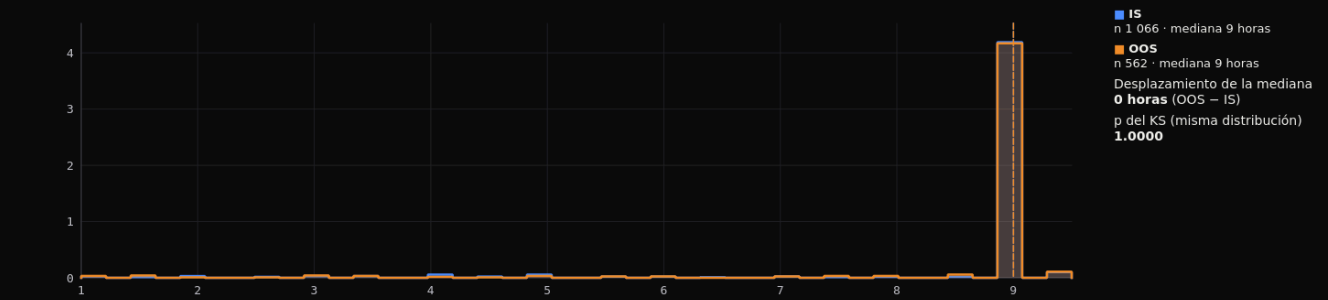
Percentiles de la unión

p1	p5	p10	p25	p50	p75	p90	p95	p99
-2 281	-1 251	-819.9	-374.5	26.03	495.9	1 001	1 370	2 310

Duración de cada operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista **✓ IS (n 1 066)** **✓ OOS (n 562)**



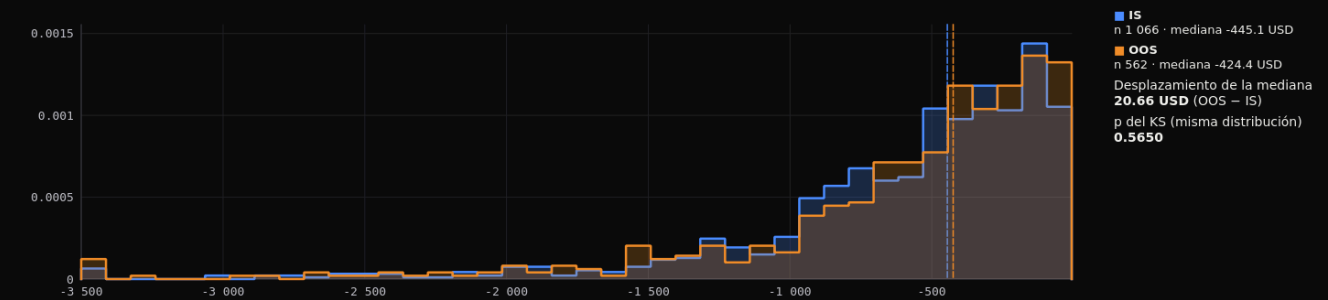
Percentiles de la unión

p1	p5	p10	p25	p50	p75	p90	p95	p99
1.5	5	9	9	9	9	9	9	9.5

MAE — lo peor que llegó a ir cada operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista **✓ IS (n 1 066)** **✓ OOS (n 562)**



Percentiles de la unión

p1	p5	p10	p25	p50	p75	p90	p95	p99
-2 859	-1 715	-1 260	-784.8	-434.7	-212.5	-93.78	-46.32	-9.993

MFE — lo mejor que llegó a ir cada operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.

Muestras a la vista **✓ IS (n 1 066)** **✓ OOS (n 562)**





Cada muestra es una **densidad**: el área bajo cada curva vale 1, así IS (1.066 operaciones) y OOS (562) se comparan por su forma, no por cuántas hay. Encima, una casilla por muestra para enseñarla u ocultarla. A la derecha:

- cada muestra con su **n** y su **mediana** (la línea discontinua de su color);
- **desplazamiento de la mediana**: mediana OOS menos mediana IS, en la unidad del gráfico. En el retorno por operación de la captura, +49,31 USD: fuera de muestra la operación típica ganó más;
- **p del KS**: la probabilidad de ver una diferencia de forma así de grande si las dos muestras vinieran de la misma distribución. **Pequeño (< 0,05) = las operaciones cambiaron de verdad**. Aquí 0,27 en el retorno: no hay degradación que se distinga del azar.

Debajo, la fila de **percentiles de la unión** de las dos muestras. El ratón sobre una barra dice el tramo y qué parte de las operaciones de cada muestra cae en él.

Superficies de varios mercados, lado a lado

la región no viaja falla

Sólo 0 de 9 en build, 0 de 9 en oos1 ordenan las variantes como el mercado principal con el decil superior compartido: lo que es bueno aquí no lo es allí. Ganar en otro mercado, si gana, no es la misma región. Costes PROVISIONALES: los 9 pares y el principal se retestearon con los defaults de fábrica de SQX, comisión CERO. El coste por operación mueve más a las variantes que más operan, así que puede reordenar una superficie: lee esto como provisional hasta que el dueño fije los costes de the5ers.

EURUSD build 0.4625 vigilar	GBPUSD build -0.1384 falla	USDCHE build -0.2532 falla	AUDUSD build -0.04667 falla
USDCAD build 0.2835 vigilar	EURJPY build 0.2405 vigilar	GBPJPY build 0.4895 vigilar	AUDJPY build 0.3707 vigilar
CADJPY build -0.2961 falla	EURUSD oos1 0.3982 vigilar	GBPUSD oos1 0.4569 vigilar	USDCHE oos1 0.2271 vigilar
AUDUSD oos1 0.04901 falla	USDCAD oos1 0.3605 vigilar	EURJPY oos1 0.382 vigilar	GBPJPY oos1 0.5029 vigilar
AUDJPY oos1 0.3013 vigilar	CADJPY oos1 0.02056 falla		

Strategy_9.27.83 · calculado 2026-09-27T20:37:06 · config 1dc2f1de9c2d8b02 · 3.34 s

Informe de lo que ves

La Lectura Todos los pares Una superficie por mercado **Superficies por pareja de parámetros** Mapa de consenso Verificación

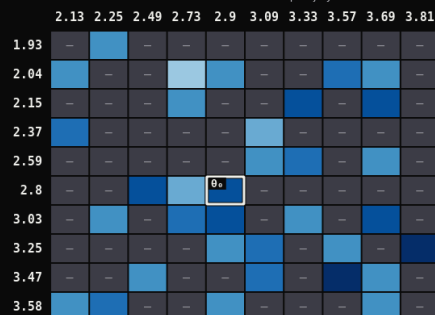
Elige mercado, tramo y un parámetro para cada eje. Cada celda es la mediana del beneficio neto de las variantes que usaron esa pareja de niveles, sin contar a la madre; θ_0 , sus parámetros, va marcado. Todos los mercados de una pareja y tramo comparten escala de color: el nivel depende del coste (provisional), la forma es lo que se compara.

Tramo **build** Eje X **BBERDeviation1** Eje Y **BBERDeviation2**

Mercados a la vista (hasta 3) **USDJPY** **EURUSD** **GBPUSD** **USDCHE** **AUDUSD** **USDCAD** **EURJPY** **GBPJPY** **AUDJPY** **CADJPY**

USDJPY — build: BBERDeviation2 contra BBERDeviation1

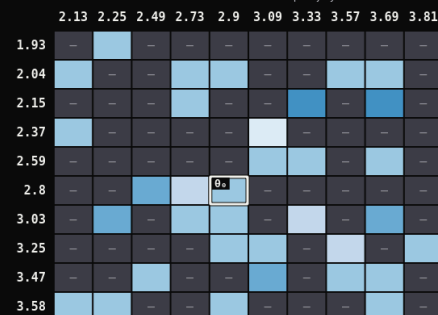
Mediana del beneficio neto de las variantes de cada celda en este mercado. La misma escala de color en todos los mercados de esta pareja y tramo.



■ < -72.250 ■ -72.250 ... -48.322 ■ -48.322 ... -24.393 ■ -24.393 ... -464.1 ■ -464.1 ... 23.465 ■ 23.465 ... 47.394 ■ 47.394 ... 71.322 ■ 71.322 ... 95.251 ■ ≥ 95.251 Sin dato — θ_0 — La celda de la madre (sus parámetros) ■ Escala común a las rejillas: -96.179 ... 119.2 K

EURUSD — build: BBERDeviation2 contra BBERDeviation1

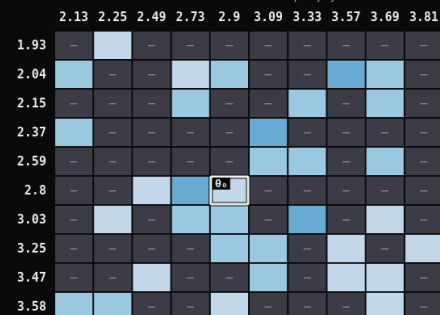
Mediana del beneficio neto de las variantes de cada celda en este mercado. La misma escala de color en todos los mercados de esta pareja y tramo.



■ < -72.250 ■ -72.250 ... -48.322 ■ -48.322 ... -24.393 ■ -24.393 ... -464.1 ■ -464.1 ... 23.465 ■ 23.465 ... 47.394 ■ 47.394 ... 71.322 ■ 71.322 ... 95.251 ■ ≥ 95.251 Sin dato — θ_0 — La celda de la madre (sus parámetros) ■ Escala común a las rejillas: -96.179 ... 119.2 K

GBPUSD — build: BBERDeviation2 contra BBERDeviation1

Mediana del beneficio neto de las variantes de cada celda en este mercado. La misma escala de color en todos los mercados de esta pareja y tramo.

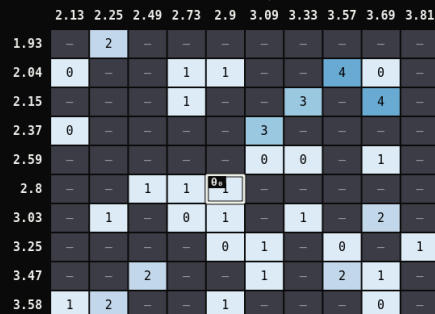


■ < -72.250 ■ -72.250 ... -48.322 ■ -48.322 ... -24.393 ■ -24.393 ... -464.1 ■ -464.1 ... 23.465 ■ 23.465 ... 47.394 ■ 47.394 ... 71.322 ■ 71.322 ... 95.251 ■ ≥ 95.251 Sin dato — θ_0 — La celda de la madre (sus parámetros) ■ Escala común a las rejillas: -96.179 ... 119.2 K

CONSENSO DE TODOS LOS MERCADOS

Consenso — build: BBERDeviation2 contra BBERDeviation1

En cuántos de los 10 mercados cada celda está en el 10 % superior de las celdas de ese mercado. La celda de θ_0 está en el decil superior en 1 de 10 mercados.



■ < 0 ■ 0 ... 1 ■ 1 ... 2 ■ 2 ... 3 ■ 3 ... 4 ■ 4 ... 5 ■ 5 ... 6 ■ 6 ... 7 ■ 7 ... 8 ■ 8 ... 9 ■ 9 ... 10 ■ ≥ 10 Sin dato — θ_0 — La celda de la madre (sus parámetros)

AVISOS (1 DISTINTOS)

Los 9 mercados son de la misma familia macro que el principal (markets.yaml, sin structural): pasar aquí es la prueba fácil; fallar sí dice algo.

GLOSARIO

rho_ab — Spearman del beneficio neto entre las variantes de dos mercados: si ordenan igual las combinaciones.

j_ab — Jaccard de los deciles superiores: qué parte de las mejores combinaciones son las mismas en los dos mercados.

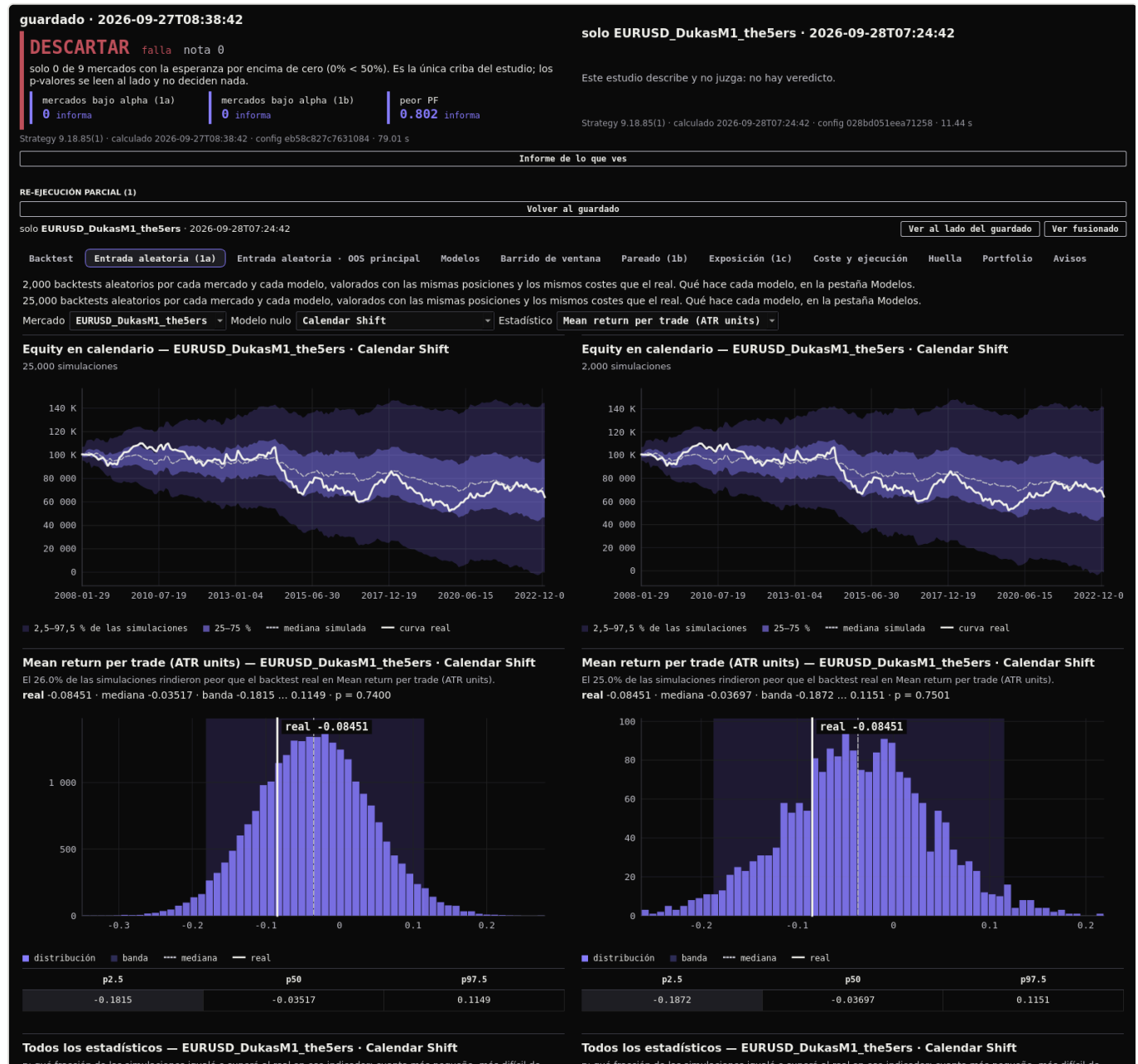
n_eff — Variantes con un resultado distinto en el par: dos tuplas que dan el mismo backtest cuentan una vez.

En la pestaña de superficies por pareja de parámetros de Market Surfaces:

- «**Mercados a la vista (hasta 3)**»: un botón por mercado. Pulsar un cuarto suelta el más antiguo; nunca quedan menos de uno. No recalcula nada.
- Las tres superficies comparten **la misma escala de color** (la leyenda dice «Escala común a las rejillas»): un mismo azul es el mismo beneficio en los tres mercados.
- θ_0 , la celda de los parámetros de la madre, va recuadrada en blanco en cada una.

- Debajo, el **consenso de todos los mercados**: en cuántos de los diez cada celda está en su 10 % superior. Una región que viaja se ve como una mancha alta y continua; θ_0 debería caer dentro. En la captura cae en 1 de 10: la madre no está en la meseta compartida.
- Si la celda es demasiado estrecha para su número, el número sale al pasar el ratón.

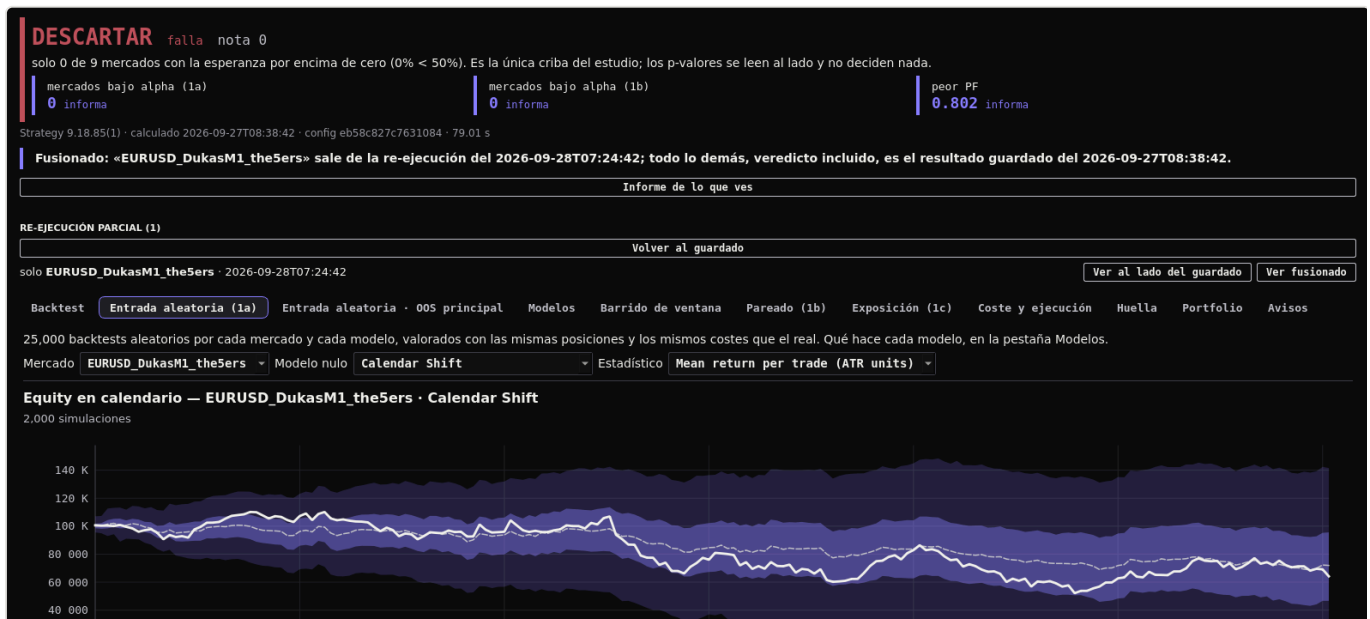
Re-ejecutar una subprueba: al lado y fusionado



Cuando una estrategia tiene re-ejecuciones parciales, sale la tira **RE-EJECUCIÓN PARCIAL (n)** con una línea por cada una:

- **Ver al lado del guardado**: el resultado entero a la izquierda y la re-ejecución a la derecha, gráfico con gráfico, abiertos en la misma subprueba (el mismo mercado o la misma prueba). En la captura, 25.000 sorteos guardados contra 2.000 nuevos: el valor real es el mismo y el p pasa de 0,74 a 0,75.
- **Ver fusionado**: el resultado entero con esa subprueba tomada de la re-ejecución. Una línea bajo el veredicto dice qué parte es nueva y de cuándo. El veredicto sigue siendo el guardado. Una re-ejecución vista sola no tiene veredicto: la cabecera dice que sale del análisis entero.

- Volver al guardado.



Un mercado que la corrida entera no llegó a ver entra en el selector al fusionar; uno que no está ni en la corrida ni en ninguna re-ejecución no aparece en el selector.

Lo que ya estaba y sigue igual

- **La distribución con el valor real marcado**, su banda, su mediana y la fila de percentiles debajo (la casilla donde cae el real, resaltada): el dibujo base de todos los tests.
- **El cono de equity con la curva real encima**, en cada estudio que lo trae (Monte Carlo, cross-market, estrés).
- **Los selectores filtran**, no recalculan: el estudio guardó todas las combinaciones.
- **Los avisos colorean y nunca eliminan**: los iguales se cuentan ($\times N$), ninguno se esconde.
- **El glosario** va en la misma página, al final, con cada término que usa el estudio.

El informe de lo que ves

«Informe de lo que ves» escribe una página HTML con exactamente lo que hay en pantalla: cada pestaña con la combinación de selectores elegida (dicha en la nota de la pestaña), los mercados lado a lado que elegiste, los avisos y el glosario. Sale del mismo resultado y del mismo dibujante que los informes del estudio (`core.study.render`), no de una segunda copia. En una comparación van las dos columnas, una detrás de otra.

Un ejemplo completo

1. `bin/algoui` . Proyecto `Test_USDJPY_donchianUpperCrossUp_M30` , databank `Retest_Markets_-_Family` , estrategia `Strategy 9.18.85(1)` ; en Estrategia, pestaña **Transferencia**, estudio **Cross-market**. Sale el resultado del 27-09: **DESCARTAR** , 0 de 9 mercados.
2. En la barra de correr, «`U` solo ...» → `EURUSD_DukasM1_the5ers` , con el cajón en `nulls.draws=2000` . A los 11 s el trabajo termina: -> `.../crossmarket/parciales/20260928-072442_EURUSD-DukasM1-the5ers/estrategias/Strategy 9.18.85(1).json` .
3. El resultado vuelve a salir con la tira «RE-EJECUCIÓN PARCIAL (1)». «Ver al lado del guardado»: izquierda el guardado (25.000 sorteos), derecha EURUSD con 2.000. «Ver fusionado»: el resultado entero, EURUSD del 28-09, veredicto **DESCARTAR** intacto.
4. «Informe de lo que ves» → se abre `.../crossmarket/pantalla/Strategy 9.18.85(1)-20260928-072535.html` .

Qué NO te dice

- **Una re-ejecución parcial no es un veredicto.** Ni al lado ni fusionada cambia la etiqueta; para eso se vuelve a correr el estudio entero.
- **El p del KS no dice si la estrategia sirve:** dice si la forma de las operaciones cambió. Un OOS mejor que el IS también da p pequeño.
- **La misma escala de color en las superficies depende del coste** (provisional en Market Surfaces): compara formas entre mercados, no niveles.
- **Monte Carlo ofrece las pruebas de su última corrida entera en ese databank.** Si una estrategia tiene menos operaciones, algún tamaño de bloque puede no existir para ella y la re-ejecución falla con ese motivo en el log.
- **MC Retest (paso 14) no tiene «↺ solo ...»:** cada subprueba es una tarea de SQX, y una re-ejecución parcial nunca lanza SQX. Lo único que la ventana lanza en SQX es la tarea siguiente del workflow, con «Continuar workflow» (capítulo 69).

Si algo falla

- **« no sabe correr una sola subprueba: córrelo entero»:** solo cross-market y Monte Carlo re-ejecutan una subprueba, y solo sobre una estrategia.
- **«monteCarlo: el export de este databank es cross-market...»:** Monte Carlo lee la estrategia en su propio mercado; elige el databank del retest de un solo mercado.
- **«Este resultado no viene de un informe guardado»** al pulsar el informe: la vista no sabe dónde escribir la página (por ejemplo, un resultado de un lote de variantes).
- **La tira de re-ejecuciones no sale:** el demonio es anterior a este cambio; cierra y abre la app.

70. OPERACIÓN — «En marcha», «Registro de búsquedas» y la franja de trabajos

El grupo OPERACIÓN de la barra lateral responde a dos preguntas distintas: ¿qué está corriendo ahora y por dónde va? («En marcha») y ¿cuánto he buscado ya? («Registro de búsquedas», que enseña el Ledger). Abajo a la derecha, en la barra de estado, la franja de trabajos enseña cada trabajo de la ventana con su barra de 0 a 100 %.

Qué pregunta responde

- **«En marcha»:** qué proyecto de SQX corre, qué tarea, cuántas estrategias lleva de cuántas, y cómo va la máquina mientras tanto (memoria del JVM del custodio contra su techo, CPU, RAM libre, cuántos backtests caben todavía). El pulso y la tabla de tareas son **del mismo proyecto y en la misma pantalla**: ya no hay que cruzar dos zonas.
- **«Registro de búsquedas»:** cuántas veces se ha mirado cada estudio (activo + timeframe + familia) para reducir una población, con qué criterio, cuántas entraron y cuántas salieron, qué tramos del histórico se han gastado, y si la puerta ciega del paso 20 está abierta. Los filtros que aplicas en «Proyecto» salen aquí como una búsqueda más.
- **La franja de trabajos:** qué análisis de Python lanzó la ventana, por dónde va cada uno (barra), cuáles esperan turno, y qué proyecto de SQX corre en los workers.

Cuándo lo usas, y cuándo no

Lo usas mientras corre un paso largo en el custodio, para no abrir SQX y no mandarle nada más que la línea de estado; al volver a una sesión, para ver qué acabó; y antes de decidir otro filtro o otra búsqueda, para ver en el Registro cuánto llevas gastado en ese estudio.

No lo usas para lanzar ni parar SQX: aquí no hay botón, a propósito. Arrancar y parar sigue siendo `bin/sqx-worker.sh` y las skills; lo único que la ventana lanza en SQX es la tarea siguiente, con «Continuar workflow» en la zona Proyecto (capítulo 69), y ese trabajo se sigue aquí. El **×** de la franja cancela un trabajo **de Python** de la ventana; los chips de SQX no tienen **×**. El Registro tampoco se edita: es de solo lectura.

Antes de empezar

Nada. Vale con el maestro abierto (a él no se le pide nada nunca) y con el custodio en mitad de un trabajo: lo único que la ventana le pide es `-project action=status`, y solo mientras su log dice que un proyecto corre (es el único comando que el custodio puede recibir entre `start` y `collect`, porque no sincroniza nada).

Cómo se ejecuta

```
bin/algoui --zone "En marcha"
bin/algoui --zone "Registro de búsquedas"
```

O desde la barra lateral, grupo OPERACIÓN. La franja de trabajos está siempre a la vista.

control	dónde	qué hace
install	«En marcha»	custodian (SQX_w2), conductor (SQX_w1) o master (solo se lee)
proyecto	«En marcha»	cualquier proyecto de ese install. Al abrir se elige solo el que corre en un worker (el custodio primero); si nada corre, el proyecto elegido en la ventana; si tampoco, el primer Test_ / Trade_ del custodio. Cuando lo eliges tú, la zona deja de elegir
Leer ahora	«En marcha», pulso	una lectura del pulso sin esperar al minuto
estudio	«Registro de búsquedas»	activo + timeframe + familia; por defecto el último escrito
×	franja	cancela ese trabajo de Python (y todos sus procesos hijos)
Recargar	pie de la barra lateral	vuelve a leer las zonas que se cargan una vez; su tooltip dice cuáles

El tooltip de **Recargar** dice, literalmente: «Vuelve a leer del demonio: Cobertura, Plantillas, Paletas, Activos, Proyectos, En marcha. Las demás zonas leen solas al abrirlas o al cambiar la selección, y no necesitan este botón.»

«En marcha» refresca la tabla cada tres segundos y el pulso cada minuto, **solo mientras está a la vista**. La franja lee los trabajos cada dos segundos y los proyectos de SQX cada dieciséis.

Qué produce

Nada. No escribe ni un fichero: lee /proc , el log de SQX de cada install, sus project.cfx y databanks, los logs de los trabajos de la ventana (AlgoData/logs/ui/) y el ledger (AlgoData/ledger/*.jsonl).

Cómo se lee el resultado

AlgoProject

BIBLIOTECA

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Configuración SQX

Datos

PROYECTO

Proyectos

Proyecto

Estrategia

OPERACIÓN

En marcha

Registro de búsquedas

PORTFOLIOS

Portfolios

Recargar

Proyectos > elige proyecto > elige estrategia

En marcha

INSTALL

custodian

PROYECTO

Test_USDJPY_donchianUpperCrossUp_M30

El custodio está parado: el pulso solo lee la RAM libre. La tabla es de Test_USDJPY_donchianUpperCrossUp_M30 en custodio - SQX_w2.

PULSO DEL CUSTODIO - SQX_w2 - /proc, LOGS Y SU LINEA DE ESTADO

Leer ahora

08:41:38 custodio parado | libre 117 GB

Proyecto	Tarea	Ritmo	Falta
-	-	-	-
JVM / techo	Caben	Pendiente	Avance de
-	-	-	-

ULTIMAS LECTURAS - EN MEMORIA, SE PIERDEN AL CERRAR

08:41:38 custodio parado | libre 117 GB

parado el log de hoy no tiene un start de este proyecto - sin log de hoy

LAS TAREAS DEL PROYECTO, EN EL ORDEN EN QUE SQX LAS CORRE

#	Tarea	Tipo	En este start	Estado	Hechas / total	Avance	Tiempo	Por estrategia	Databank de salida	En disco
1	CONSTRUCCION	Build	-	inactiva	-	-	-	-	Results	-
2	OOS	Retest	-	inactiva	-	-	-	-	OOS	-
3	Retest Markets - Family	Retest	-	inactiva	-	-	-	-	Retest Markets - Family	-
4	MCR 1 Bar	Retest	-	inactiva	-	-	-	-	MCR 1 Bar	-
5	MCR 2 Spread	Retest	-	inactiva	-	-	-	-	MCR 2 Spread	-
6	MCR 3 Slippage	Retest	-	inactiva	-	-	-	-	MCR 3 Slippage	-
7	MCR 4 MinDist	Retest	-	inactiva	-	-	-	-	MCR 4 MinDist	-
8	MCR 5 Params	Retest	-	inactiva	-	-	-	-	MCR 5 Params	-
9	MCR 6 Exits	Retest	-	inactiva	-	-	-	-	MCR 6 Exits	-
10	MCR 7 OHLC	Retest	-	inactiva	-	-	-	-	MCR 7 OHLC	-
11	MCR 8 Stress	Retest	-	inactiva	-	-	-	-	MCR 8 Stress	-
12	SPP TS	Retest	-	inactiva	-	-	-	-	SPP TS	-

EL LOG DEL INSTALL, ÚLTIMAS LINEAS DE PROGRESO

sin líneas de progreso hoy

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

TRABAJO 0 en marcha · 0 en cola

Arriba, el pulso del custodio. La línea grande es la que el dueño eligió el 2026-09-25: 13:38:32 3987 de 15000 | JVM 63.2 GB | CPU 6845% | libre 55 GB | caben 900 más . Cada cifra de la rejilla tiene un tooltip que dice de dónde sale. En rojo, los avisos: RAM libre bajo 15 GB, el JVM al 90 % de su techo, más backtests pendientes de los

que caben, o el log de SQX callado más de diez minutos. La cifra «**avance de**» dice de dónde sale la cuenta:

avance de	qué significa
un log de trabajo (... - variants.log)	el run lo lanzó la ventana y cuenta el run entero (PROGRESS n de N); hay ritmo y minutos que faltan
estado del custodio · <tarea>	el run se lanzó fuera de la ventana (terminal, otra sesión): la cuenta es de la tarea en curso, del estado del worker. No hay ritmo ni «falta», porque la cuenta de una tarea sobre los minutos del proyecto no sería un ritmo
— y la línea dice avance ?	nada da cuenta: ni un trabajo de la ventana ni el estado del worker

La frase bajo el título dice si el pulso y la tabla son el mismo run. En ámbar, cuando no: el custodio corre un proyecto y tú miras otro (o miras el conductor). Elige arriba el que corre para verlos juntos.

Abajo, las tareas del proyecto, en el orden en que SQX las corre. Cada **start** de un proyecto es un paso del workflow: **stage** deja activa solo la tarea del paso y SQX marca las demás como saltadas.

estado	qué significa
en curso	la última tarea que escribe en el log; en negrita, con su barra ámbar en «Avance»
hecha	el log vio Task finished en este start; barra verde llena
hecha antes	inactiva en este start, pero su databank tiene estrategias: la hizo un start anterior
en cola	activa en este start y aún no empezada
saltada	inactiva en este start y con el databank vacío
inactiva	apagada, sin start en marcha

La barra de la tarea en curso es el porcentaje que SQX escribe en su log cuando lo da (las tareas walk-forward); si no, hechas ÷ total. Un build no tiene total: su barra solo se mueve, sin cifra. «En disco» son ficheros .sqx : SQX los escribe al sincronizar, así que la salida de la tarea en curso va con retraso.



La franja de trabajos: N en marcha · N en cola , y un chip por trabajo con su barra. La barra es la última línea PROGRESS <0-100> que imprimió el trabajo, y 100 % al acabar bien. **Un estudio que no imprime PROGRESS se queda en 0 % hasta que acaba** (es el caso de profitShape , que tarda uno o dos segundos por estrategia); los que sí lo imprimen (la puerta, el cribado de snooping, isOos, las superficies...) mueven la barra. Los que corren más avanzados van primero; más de cuatro se resumen en «+N». Un chip ámbar SQX custodio · <tarea> · 7 / 20 es un proyecto de SQX en un worker, con la misma barra que «En marcha»: no se cancela desde aquí.

AlgoProject

BIBLIOTECA

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Configuración SQX

Datos

PROYECTO

Proyectos

Proyecto

Estrategia

OPERACIÓN

En marcha

Registro de búsquedas

PORTFOLIOS

Portfolios

Recargar

Proyectos > elige proyecto > elige estrategia

Registro de búsquedas ESTUDIO USDJPY_M30_donchianUpperCrossUp

Una línea por búsqueda que miró datos y redujo una población, en todo el estudio (activo + timeframe + familia). Sólo lectura: el ledger no se edita. Tres supervivientes de 10.000 no valen lo que tres de 50.

Por qué importa: cada mirada a los datos es una prueba más, y cuantas más pruebas, más fácil es que el mejor resultado sea suerte. Este registro las cuenta todas – las de los estudios, las del SQX y los filtros que aplicas en la ventana («lanzado por: ventana») – para que el Sharpe desinflado y la puerta ciega del paso 20 sepan cuánto se ha buscado.

1 de 6 búsquedas de este estudio son filtros aplicados o borrados a mano en la ventana; su criterio sale en violeta.

PUERTA CIEGA DEL PASO 20 · 17 pendiente · 18 pendiente · 19 pendiente
ledger: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala

PROBADO EN TOTAL · N = 200 candidatos en 1 búsqueda(s) · $\sigma = 0.0223$ (per_period) – la σ que necesita el Sharpe desinflado

Embudo Historia gastada Búsquedas

Cuándo	Paso	Tramo	Entran	Salen	Conserva	Criterio
2026-09-27 20:30	18.5	build	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-27 20:30	18.5	oos1	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-27 20:37	18.5	build	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-27 20:37	18.5	oos1	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-28 07:18	8	oos1	200	183	91.5 %	Net profit (005) > 0.0 AND spread.operaciones [005] ≥ 30.0
2026-09-28 07:24	8	oos1	200	200	100 %	snoopingScreen/stepm

registry.csv · runs.csv · library/ — 7 filas x 6 timeframes · daemon en 127.0.0.1

TRABAJOS 0 en marcha · 0 en cola

El Registro de búsquedas abre con lo que es y por qué importa, en un recuadro: cada mirada a los datos es una prueba más, y cuantas más pruebas, más fácil que el mejor resultado sea suerte. Tres supervivientes de 10.000 no valen lo que tres de 50. Debajo:

- cuántas de las búsquedas del estudio son **filtros de la ventana** (lanzado por: ventana); su criterio sale en violeta en «Embudo» y la fila entera en «Búsquedas»;
- la **puerta ciega del paso 20**: verde cuando 17, 18 y 19 están hechos, ámbar mientras falte alguno;
- **probado en total**: N candidatos y la σ que necesita el Sharpe desinflado, o por qué no hay;
- tres pestañas: **Embudo** (entran, salen y qué parte conserva cada búsqueda: rojo si conserva menos de la mitad, ámbar si conserva menos del todo), **Historia gastada** (cada tramo, su ventana, cuántas veces se ha leído y para qué pasos está reservado: verde si sigue virgen) y **Búsquedas** (cada fila completa; al pasar el ratón, sus umbrales y su nota).

Un ejemplo completo

La sesión de las capturas, el 2026-09-28, tal cual:

1. bin/algoui --zone "En marcha". Ningún worker corría nada y no había proyecto elegido, así que la zona eligió el custodio y su primer proyecto Test_ : Test_USDJPY_donchianUpperCrossUp_M30 . La frase: «El custodio está parado: el pulso solo lee la RAM libre. La tabla es de Test_USDJPY_donchianUpperCrossUp_M30 en custodio · SQX_w2.»
2. Pulso: 07:22:14 custodio parado | libre 116 GB ; todas las cifras del JVM en «—». (La captura de arriba se rehízo a las 08:41 sobre la ventana definitiva: 08:41:38 custodio parado | libre 117 GB .)
3. Tabla: las 27 tareas del proyecto (CONSTRUCCION, OOS, Retest Markets - Family, MCR 1 Bar... MCR 8 Stress, SPP IS...), todas «inactiva», porque ningún start de hoy las tocó.
4. Desde el raíl del proyecto se lanzó profitShape (paso 8) sobre el databank 005 : tres trabajos, uno por estrategia. La franja: TRABAJOS 3 en marcha · 0 en cola y tres chips profitShape con su barra a 0 % (profitShape no imprime PROGRESS); a los dos segundos, los tres a 100 % y fuera de la franja.

5. Registro de búsquedas , estudio USDJPY_M30_donchianUpperCrossUp : «1 de 6 búsquedas de este estudio son filtros aplicados o borrados a mano en la ventana». En el Embudo, cuatro filas del paso 18.5 (marketSurfaces, 150 → 150, 100 %) y la del filtro de la ventana 2026-09-28 07:18 · 8 · oos1 · 200 → 183 · 91.5 % · Net profit (00S) > 0.0 AND spread.operaciones [00S] ≥ 30.0 , en ámbar porque conserva menos del todo. La puerta ciega: «17 pendiente · 18 pendiente · 19 pendiente». La sexta fila, 07:24 · 8 · snoopingScreen/stepm · 200 → 200 , es el cribado SPA/StepM lanzado desde el raíl.

Qué NO te dice

- **Cuánto falta con exactitud** en un run lanzado fuera de la ventana: la cuenta es de la tarea en curso, no del proyecto, y hechas × tiempo por estrategia es una estimación.
- **De quién es el start.** El log nombra el proyecto solo cuando el start llega por la API; un start desde la GUI del maestro no se atribuye.
- **Si el resultado es bueno.** «En marcha» cuenta; el Registro cuenta miradas. Ninguno juzga.
- **Las búsquedas que nadie apuntó.** Un análisis corrido a mano que no escribe su fila no está en el Registro; el WFC y el CSCV escriben la suya desde el 27-09-2026.

Si algo falla

- **«avance ?» con el custodio corriendo:** ni un trabajo de la ventana ni el estado del worker dan cuenta — por ejemplo, un build (no tiene total y su estado no siempre trae «Strategies generated») o un proyecto que el log nombra pero ya no está en disco.
- **La frase en ámbar:** no es un error. Miras un proyecto distinto del que corre el custodio.
- **«sin log de hoy»:** el install no ha escrito nada hoy. La zona solo lee el fichero del día.
- **Un trabajo se queda en 0 % y de golpe pasa a 100 %:** ese estudio no imprime PROGRESS.
- **«demonio no responde»:** el demonio de la ventana se ha caído; ciérrala y ábrela otra vez.

71. Portfolios — las estrategias archivadas, e «Importar»

La zona **Portfolios** de la ventana, el cuarto grupo de la barra lateral. Hoy hace una cosa: enseña las estrategias que has guardado con «Archivar» y, con «Importar», abre cualquiera de ellas en la página de Estrategia **tal como se guardó**, sin recalcular nada. Es la puerta por la que las estrategias entrarán en una cartera; la cartera en sí todavía no está construida.

Qué pregunta responde

- **¿Qué estrategias tengo guardadas «como oro en paño»?** Una fila por estrategia: su nombre, el activo, el timeframe, en qué paso del workflow estaba, cuántas versiones tiene, cuándo se archivó la última y **la cuenta del Ledger** de ese día.
- **¿Qué guarda cada versión y qué no?** Al elegir una versión, a la derecha: los estudios que trae por databank, lo que no se pudo archivar y por qué, y su procedencia (la ficha de costes del activo de ese día, el código con que se hizo, de dónde salió el `.sqx`).
- **¿Cómo se veía esta estrategia el día que la archivé?** «Importar» abre su página de Estrategia con las dos curvas, las estadísticas, los metadatos y cada estudio archivado, leídos del archivo.

Cuándo lo usas, y cuándo no

Lo usas para volver a mirar una estrategia que ya pasó la secuencia sin rehacer ningún test (el monkey de 25 000 corridas no se vuelve a correr), para comparar dos versiones de la misma estrategia archivadas en pasos distintos, y, cuando exista el módulo de cartera, para elegir qué entra en ella.

No lo usas para calcular nada. Una estrategia importada **no corre ningún estudio ni encola ningún trabajo**: los botones «calcular» de la ficha contestan «archivada: aquí no se calcula nada». Para calcular, abre la estrategia desde su proyecto, en Proyectos. Tampoco importa desde un databank vivo: por ahora solo desde el archivo (decisión del dueño).

Antes de empezar

- La ventana y su demonio (capítulo 35). No hace falta SQX ni ningún worker: todo sale de `AlgoData/archive/`.
- Al menos una estrategia archivada. Se archiva con el botón «Archivar» de la página de Estrategia (capítulo 68) o con `python3 -m core.archive archive ...` (capítulo 64).

Cómo se ejecuta

Dentro de la ventana: **PORTFOLIOS** → **Portfolios**. Elige una estrategia en la tabla, una versión en la lista de la derecha y pulsa «Importar esta versión».

O `bin/algoui --zone Portfolios`. Para verla sin la ventana entera, o sacar sus capturas sin pantalla, se abre sola en su propia ventana, con el demonio arrancado (`bin/algoui` lo arranca):

```
python3 -m ui.desktop.portfolios.preview
python3 -m ui.desktop.portfolios.preview --shot /tmp/portfolios --version 2026-09-27T2039
```

flag	obligatorio	qué hace
--shot DIR	no	guarda la lista (1-portfolios.png) y la estrategia importada (2-importada.png) en DIR y sale
-- identity	no	la estrategia que importa la captura; sin él, la primera de la tabla
--version	no	la versión que importa la captura; sin él, la más nueva
--port	no	el puerto del demonio; sin él, el de la ventana (8765)

Tarda un par de segundos. No toca SQX: se puede abrir con cualquier install arrancado.

Qué produce

Nada en disco. Lee AlgoData/archive/<identidad>/<versión>/ a través del demonio (/api/archive/list , /api/archive/show y las rutas de la ficha con source=archive). El archivo es de solo lectura: importar no lo cambia.

Cómo se lee el resultado

PORTFOLIOS

Estrategias archivadas

Las estrategias archivadas con «Archivar» en la ficha de Estrategia. «Importar» las abre tal como se guardaron: todos los paneles leen el archivo, no se recalcula nada ni se encola ningún trabajo. Por ahora solo se importa desde el archivo: desde un databank vivo, cuando el dueño lo pida.

1 estrategia archivada, 4 versiones.

Estrategia	Activo	Timeframe	Paso	Versiones	Última versión	N del Ledger	Búsquedas	Versiones
Strategy 10.11.79	USDJPY	M30	16.5	4	2026-09-28 07:24	200	6	Test_USDJPY_don

2026-09-28 07:24 · paso 16.5 · prueba de aceptación del frente F4 (botón Archivar de

2026-09-27 20:39 · paso 17 · tercera versión: lista lo no archivado

2026-09-27 20:25 · paso 17 · segunda versión, aceptación

2026-09-27 20:24 · paso 16 · prueba de aceptación del frente A

Strategy 10.11.79 · USDJPY M30 · paso 17 · versión 2026-09-27T2039

archivada el 2026-09-27 20:39 · desde Results · proyecto Test_USDJPY_donchianUpperCrossUp_M30

«tercera versión: lista lo no archivado»

Lo que trae (se ve al importar, sin recalcular nada):

· CrossTF: Cross-timeframe (Transferencia, paso 12)

· MCR_All: MC Retest (Rotura, paso 14)

· Results: Puerta IS/OOS (Cribado, paso 8), Panel IS/OOS (Cribado, paso 8), Spread real de Darwinex (Datos, paso 8)

· la ficha de backtest (IS/OOS, salidas), las dos curvas y los metadatos del .sqx

Lo que no se archivó y por qué:

· OOS/2026-09-27/isOos: sin filas por estrategia: solo resultado de población o tablas sin columna strategy

· Retest_Markets - Family/2026-09-27/crossmarket: sus filas no llevan identidad: emparejar por nombre sería adivinar

· SPP_IS/2026-09-27/spp: sus filas no llevan identidad: emparejar por nombre sería adivinar

· workflow-steps.csv: no nombra estrategias por identidad

Procedencia

N = — candidatas puntuadas en todas las búsquedas del estudio · 4 filas del Ledger (estudio del Ledger USDJPY_M30_donchianUpperCrossUp)

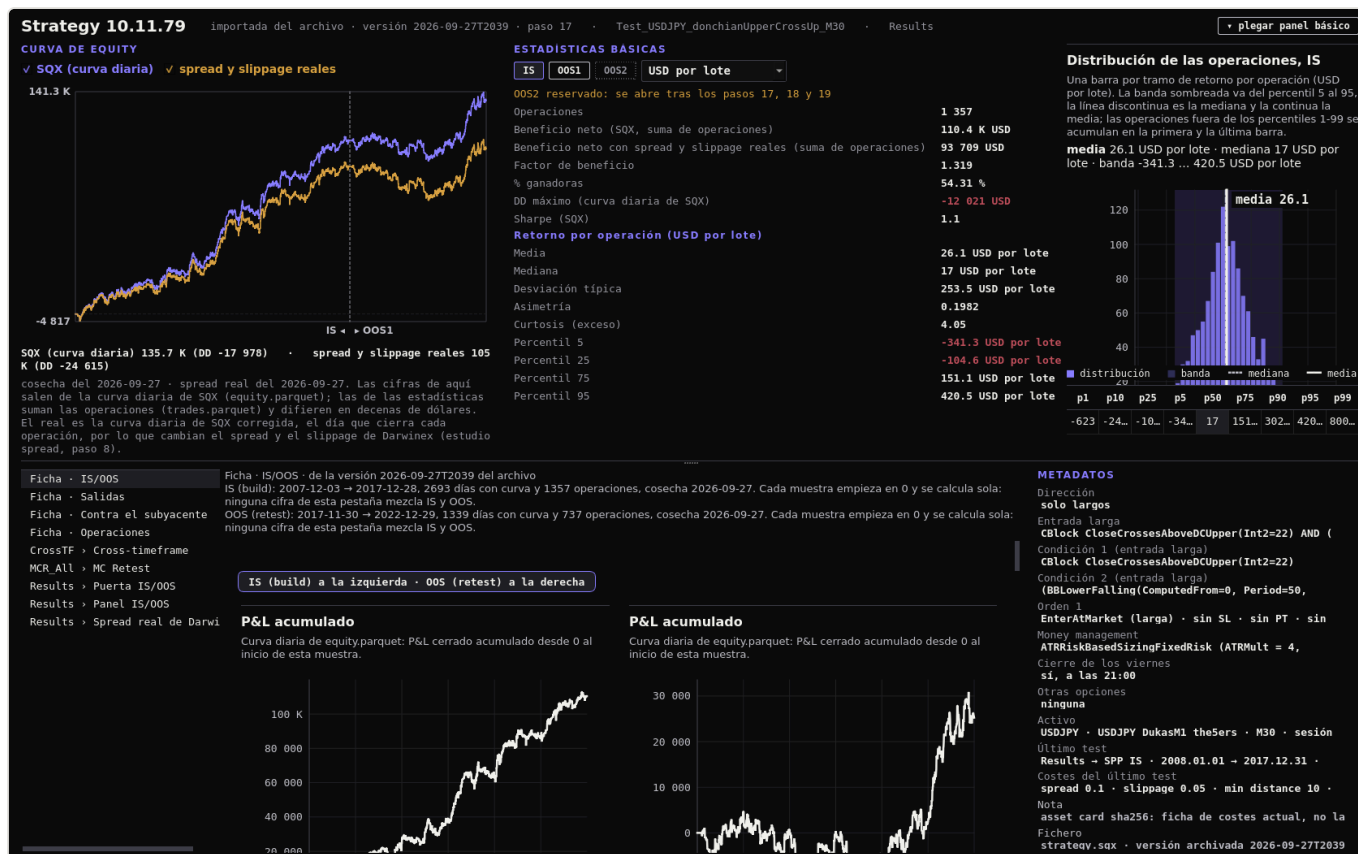
ficha de costes de USDJPY congelada ese día · código 7d038a6-dirty

el .sqx se copió de un export (no del install)

Importar esta versión

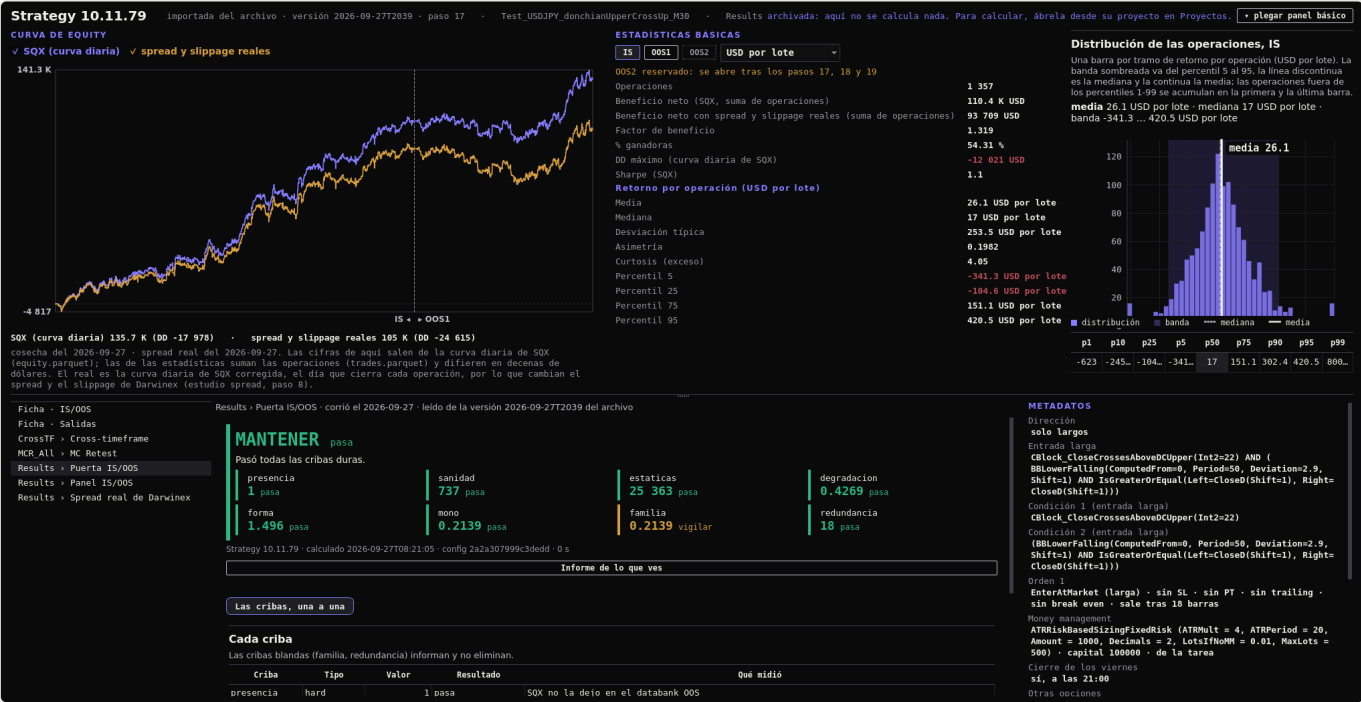
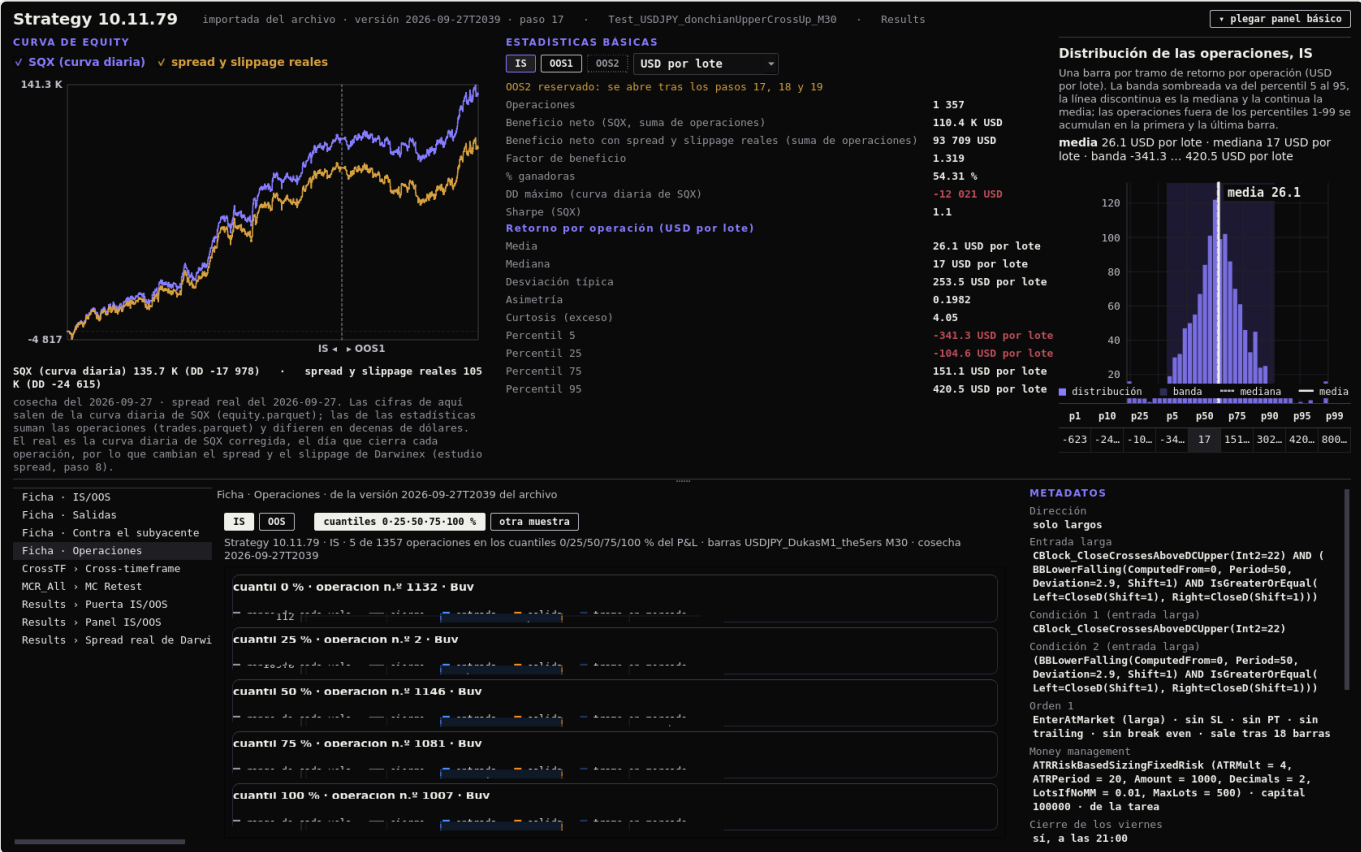
- Tabla de la izquierda.** Una fila por estrategia. **N del Ledger** es cuántas candidatas se puntuaron en **todas** las búsquedas del estudio del Ledger (la plantilla en ese activo y timeframe), no solo en la búsqueda que encontró esta estrategia, según el Ledger el día que se archivó su última versión. Es lo que hará falta para deflactor su Sharpe (un Sharpe alto entre 200 probadas vale menos que entre 5). **Búsquedas** son todas las filas del Ledger de ese estudio: cada build, retest o filtro que miró sus datos, apuntara candidatas o no. Si hay búsquedas pero ninguna apuntó candidatas, N sale como «—», no como 0. La identidad de la estrategia no se imprime: está en el tooltip del nombre.

- **Versiones.** La más nueva arriba, con su fecha, el paso y la nota que escribiste al archivar. Una versión nunca se sobrescribe: archivar otra vez crea otra.
- **Lo que trae / Lo que no se archivó.** Los estudios congelados por databank, y los que se quedaron fuera con su motivo (por ejemplo, un crossmarket cuyas filas no llevan identidad: emparejarlas por nombre sería adivinar). Una versión antigua, anterior a esa lista, lo avisa.
- **Procedencia.** La cuenta del Ledger de esa versión (puede ser distinta de la de la tabla), la ficha de costes congelada y el código.



La página importada es la de Estrategia (capítulo 68) con tres diferencias: la cabecera dice «importada del archivo · versión ... · paso ...»; no hay botón «Archivar»; y debajo del panel básico, en vez de las pestañas de familias, hay una lista con la **Ficha** (IS/OOS, Salidas, Contra el subyacente y Operaciones) y cada estudio archivado, por databank. Un estudio que solo escribe el resultado del databank entero (el spread real, el cross-timeframe) enseña ese, y la línea de arriba lo dice.

Lo que no sale en una estrategia importada: la pestaña «Lote» (las variantes de la madre se leen en vivo por su nombre y el archivo no las congela); las **pestañas de familias** de la página viva con sus **puntos de estado**, el **historial** de corridas, «comparar» (dos corridas o dos estrategias) y la **barra de correr**, porque todas leen el proyecto vivo; y el botón «Informe de lo que ves», que escribiría su página en la carpeta reports/ viva.



Un ejemplo completo

La estrategia Strategy 10.11.79 del proyecto Test_USDJPY_donchianUpperCrossUp_M30 tiene cuatro versiones archivadas (pasos 16, 17, 17 y 16.5). Se elige la de 2026-09-27 20:39 (paso 17) y se pulsa «Importar esta versión».

La prueba de que nada se recalculó y de que se ve lo mismo que en vivo, hecha el 2026-09-28 comparando cada ruta con source=archive contra la misma ruta en vivo:

```

IGUAL      /api/result CrossTF crossTF
DISTINTO   /api/result MCR_All mcRetest   diferencias: $.result.partials solo en vivo
DISTINTO   /api/result Results gate       diferencias: $.result.partials solo en vivo
DISTINTO   /api/result Results is0os      diferencias: $.result.partials solo en vivo
IGUAL      /api/result Results spread
IGUAL      /api/tearsheet Results IS
IGUAL      /api/tearsheet Results 00S
{"jobs":[]} ← ningún trabajo nuevo en el demonio
hijos: []   ← el demonio no lanzó ningún proceso

```

La única diferencia es `partials`, una clave que la ventana empezó a servir el 28 de septiembre, después de archivar: el archivo guarda lo que se veía **ese día**. La versión de 2026-09-28 07:24, archivada ya con esa clave, sale igual en todas las rutas.

Qué NO te dice

- **No te dice si la estrategia sigue valiendo hoy.** Enseña lo que se calculó el día de archivar, con la configuración y los costes de ese día. Si un estudio se ha vuelto a correr después, o la ficha de costes del activo ha cambiado, la versión archivada no lo sabe.
- **No compone una cartera.** Correlaciones entre curvas, drawdown agregado, reglas de cuenta fondeada: nada de eso existe aún.
- **La cuenta del Ledger es la de ese día**, por estudio (plantilla), no por estrategia. Si el fichero del Ledger se movió, cuenta 0 búsquedas; el manifiesto guarda su huella para verlo.

Si algo falla

- **«El archivo está vacío»:** no hay ninguna estrategia archivada. Archiva una desde su ficha.
- **«No hay demonio en el puerto 8765»** (la vista previa): arranca la ventana con `bin/algoui` o el demonio con `python3 -m ui.daemon.serve`.
- **«No pude leer el archivo: ...»:** una carpeta de `AlgoData/archive/` se tocó a mano o quedó a medias. Las carpetas a medias empiezan por punto y no se listan; cualquier otra cosa, dilo.

35. La ventana — cómo se abre, el mapa de sus zonas, y las plantillas

La ventana. No es una pestaña del navegador ni un panel más: es la aplicación de escritorio que envuelve SQX y Python, la única de esta máquina. Este capítulo cuenta cómo se abre y las cuatro primeras zonas de BIBLIOTECA, las de las **plantillas**: Cobertura, Plantillas, Nueva plantilla y Paletas. El mapa entero de la barra lateral está al final, en «Las demás zonas».

Qué pregunta responde

Tres, y las tres se contestaban hasta hoy abriendo dos CSV en Excel y una carpeta en el explorador:

- **¿Qué plantillas tengo, de qué tipo, y en qué mercados y timeframes las he probado?**
- **¿De qué tengo poco?** Qué arquetipo no he tocado, qué timeframe está vacío, qué idea probé una sola vez y no volví a mirar.
- **¿Cómo empiezo una plantilla nueva sin olvidarme de ninguna de las preguntas?** La ventana te entrevista, escribe el borrador del brief y te deja el comando listo para pegar en Claude Code.

Cuándo lo usas, y cuándo no

Lo usas para mirar la librería, para decidir qué construir a continuación, para anotar el veredicto de una corrida en cuanto lo tengas, y para redactar la idea de una plantilla nueva.

No lo usas para construir nada. Estas zonas **no arrancan SQX, no instalan bloques y no emiten ningún .sqx**. Todo eso lo hace `/sqx-strategy-template` y `/template-run`, que escriben en instalaciones de verdad y son el carril del conductor y el del custodio. Una ventana que escribiera en `SQX_w1` por su cuenta rompería la tabla de carriles de `sqx/CLAUDE.md` en cuanto hubiera dos sesiones abiertas.

Tampoco es donde se editan los CSV. Escribe tres cosas y solo tres: el **estado** de una plantilla, su **nota**, y el **veredicto** de una corrida.

Antes de empezar

- `pip install -r requirements.txt` — la app añade PySide6, FastAPI, uvicorn y httpx.
- Nada más. No hace falta SQX abierto ni cerrado, no hace falta ningún worker arrancado, y no toca ninguna instalación. Si `~/Desktop/AlgoData/templates/` no existe todavía, la ventana abre vacía en vez de fallar.

Cómo se ejecuta

Tres formas, las tres hacen exactamente lo mismo:

dónde	qué haces
el escritorio	doble clic en el icono AlgoProject
el menú de aplicaciones	buscas <i>AlgoProject</i>
cualquier terminal, desde cualquier carpeta	<code>algoui</code>

```
algoui
```

Las tres acaban en `bin/algoui`, que no hace más que llamar a `python3 -m ui.desktop.launch` desde la raíz del proyecto. El icono es la propia matriz de cobertura en miniatura, y es el mismo que sale en la barra de tareas y en el alt-tab.

`algoui` funciona desde cualquier sitio porque `~/.local/bin/algoui` es un enlace al del repositorio y el script resuelve el enlace antes de buscar la raíz. Si alguna vez mueves la carpeta del proyecto, rehaz el enlace y la línea `Exec=` de `~/.local/share/applications/algoproject.desktop`: las dos llevan la ruta escrita.

En Windows la ventana funciona igual, sin SQX: doble clic en `bin\algoui.cmd`, o `bin\algoui.cmd` desde una consola. Hace lo mismo que `bin/algoui` y además activa el modo UTF-8 de Python, sin el cual los textos en español se leen en cp1252. Lo único que cambia sin SQX es la ficha de una plantilla: sus huecos atados a un grupo salen como «*atado a un grupo que esta instalación no tiene*», porque los nombres de los grupos viven en el `blockGroups.xml` de la instalación. Cómo montarla en un PC nuevo está en el capítulo de GitHub.

flag	obligatorio	qué hace
--zone	no	la zona que se abre, por su nombre en la barra lateral (<code>algoui --zone Proyectos</code>); sin él, Cobertura
--shot	no	guarda una captura PNG de la ventana en esa carpeta y se cierra (capítulo 60)
--port	no	el puerto del demonio; sin él, <code>ui_port</code> de <code>config/machine.yaml</code> (8765). Para pruebas y capturas usa otro puerto: el 8765 es el de tu ventana (capítulo 60)

Si algo va mal y la ventana no abre, el demonio se arranca por separado para ver su error:

```
python3 -m ui.daemon.serve
```

Dos procesos, no uno. `bin/algoui` levanta el demonio si no está ya escuchando y abre la ventana. Al cerrar la ventana, el demonio se apaga **solo si lo arrancó ella**: dos ventanas comparten uno, y matarlo le quitaría la librería a la otra. El demonio escucha en 127.0.0.1 y en ningún sitio más.

Qué produce

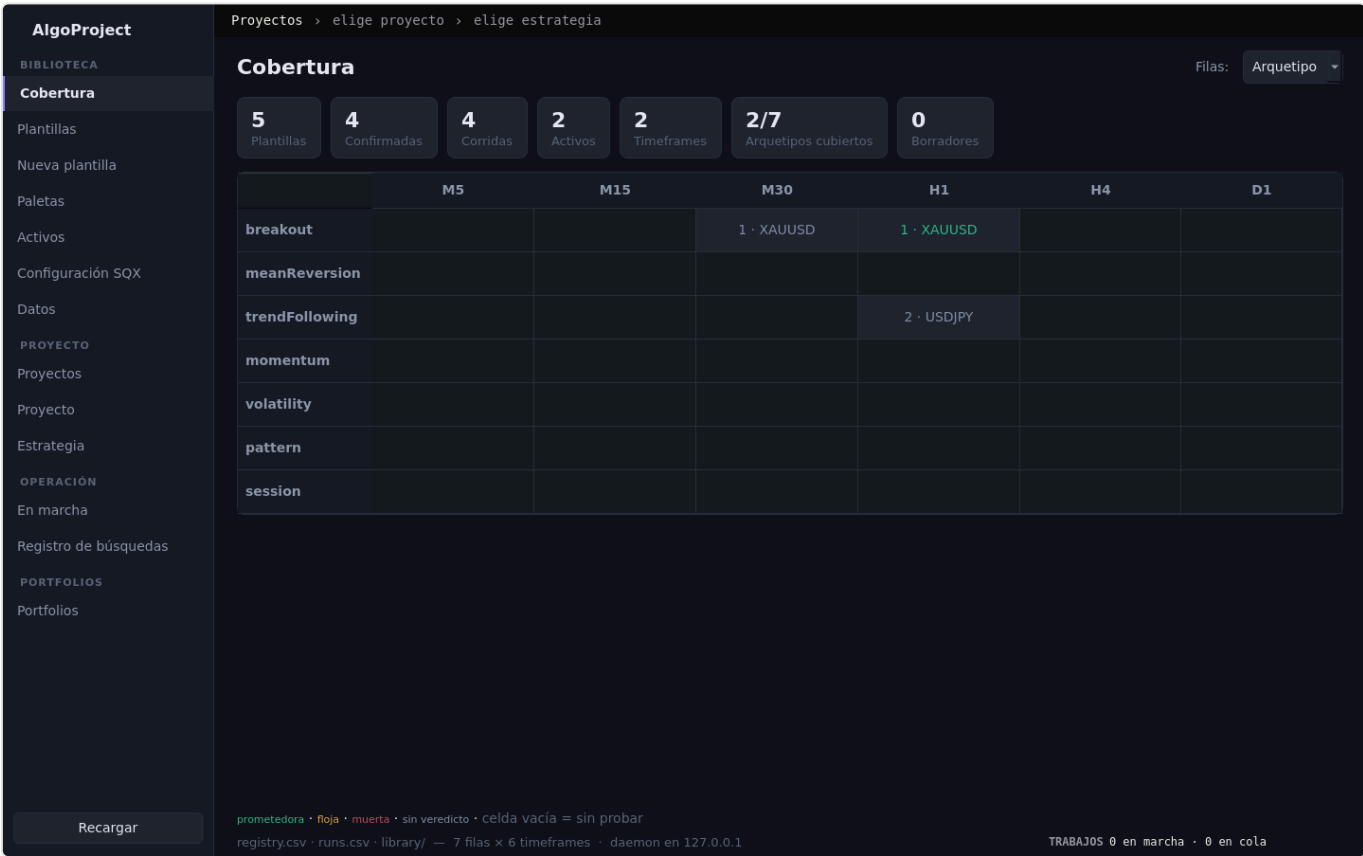
ruta	qué es	cuándo se escribe
AlgoData/templates/registry.csv	la fila de la plantilla, con su <code>status</code> y su <code>notes</code>	al pulsar Guardar en la ficha
AlgoData/templates/runs.csv	la fila de la corrida, con su <code>verdict</code>	al cambiar el desplegable de veredicto
AlgoData/templates/drafts/<nombre>.json	el borrador del brief que redacta el chat	al terminar la entrevista
sqx/blocks/palettes/<familia>.yaml	la paleta de bloques de un arquetipo	al pulsar Guardar en Paletas

Los dos CSV los escribe `sqx/templates/registry.py`, **la misma función que usa la línea de comandos**. La ventana no tiene una segunda forma de escribir un CSV: dos escritores para un fichero es como se descuadra una columna.

⚠ **La ventana escribe paletas, nunca la taxonomía.** `sqx/blocks/taxonomy.yaml` lleva las etiquetas y lo rellena otra persona (`docs/encargos/6-taxonomia-bloques.md`); desde aquí solo se escriben los `overrides` de una paleta. Son dos ficheros distintos a propósito.

Cómo se lee el resultado

Cobertura — la primera pantalla



Arriba, siete contadores. **Todos explican qué cuentan al pasar el ratón por encima**, porque un número que nadie sabe definir es un número con el que nadie debería decidir. El que más dice es *Arquetipos cubiertos*: **2/7** significa que de los siete arquetipos que el catálogo conoce, has tocado dos. Esa distancia **es** la respuesta a «de qué tengo poco».

Debajo, la rejilla. Las columnas son siempre los timeframes —es el eje con vocabulario corto, fijo y ordenado—. Las filas las eliges arriba a la derecha: **Arquetipo**, **Activo** o **Plantilla**.

Cada celda dice cuántas corridas hay y en qué mercados, y **su color es el mejor veredicto que hay dentro**, no un promedio. Es deliberado: una celda es una invitación a abrirla, y promediar haría que una celda con una corrida prometedora y dos muertas argumentase en contra de mirarla.

color	significa
verde	alguna corrida quedó como promising
ámbar	la mejor quedó como weak
rojo	todas dead
gris	probada, sin veredicto escrito todavía
celda vacía	sin probar . Es el hueco que la matriz existe para enseñar

Las filas de arquetipo que nadie ha tocado **siguen apareciendo**, vacías. Si se generaran a partir de los datos desaparecerían, y con ellas justo la información que buscas.

Un clic en una celda con datos abre esa plantilla.

Plantillas — la ficha

AlgoProject

BIBLIOTECA

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Configuración SQX

Datos

PROYECTO

Proyectos

Proyecto

Estrategia

OPERACIÓN

En marcha

Registro de búsquedas

PORTFOLIOS

Portfolios

Recargar

Proyectos > elige proyecto > elige estrategia

Plantillas

Buscar por nombre, arquetipo o activo

Todos los estados

donchianUpperCrossUp

breakout — sin corridas

crossAboveHMA v1

trendFollowing — sin corridas

emaCloseCrossUp

trendFollowing — USDJPY-H1

emaCloseAbove

trendFollowing — USDJPY-H1

keltnerUpperCrossUp

breakout — XAUUSD-H1 XAUUSD-M30

keltnerUpperCrossUp

buildConfirmed breakout market_long authored 2026-09-22

template.sqx · brief.md · deps/ (2)

close cruza por encima de la banda superior de Keltner (fijo) AND una condicion aleatoria libre

Estado: buildConfirmed

Nota — por qué está en este estado, qué falta

Guardar

Qué rellena el builder

1 hueco(s) libre(s): la paleta sí los gobierna. Otros 1 están atados a un grupo y se le escapan.

RandomCondition1 (condition) atado a un grupo que esta instalación no tiene

RandomCondition2 (condition) libre — la paleta lo gobierna

Fijos en la plantilla, fuera de toda paleta: MarketPositionIsLong

Corridas

Activo	TF	Fecha	Proyecto	Constr.	Retenidas	Veredicto
XAUUSD	H1	2026-09-22	smoke_kelt...	30	30	prometedora
XAUUSD	M30	2026-09-23	XAU_ISO...	120	115	sin veredicto

Brief

keltnerUpperCrossUp

La idea. Entrar en largo cuando el cierre cruza por encima de la banda superior del canal de Keltner.

Arquetipo: breakout · **Dirección:** long · **Forma:** condición fija `AND` una aleatoria

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

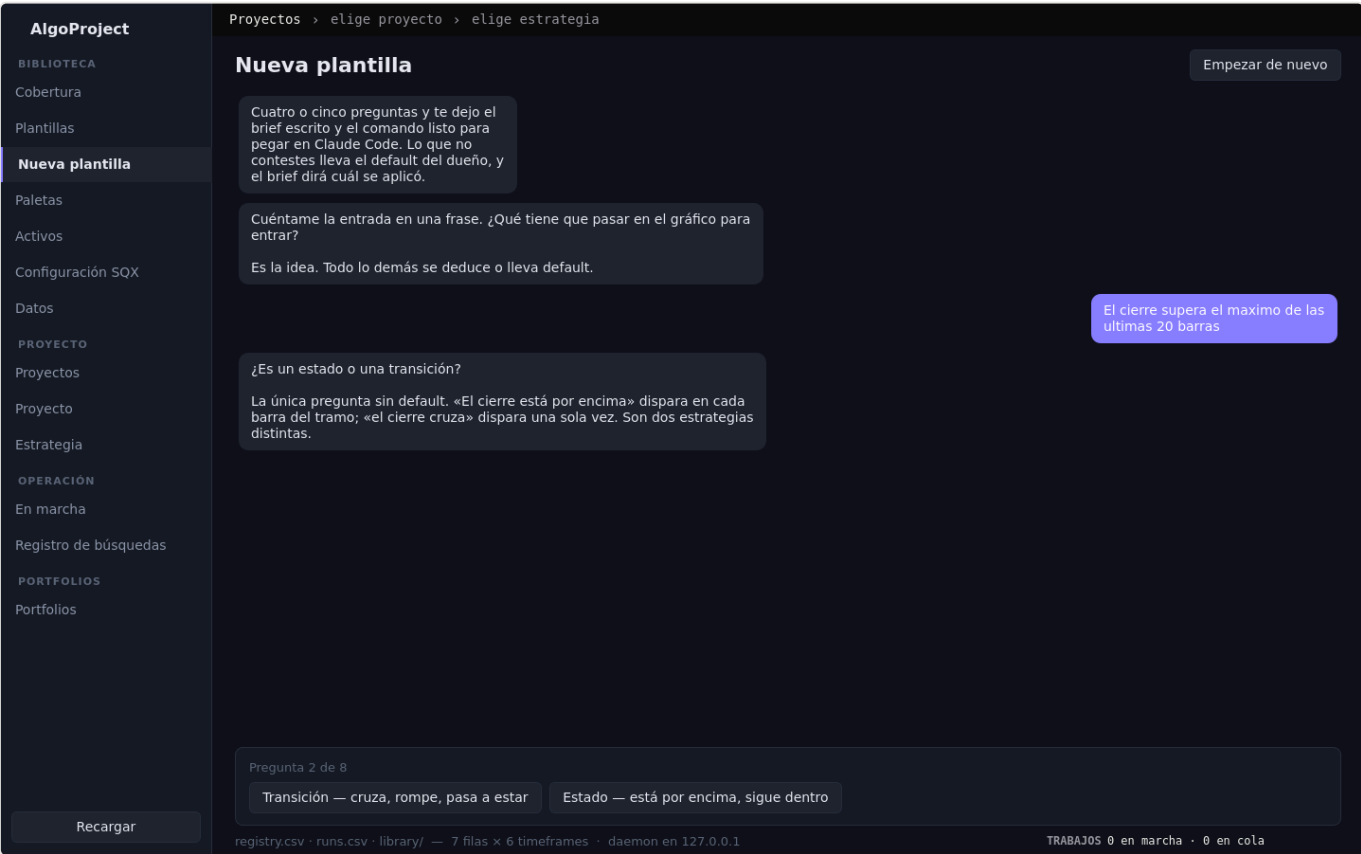
TRABAJOS 0 en marcha · 0 en cola

A la izquierda la lista, con buscador (nombre, arquetipo o activo) y filtro por estado. Los borradores del chat aparecen marcados como tales: **no son plantillas**, no tienen `.sqx`, y por eso no cuentan en la matriz.

A la derecha la ficha. En orden:

- **Las etiquetas:** estado, arquetipo, forma, origen y fecha. El estado explica su significado al pasar el ratón: `draft` es XML sin comprobar, `validated` es XML que resuelve, `buildConfirmed` es el único que prueba algo — una construcción real sacó estrategias y llevan de verdad su bloque.
- **La línea de disco:** `template.sqx`, `brief.md`, `deps/`. En verde lo que está, en rojo lo que falta. El registro es una **afirmación**; esta línea es la **prueba**. Importa porque SQX no da error con una plantilla cuyos bloques no están instalados: la quita del builder en silencio.
- **Estado y nota**, con su botón Guardar.
- **Qué rellena el builder:** qué huecos de la plantilla son libres y cuáles están atados a un grupo (se explica más abajo, en Paletas).
- **Corridas:** una fila por mercado y timeframe, con las estrategias construidas y retenidas, y el veredicto como desplegable. El veredicto se escribe **al cambiarlo**, sin botón: es un juicio tuyo, no un formulario.
- **Brief:** el texto en castellano tal cual lo escribió quien la autoró. La ventana no lo resume y no lo deja editar por detrás del fichero.

Nueva plantilla — el chat



Ocho preguntas como mucho. Las que llevan default del dueño traen un botón **Elige tú** que lo aplica y lo deja escrito en el brief; las que no lo llevan, no se pueden saltar.

Una pregunta no tiene default y nunca lo tendrá: estado contra transición. «El cierre *está* por encima de la banda» dispara en cada barra del tramo; «el cierre *cruza* la banda» dispara una sola vez. Son dos estrategias distintas, no dos maneras de decir lo mismo, y adivinarla gasta CPU para construir lo que no pediste.

Al terminar:

Nueva plantilla

Empezar de nuevo

elige tu

Nombre de la plantilla, en camelCase. Sin símbolo y sin timeframe.

Una plantilla no pertenece a ningún mercado: eso es una fila de runs.csv.

high20BreakStop

Listo. Brief escrito en /home/sergioguslw/Desktop/AlgoData/templates/drafts/high20BreakStop.json.

Defaults aplicados: direction, random, exit.

Pega esto en Claude Code

```
/strategy-template high20BreakStop
```

Idea: El cierre supera el maximo de las ultimas 20 barras
Lógica: transición (cruza / rompe / pasa a estar) — ya preguntado, no lo vuelvas a preguntar.
Dirección: long
Arquetipo: breakout
Aleatorias: una condición aleatoria libre (#Group# vacío, 500 condiciones nativas)
Orden de entrada: stop
Salidas: la pila del esqueleto sin tocar

Y esto es lo que la skill ejecutará:

- ¿Existe ya la condición, y qué grupos la agrupan?
python3 -m sqx.inspect.vocabulary <término>
- Instala el bloque en las dos instalaciones (si hay que autorarlo)
python3 -m sqx.blocks.install /home/sergioguslw/Desktop/AlgoData/templates/library/high20BreakStop/deps/blocks.xml --role conductor
- Comprueba que la instalación que construye no se queda sin el bloque
python3 -m sqx.inspect.vocabulary --diff custodian
- Emite la plantilla
python3 -m sqx.templates.build high20BreakStop /home/sergioguslw/Desktop/AlgoData/templates/library/high20BreakStop/deps/blocks.xml <CBlock_key> /home/sergioguslw/Desktop/AlgoData/templates/library/high20BreakStop/template.sqx
- Da de alta la plantilla en el registro
python3 -m sqx.templates.registry --set name=high20BreakStop --set archetype=breakout --set status=validated

Copiar el prompt

Copiar el brief en JSON

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

Te quedan tres cosas: el **borrador del brief** escrito en disco, el **prompt** para pegar en Claude Code —que ya lleva todas las respuestas, para que la skill no vuelva a preguntar lo que ya contestaste—, y **la lista de comandos que la skill va a ejecutar**, para que sepas lo que viene. La ventana no ejecuta ninguno.

Fíjate en la línea *Defaults aplicados: direction, random, exit*. El brief deja constancia de qué preguntas te saltaste y qué default las rellenó. Un default aplicado en silencio y nunca escrito es como se pierde una decisión.

Paletas — qué puede sortear el builder

AlgoProject

BIBLIOTECA

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Configuración SQX

Datos

PROYECTO

Proyectos

Proyecto

Estrategia

OPERACIÓN

En marcha

Registro de búsquedas

PORTFOLIOS

Portfolios

Recargar

Proyectos > elige proyecto > elige estrategia

Paletas de bloques

Ruptura · ★ Ruptura — base

Clonar

Borrar

Sin etiquetar → fuera (la paleta I

Buscar bloque o forma en la...

Guardar

condiciones 128 de 588 · indicador 44/171 · nivel 32/85 · 177 elegidos a mano · 590 sin etiquetar

Ruptura default Paleta por defecto de la familia. Sin curar todavía: el criterio de qué bloques la forman lo pone el encargo de autoría, no el generador.

Una paleta gobierna los huecos LIBRES de una plantilla. Un hueco atado a un grupo sortea ese grupo y la ignora; un bloque fijo es parte del esqueleto. Medido el 2026-09-24. 52 apagados están en un grupo — esos se escapan por esa vía.

Bloque	Forma escrita	Papeles	Grupos	Etiqueta	Peso
ADXChangesDown	ADX(@Chart@#Period#)[#Shift#] change...	señal	—	— sin etiquetar	— taxonomía
ADXChangesUp	ADX(@Chart@#Period#)[#Shift#] change...	señal	—	— sin etiquetar	1 neutro
ADXCrossDown	ADX(@Chart@#Period#)[#Shift#] crosses...	señal	—	— sin etiquetar	— taxonomía
ADXCrossUp	ADX(@Chart@#Period#)[#Shift#] crosses...	señal	—	— sin etiquetar	2 encaja
ADXFalling	ADX(@Chart@#Period#)[#Shift#] is falling	señal	—	— sin etiquetar	— taxonomía
ADXHigher	ADX(@Chart@#Period#)[#Shift#] > ...	señal	—	— sin etiquetar	— taxonomía
ADXLower	ADX(@Chart@#Period#)[#Shift#] < ...	señal	—	— sin etiquetar	1 neutro
ADXRising	ADX(@Chart@#Period#)[#Shift#] is rising	señal	—	— sin etiquetar	1 neutro

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

TRABAJOS 0 en marcha · 0 en cola

Esta es la vista que estrecha el vocabulario del builder, y es una **librería**: no hay tres paletas fijas, hay las que haya. Vienen tres por defecto, una por familia, marcadas con ★, y encima de ellas se clona todo lo demás.

Arriba, de izquierda a derecha:

- **El selector de paleta**, agrupado por familia: Ruptura · ★ Ruptura — base . Las ★ son las de por defecto.
- **Clonar**: copia la paleta abierta con el nombre que le des y la abre. Es el camino normal — parte de una por defecto y personalízala.
- **Borrar**: solo para las tuyas. En una por defecto está deshabilitado y dice por qué.
- **Qué hacer con lo no etiquetado**: entra a peso 1 (la paleta hereda la taxonomía y tú solo corriges) o fuera (la paleta ES su lista: solo entra lo que hayas elegido). La segunda es la que hace útil una paleta curada a mano sin esperar a que la taxonomía esté etiquetada.
- **El buscador**, que ignora la categoría: encuentra un bloque sin saber en cuál de las 83 lo archivó SQX.
- **Los recuentos**, en su propia línea. El primero, **condiciones**, va en verde o en rojo según caiga dentro del **90–170 que pide el dueño** por build: por debajo el builder no tiene de dónde combinar, por encima vuelve el sobreajuste. Si una paleta se sale, la ficha de debajo lo dice y además apunta hacia dónde — faltan bloques o sobran.

Debajo, la ficha de la paleta abierta: su familia, si es por defecto o tuya, de cuál es clon, y su nota. Y si no ha elegido nada todavía, un aviso en ámbar diciéndolo con todas las letras — porque una paleta vacía y una paleta que deja pasar todo se ven igual, y no son lo mismo.

A la izquierda, las categorías con cuántos bloques tienen encendidos. A la derecha, los bloques: su clave, **su forma escrita** —que es lo único que dice de verdad qué comprueba—, sus papeles, los grupos que lo agrupan, su etiqueta y el peso.

La lista de bloques es siempre la misma — son los 767 del vocabulario del builder — y eso no cambia entre familias. Lo que cambia por familia son las dos últimas columnas y los recuentos.

- **Etiqueta:** lo que `taxonomy.yaml` dice de ese bloque **para la familia que tienes elegida**. Es la columna que hace visible el cambio de familia. `— sin etiquetar` significa que nadie ha dicho nada todavía, y entonces las tres familias lo tratan igual.
- **Peso:** lo único editable, y es un *override*. `— taxonomía` significa «haz lo que diga la etiqueta»; cualquier otro valor la pisa **solo para esta paleta**. `0` apaga el bloque y lo pone en gris, `3` lo prefiere. Se guarda con el botón; hasta entonces el recuento dice cuántos llevas sin guardar.

Un bloque apagado se ve en gris en toda su fila, y el contador de su categoría baja: `ADX 8/8` pasa a `ADX 5/8`. Ésa es la forma rápida de comprobar que una paleta está estrechando de verdad.

⚠ La línea gris de debajo repite el límite, y hay que tomárselo en serio: **una paleta gobierna los huecos LIBRES de una plantilla**. Un hueco atado a un grupo sorteja ese grupo y la ignora, y un bloque fijo es parte del esqueleto. Está medido el 2026-09-24 y está en `knowhow/authoring/builder-block-switches.md`. Cuando un bloque apagado está en un grupo, su celda de **Grupos** se pone en ámbar y lo dice.

keltnerUpperCrossUp

`buildConfirmed` breakout market_long authored 2026-09-22

`template.sqx` • `brief.md` • `deps/ (2)`

close cruza por encima de la banda superior de Keltner (fijo) AND una condicion aleatoria libre

Estado: `buildConfirmed`
Nota — por qué está en este estado, qué falta
Guardar

Qué rellena el builder

1 hueco(s) libre(s): la paleta sí los gobierna. Otros 1 están atados a un grupo y se le escapan.

RandomCondition1 (condition) `atado a un grupo que esta instalación no tiene`

RandomCondition2 (condition) `libre — la paleta lo gobierna`

Fijos en la plantilla, fuera de toda paleta: `MarketPositionIsLong`

Por eso la ficha de cada plantilla lleva ahora **Qué rellena el builder**: un hueco por línea, diciendo si es libre —y entonces la paleta lo gobierna— o a qué grupo está atado, y debajo los bloques fijos, que están fuera de toda paleta. Si esa línea dice que no hay huecos libres, la vista de paletas no puede hacer nada por esa plantilla y hay que tocar el grupo.

En la captura, `RandomCondition1` sale en rojo, «*atado a un grupo que esta instalación no tiene*», y no es un fallo de la captura. La ficha resuelve los grupos con el `blockGroups.xml` **del maestro**, y el grupo de esta plantilla (`keltnerUpperCrossUpSignal`) se instaló solo en los workers, conductor y custodio, que son los que construyen. Comprobado el 28-09-2026: contra el conductor o el custodio ese hueco se lee «atado a `keltnerUpperCrossUpSignal`». El rojo quiere decir «el maestro no lo tiene», no «ningún install lo tiene».

Las demás zonas

La barra lateral de la izquierda (se ve en la captura de Cobertura) va en cuatro grupos, y cada zona tiene su capítulo en este mismo PDF:

grupo	zonas	capítulo
BIBLIOTECA	Cobertura, Plantillas, Nueva plantilla, Paletas	este
	Activos	38
	Configuración SQX	66
	Datos	67
PROYECTO	Proyectos (la galería)	61
	Proyecto (el raíl, el embudo y el panel de databanks)	65
	Proyecto: los filtros y «Continuar workflow»	69
	Estrategia (la ficha)	68, y 58 para leer el resultado de un estudio
OPERACIÓN	En marcha, Registro de búsquedas y la franja de trabajos	70
PORTFOLIOS	Portfolios	71

Arriba de todas, la barra de contexto `Proyectos > proyecto > estrategia` dice qué hay elegido, y cada miga abre su zona. **Ctrl+K**, desde cualquier sitio, abre la paleta de comandos: escribe el nombre de una zona, de un proyecto, de un databank o de una estrategia y Enter la abre. Cómo se lee todo esto de un vistazo —las etiquetas, los números, el modo lectura— está en el capítulo 60.

Un ejemplo completo

Idea: «el cierre supera el máximo de las últimas 20 barras».

1. `bin/algoui`. Abre en Cobertura. El 28-09-2026: `2/7` arquetipos, cuatro corridas, dos activos (XAUUSD y USDJPY).
2. **Nueva plantilla**. Escribes la idea. Contestas **Transición** —quieres que dispare al romper, no en cada barra por encima. Arquetipo **breakout**. Orden de entrada **Stop**. Dirección, aleatorias y salidas: **Elige tú** en las tres. Nombre: `high20BreakStop`.
3. La ventana escribe `~/Desktop/AlgoData/templates/drafts/high20BreakStop.json` y te da el prompt.
4. **Copiar el prompt** y pegarlo en Claude Code. Ahí sí: `/sqx-strategy-template` mira el vocabulario de la instalación, autora el bloque si no existe, emite el `.sqx` y da de alta la fila.
5. Vuelves a la ventana, **Recargar**. La plantilla ya está en la lista, el borrador ha desaparecido, y la fila `breakout` de la matriz sigue sin corrida nueva —porque autorar no es construir. Eso es `/template-run`.
6. Cuando esa corrida termine y la hayas mirado, abres su ficha y le pones el veredicto.

Qué NO te dice

- **No te dice si una plantilla es buena**. `buildConfirmed` significa que SQX la acepta y que las estrategias llevan su bloque. No dice que gane dinero. Eso lo deciden los pasos 8 a 20 del workflow, que se leen en la zona Proyecto, no en estas.
- **El veredicto de una corrida es tuyo, no suyo**. La ventana lo guarda y lo pinta; no lo calcula. Una celda verde significa «tú dijiste que era prometedora», nada más.
- **La matriz cuenta corridas, no calidad ni tamaño**. Una celda con `1` puede ser una construcción de humo de 30 estrategias y 29 segundos. El número de estrategias está en la ficha, no en la celda.
- **No sabe lo que hay dentro de un `.sqx`**. La línea de disco dice si el fichero existe, no si su contenido es el que el registro afirma. Para eso está `sqx/inspect/template_check.py`. La excepción es el panel de huecos, que sí lo abre y lee sus bloques aleatorios.
- **Una paleta guardada todavía no llega a SQX**. La ventana escribe el YAML; quien lo tiene que aplicar al `<BuildingBlocks>` de la tarea Build es el builder, y esa pieza no está escrita. Hasta entonces una paleta es una decisión anotada, no un efecto.

- **La paleta no etiqueta.** Los pesos salen de `taxonomy.yaml`, que está casi entero sin etiquetar: el 28-09-2026 la línea de recuentos de la paleta Ruptura dice `590 sin etiquetar` de los 767. Lo no etiquetado lo tratan igual las tres familias, así que lo que hoy distingue una paleta de otra son sus pesos puestos a mano (`177 elegidos a mano` en esa misma línea), no la taxonomía.

Si algo falla

- **el demonio no respondió en el puerto 8765** — hay otra cosa ocupando el puerto, o el demonio se cayó al arrancar. Arráncalo a mano con `python3 -m ui.daemon.serve` y lee el error; si el puerto está ocupado, cámbialo en `ui_port` de `config/machine.yaml`.
- **La ventana abre vacía y los contadores están a cero** — `AlgoData/templates/` no existe o no tiene CSV. No es un fallo: es una librería sin nada dentro.
- **Una plantilla aparece en rojo en la línea de disco** — su fila está en el registro pero su carpeta no está completa. Suele pasar al traer un registro de otra máquina sin traer `library/`.
- **qt.qpa.plugin: could not load the Qt platform plugin "xcb"** — **pasó en esta máquina**, el 2026-09-24, la primera vez que se abrió la ventana de verdad. El plugin `xcb` de Qt necesita cuatro librerías que Ubuntu no trae puestas: `libxkbcommon-x11.so.0`, `libxcb-icccm.so.4`, `libxcb-keysyms.so.1` y `libxcb-xkb.so.1`. El mensaje que da Qt culpa a `libxcb-cursor0`, que **sí estaba** — o sea que el mensaje miente sobre cuál falta. Quién falta de verdad lo dice esto:

```
bash ldd ~/.local/lib/python3.10/site-packages/PySide6/Qt/plugins/platforms/libqxcb.so | grep "not found"
```

Como `sudo` pide contraseña aquí, no se instalan con `apt install`: se bajan y se extraen en el home, que es lo que ya está hecho en `~/.local/lib/qt-xcb/`.

```
bash apt-get download libxkbcommon-x11-0 libxcb-icccm4 libxcb-keysyms1 libxcb-xkb1 for d in *.deb;
do dpkg -x "$d" x; done mkdir -p ~/.local/lib/qt-xcb && cp -P x/usr/lib/x86_64-linux-gnu/*.so*
~/.local/lib/qt-xcb/
```

`bin/algoui` añade esa carpeta al `LD_LIBRARY_PATH` él solo, así que una vez hecho esto no hay que acordarse de nada. En un ordenador nuevo con `sudo` a mano, `sudo apt install libxkbcommon-x11-0 libxcb-icccm4 libxcb-keysyms1 libxcb-xkb1` hace lo mismo y la carpeta del home sobra.

38. Activos — costes, tramos, MC Retest y Cross Market de cada instrumento

La zona **Activos** de la ventana. Enseña **cada instrumento de assets/ en una ficha propia**, dos por fila cuando la ventana es ancha: lo que cuesta operarlo, entre qué fechas se construye y se testea, qué sortea su MC Retest y contra qué otros mercados se comprueba su edge — y todo eso se cambia desde ahí.

Qué pregunta responde

- **¿Qué activos tengo y cuáles están bloqueados?** Un activo con un coste obligatorio sin pactar no deja escribir ni una plantilla ni un proyecto (regla dura 5). El índice de la izquierda lo pinta en rojo, y la ficha dice exactamente qué campo falta y qué pasa por eso.
- **¿Qué le estoy cobrando de verdad a este instrumento, y en qué unidad?** Spread, comisión, slippage y los dos swaps, cada uno con su unidad y al lado de lo que SQX lleva hoy, que casi nunca está en la misma unidad.
- **¿Qué datos hay y entre qué fechas va cada tramo?** Las fechas con datos (desde / hasta) y los tramos Build, OOS 1 y OOS 2, cada uno con su selector de fechas.
- **¿Contra qué mercados compruebo que el edge sobrevive?** El Check de Cross Market, fila a fila, agrupado en Family y Structural.

Cuándo lo usas, y cuándo no

Lo usas cuando el bróker te da una cifra de verdad y hay que sustituir una provisional; cuando das de alta un instrumento; cuando mueves la fecha de un tramo; cuando fijas los mercados del Cross Market **antes** de mirar ningún resultado; y antes de construir sobre un activo, para ver qué arrastra.

No lo usas para nada que hable con SQX. La ventana no abre el maestro, no arranca un worker y no lee un instrumento del install: SQX hoy y los hechos del instrumento son fotos leídas de SQX, y quien las refresca sigue siendo `python3 -m sqx.inspect.instruments`.

Tampoco para lo que decide para todos los activos a la vez — la doctrina de construcción, las dos clases de coste, qué es cada tramo, los múltiplos por defecto del MC Retest. Eso salió de aquí y vive en la zona **Configuración SQX** (su propio capítulo).

Y no sustituye a la preflight: antes de autorizar nada sigue siendo obligatorio `python3 -m core.assets <SÍMBOLO>`. La ventana enseña lo mismo, pero el código de salida — 2 bloqueado, 3 esquema roto — solo lo da el comando.

Antes de empezar

- `pip install -r requirements.txt` (la zona necesita `ruamel.yaml`, el YAML que conserva los comentarios).
- Nada más. No hace falta SQX, ni un worker, ni datos exportados.

Cómo se ejecuta

```
algoui
```

La ventana de siempre. En la barra de la izquierda, BIBLIOTECA → **Activos**. Para una captura sin pantalla: `python3 -m ui.desktop.launch --zone Activos --shot <carpeta>` con `QT_QPA_PLATFORM=offscreen`.

Qué produce

Escribe, y solo escribe, dentro de `assets/` , siempre a través de `core.assetwrite` :

lo que tocas	dónde acaba
un coste y su «por qué», un rango de MC Retest, cualquier valor de «Fichero entero»	<code>assets/symbols/<SÍMBOLO>.yaml</code>
la fecha de un tramo	<code>assets/_policy.yaml</code> , bajo <code>segments: <SÍMBOLO>:</code> — una línea
añadir o quitar un mercado del Check de Cross Market	<code>assets/_markets.yaml</code> — una línea (dos si la categoría estaba vacía)
dar de alta un activo	un <code>.yaml</code> nuevo en <code>symbols/</code> y su bloque en <code>_policy.yaml</code>
retirar un activo	se mueve a <code>assets/symbols/_retired/<SÍMBOLO>.yaml</code>

Los comentarios se conservan. Cambiar una fecha cambia esa línea y nada más; añadir un mercado añade su línea y quitarlo la quita, sin tocar los párrafos que explican por qué está cada uno. Después de cada escritura se regenera `assets/INDEX.md` .

Cómo se lee el resultado

La zona

AlgoProject

BIBLIOTECA

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Configuración SQX

Datos

PROYECTO

Proyectos

Proyecto

Estrategia

OPERACION

En marcha

Registro de búsquedas

PORTFOLIOS

Portfolios

Recargar

Proyectos > elige proyecto > elige estrategia

Activos

18 activos · 0 bloqueados · 2 sólo con cifras provisionales

Nuevo activo

ACTIVOS

AUDJPY

AUDUSD

BRENT

CADJPY

DAX40

DJ30

EURJPY

EURUSD

GBPJPY

GBPUSD

NIKKEI225

USA500

USATEC

USDCAD

USDCHF

USDJPY

XAGUSD

XAUUSD

AUDJPY

Fichero entero

Retirar

Forex the5ers AUDJPY_DukasM1_the5ers Sesión AUDJPY_fltmo Datos 2003-12-01 → 2026-09-25

al día Nada sin decidir y nada provisional.

Costes — doble clic para cambiar

Campo	Usar	Unidad	SQX hoy
Spread	1.58	puntos	0.6
Comisión	8	USD por lote	0 USD por lote
Slippage (IS)	0.79	puntos	0
Slippage (OOS)	0.79	puntos	0
Swap (Long)	6.01	puntos por noche	0.41 puntos
Swap (Short)	-14.71	puntos por noche	-7.84 puntos

assets/symbols/AUDJPY.yaml · sus tramos viven en assets/_policy.yaml · proyectos del maestro que lo usan: AUDJPY, CADJPY_H1, EURJPY_H1, EURUSD, GBPJPY_H1, USDCHF, USDJPY, XAUUSD, XAUUSD_BreakOut_H1

Tramos — elige fechas y Aplicar

Datos 2003-12-01 → 2026-09-25

Build

2008-01-01

2017-12-31

Aplicar

Spread IS

OOS 1

2018-01-01

2022-12-31

Aplicar

Spread OOS

OOS 2

2023-01-01

2026-08-30

Aplicar

Spread OOS · reservado para WFC, CSCV, MarketSurfaces, WFM, ATRStop

MC Retest — rangos en puntos

Rango	Mín	Máx	SQX hoy
-------	-----	-----	---------

AUDUSD

Fichero entero

Retirar

Forex the5ers AUDUSD_DukasM1_the5ers Sesión AUDUSD_fltmo Datos 2003-08-04 → 2026-09-25

al día Nada sin decidir y nada provisional.

Costes — doble clic para cambiar

Campo	Usar	Unidad	SQX hoy
Spread	0.85	puntos	0.1
Comisión	8	USD por lote	0 USD por lote
Slippage (IS)	0.425	puntos	0
Slippage (OOS)	0.425	puntos	0
Swap (Long)	-3.1	puntos por noche	-3.36 puntos
Swap (Short)	-0.13	puntos por noche	0.29 puntos

assets/symbols/AUDUSD.yaml · sus tramos viven en assets/_policy.yaml · proyectos del maestro que lo usan: AUDJPY, CADJPY_H1, EURJPY_H1, EURUSD, GBPJPY_H1, Retester, USDCHF, USDJPY, XAUUSD

Tramos — elige fechas y Aplicar

Datos 2003-08-04 → 2026-09-25

Build

2008-01-01

2017-12-31

Aplicar

Spread IS

OOS 1

2018-01-01

2022-12-31

Aplicar

Spread OOS

OOS 2

2023-01-01

2026-08-30

Aplicar

Spread OOS · reservado para WFC, CSCV, MarketSurfaces, WFM, ATRStop

MC Retest — rangos en puntos

Rango	Min	Máx	SQX hoy
-------	-----	-----	---------

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

TRABAJOS 0 en marcha · 0 en cola

A la izquierda, **el índice**: pulsar un activo lleva su ficha a la vista. El color es su estado y el tooltip lo deletrea:

color	qué significa
rojo	bloquea la autoría: falta un coste obligatorio, o el esquema está roto
ámbar	funciona, pero alguna cifra es provisional y todo resultado con coste la arrastra
normal	nada que decidir

Arriba, la cuenta: cuántos activos, cuántos bloqueados, cuántos solo con cifras provisionales. Los retirados ya no se enseñan.

Una ficha

XAUUSD

Fichero entero

Retirar

No forex Infinox XAUUSD_DukasM1_Infinox Sesión XAUUSD_ftmo Datos 2003-05-05 → 2026-09-25

RANGOS MC RETEST SIN DECIDIR Min Distance — No bloquea: sólo vuelve ininterpretable esa tarea del MC Retest.

Costes — doble clic para cambiar

Campo	Usar	Unidad	SQX hoy
Spread (IS)	16.37	puntos	5
Spread (OOS)	22.54	puntos	10
Spread (OOS 2)	37.49	puntos	—
Comisión	0.001778	% del nocional	8 USD por lote
Slippage (IS)	8.19	puntos	0
Slippage (OOS)	11.27	puntos	0
Slippage (OOS 2)	18.75	puntos	—
Swap (Long)	-7	% anual	-73.42 puntos
Swap (Short)	-7	% anual	38.76 puntos

assets/symbols/XAUUSD.yaml · sus tramos viven en assets/_policy.yaml · proyectos del maestro que lo usan: AUDJPY, Builder, EURUSD, Retester, USDJPY, XAUUSD, XAUUSD_Breakout_H1

Tramos — elige fechas y Aplicar

Datos 2003-05-05 → 2026-09-25

Build 2008-01-01 2017-12-31 Aplicar

Spread IS

OOS 1 2018-01-01 2022-12-31 Aplicar

Spread OOS

OOS 2 2023-01-01 2026-08-30 Aplicar

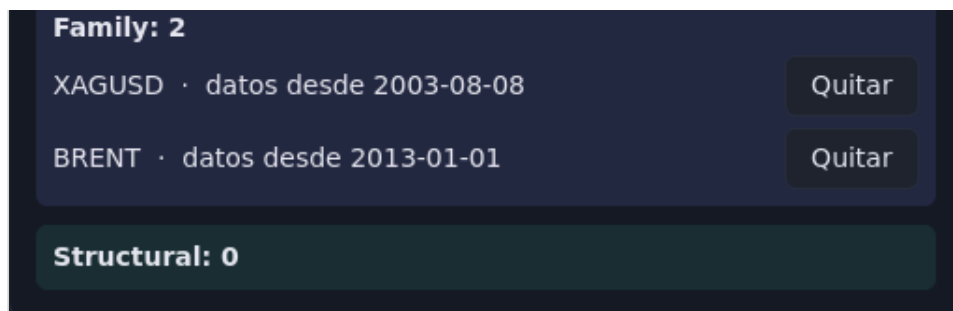
Spread OOS 2 · reservado para WFC, CSCV, MarketSurfaces, WFM, ATRStop

MC Retest — rangos en puntos

Rango	Mín	Máx	SQX hoy
Spread	8.7	31.98	1 - 5 · 5 - 12
Slippage	5	20	0 - 5 · 0 - 10
Min Distance	0	0	0 - 10

Check de Cross Market

Añadir



De arriba abajo:

1. **El nombre**, con dos botones: **Fichero entero** despliega todos los valores del YAML del activo, cada uno con el comentario que el propio fichero lleva al lado; **Retirar** lo saca de la librería.
2. **Las fichas**: clase, bróker, feed, sesión (en rojo si está sin decidir) y el rango de datos que SQX tiene.
3. **La preflight**, en el orden en el que se actúa: lo que rompe, lo que bloquea, lo que solo mancha el resultado.
4. **Costes**, con nombres legibles — Spread (IS), Slippage (OOS), Swap (Short)... — su unidad y lo que SQX lleva hoy. Pasa el ratón por una fila para leer **por qué** vale lo que vale; doble clic para cambiarla.
5. **Tramos**: primero **Datos** (lo que hay, no se edita), luego Build, OOS 1 y OOS 2 con su selector de fechas. Debajo de cada tramo, qué spread usa y, en ámbar, para qué pruebas está reservado.
6. **MC Retest**: el rango (mínimo y máximo, en puntos) que sortea cada método — Spread, Slippage y Min Distance — al lado de lo que SQX lleva hoy. **Min Distance sin decidir se lee 0**: sin perturbación. El fichero sigue diciendo `null`, y por eso la tarea `MCR 4 MinDist` no se escribe y la preflight lo sigue avisando.
7. **Check de Cross Market** (antes «universo de retest»): los mercados agrupados por categoría, cada grupo con su color y su cuenta («Family: 2», «Structural: 0»).

Una ficha con trabajo pendiente se ve así — BRENT tiene cifras, pero provisionales, y ningún tramo con fechas:

BRENT

Fichero entero

Retirar

No forex ftmo BRENTCMDUSD_ftmo Sesión SIN DECIDIR Datos 2013-01-01 → 2026-09-24

COSTES PROVISIONALES Spread (IS), Spread (OOS), Comisión, Slippage (IS), Slippage (OOS), Swap (Long), Swap (Short) — Hay cifra y no bloquea, pero no está pactada con el bróker: TODO resultado con coste producido con ella arrastra esa advertencia.

TRAMOS SIN FECHAS Build, OOS 1, OOS 2 — No bloquea, pero `window()` se niega a inventar una ventana, así que una tarea sobre ese tramo falla al configurarse.

Costes — doble clic para cambiar

Campo	Usar	Unidad	SQX hoy
Spread (IS)	16	puntos	32
Spread (OOS)	32	puntos	32
Comisión	0	% del nocional	0 USD por lote
Slippage (IS)	8	puntos	0
Slippage (OOS)	16	puntos	0
Swap (Long)	0.0111	% anual	0.03 puntos
Swap (Short)	-9.537	% anual	-25.76 puntos

assets/symbols/BRENT.yaml · sus tramos viven en assets/_policy.yaml · proyectos del maestro que lo usan: XAUUSD

Tramos — elige fechas y Aplicar

Datos 2013-01-01 → 2026-09-24

Build

Spread IS

OOS 1

Spread OOS

OOS 2

Spread OOS · reservado para WFC, CSCV, MarketSurfaces, WFM, ATRStop

MC Retest — rangos en puntos

Rango	Mín	Máx	SQX hoy
Spread	sin decidir	sin decidir	1 - 5
Slippage	sin decidir	sin decidir	0 - 5

Check de Cross Market — este activo no es un main declarado en _markets.yaml

Cambiar la fecha de un tramo

Tramos — elige fechas y Aplicar

Datos 2003-12-01 → 2026-09-25

Build

2008-01-01

2017-12-31

Aplicar

Spread IS

OOS 1

2018-01-01

2022-06-30

Aplicar

Spread OOS

OOS 2

2023-01-01

2026-08-30

Aplicar

Spread OOS · reservado para WFC, CSCV, MarketSurfaces, WFM, ATRStop

MC Retest — rangos en puntos

Rango	Mín	Máx	SQL hoy
Spread	0.94	2.31	1 - 5
Slippage	sin decidir	sin decidir	0 - 5

Check de Cross Market — este activo no es un main declarado en _markets.yaml

Elige el día en el selector (tiene calendario, o se teclea). **Aplicar** solo se enciende cuando algo cambió y el «desde» no queda detrás del «hasta»; al pulsarlo escribe **solo el extremo que se movió**. Un 1 de enero como inicio o un 31 de diciembre como final se guardan como el año entero (2018), que es la convención del fichero; cualquier otro día, como fecha (2022-06-30). Los dos extremos están siempre INCLUIDOS. Bajar el selector hasta el fondo lo deja en «sin decidir» (null).

Cambiar un coste

XAUUSD · Spread (IS)

Unidad: puntos. SQX lleva hoy 5, que no siempre está en la misma unidad. Un coste sin decidir (`null`) BLOQUEA la autoría: la preflight sale con 2 y ninguna plantilla ni proyecto se escribe para este activo. Es a propósito — un número inventado parece decidido.

Usar (puntos)

Por qué

La app propone una línea con la fecha y el valor que sustituye; acábala. No escribe PROVISIONAL por ti: esa palabra es la que lee `assetcheck.provisional` para marcar el activo, y ponerla sería decidir por ti cuán provisional es.

onboard 2026-09-27 — spread medio de Darwinex del tramo x factor 1.25, modelo `volatilidad` donde no hay ticks (XAUUSD_DarwTick_Infinox).
studies.data.spread.onboard

Guardar

Cancelar

Doble clic en la fila. El valor y su «por qué» se escriben juntos o no se escribe ninguno. Vacío es `null`: el campo vuelve a «sin decidir» y, si es obligatorio, bloquea la autoría a propósito. Al salir del valor la app propone un «por qué» con la fecha y la cifra que sustituye; no escribe PROVISIONAL por ti.

Añadir y quitar un mercado del Cross Market

XAUUSD · Añadir un mercado

Family: mismo motor económico que el activo, la prueba fácil. Structural: misma estructura y ningún motor compartido, la prueba dura. LA LISTA SE FIJA ANTES DE MIRAR NINGÚN RESULTADO: elegir los mercados después de ver dónde funcionan las estrategias convierte la prueba en una selección y sus p-valores en decoración.

Categoría

Activo

Guardar

Cancelar

Añadir abre la caja: primero la categoría (Family / Structural), después el activo, por su nombre y con la primera fecha con datos. Solo ofrece activos que aún no están en ninguna de las dos. **Quitar**, en cada fila, pregunta antes. Las dos cosas recuerdan lo mismo: **la lista se fija antes de mirar ningún resultado**; elegirla después convierte la prueba en una selección.

Añadir el primer mercado a una categoría vacía son **dos líneas** en el diff: `structural: []` pasa a `structural:` y debajo la fila. Quitarlo después la devuelve a `[]`, sin rastro.

Solo un activo declarado como *main* en `_markets.yaml` (hoy XAUUSD y USDJPY) tiene Check de Cross Market; en los demás la ficha lo dice y no ofrece Añadir.

Dar de alta y retirar

Nuevo activo

La clase decide QUÉ CAMPOS de coste tendrá el fichero: ``forex`` Pares de divisas. Hoy: los 10 de the5ers. · ``no_forex`` Todo lo demás: índices CFD y metales. Hoy: DAX40, DJ30, NIKKEI225, USA500, USATEC, XAUUSD, XAGUSD, BRENT.

Nombre	XAUUSD — es también el nombre del fichero
Bróker	
Símbolo SQX	tal y como lo imprime -symbol action=list
Sesión	vacío = sin decidir, y eso bloquea la autoría
Clase	forex
Tamaño de tick	
Valor del punto	
Distancia mínima	

Los tres últimos son hechos del feed, no decisiones: sácalos con ``python3 -m sqx.inspect.instruments``. Todos los costes nacen ``null`` y el activo queda bloqueado hasta que los pactes.

GuardarCancelar

Nuevo activo, arriba a la derecha. La clase decide qué campos de coste tendrá el fichero. Los tres hechos del instrumento se teclean (sácalos de `python3 -m sqx.inspect.instruments`) y **todos los costes nacen null**: el activo entra bloqueado, que es lo que se quiere. **Retirar**, en la ficha, mueve el fichero a `symbols/_retired/`; su bloque de tramos se queda en `_policy.yaml`.

Un ejemplo completo

Quieres acortar el OOS 1 de AUDJPY para que acabe a mitad de 2022.

1. `algoui` → **Activos** → AUDJPY en el índice.
2. En Tramos, fila **OOS 1**, el segundo selector: `2022-06-30`. Se enciende **Aplicar**.
3. Aplicar. La ficha se redibuja con la fecha nueva.
4. En la terminal, lo que cambió:

```
$ git diff assets/_policy.yaml
AUDJPY:
  data: {from: 2003-12-01, to: 2026-09-25} # AUDJPY_DukasM1_the5ers, 8.533.415 barras M1
  build: {from: 2008, to: 2017}
-   oos1: {from: 2018, to: 2022}
+   oos1: {from: 2018, to: 2022-06-30}
  oos2: {from: 2023, to: 2026-08-30}
```

Una línea, y el comentario de al lado intacto. Antes de construir nada sobre AUDJPY: `python3 -m core.assets AUDJPY`, que ya imprime el tramo nuevo en milisegundos.

Lo mismo con el Cross Market: **Añadir** → Family → USA500 en la ficha de XAUUSD deja en `_markets.yaml` exactamente una línea nueva, `- {feed: USA500_DukasM1_Infinox, data_from: 2012-01-19}`, y **Quitar** la borra sin dejar rastro.

Qué NO te dice

- **No dice si una cifra es correcta**, solo si está decidida. Eso lo dice su «por qué», que es texto y no lo valida nadie.
- **No sabe qué lleva SQX ahora mismo.** `SQX hoy` es una foto del día en que alguien corrió `sqx.inspect.instruments`.
- **No comprueba que los tramos no se solapen** ni que caigan dentro de los datos; la preflight (`python3 -m core.assets`) sí avisa cuando un tramo pide datos que no existen.
- **No propaga nada.** Cambiar un coste o una fecha no reconfigura ningún proyecto de SQX ya escrito: los costes y las ventanas viven dentro de cada tarea.
- **No crea un main nuevo** del Cross Market: un bloque nuevo en `_markets.yaml` necesita su feed y su timeframe, y eso no es una fila.

Si algo falla

- **ModuleNotFoundError: ruamel** — `python3 -m pip install --user ruamel.yaml==0.18.16`.
- **La zona se queda con datos viejos después de editar un YAML a mano** — el botón **Recargar** de abajo a la izquierda relea todo.
- **KeyError al abrir un activo copiado a mano** en `symbols/` — le falta su bloque en `_policy.yaml`. Los que se dan de alta desde la ventana lo llevan.
- **Una sola ficha por fila** — la ventana es estrecha: por debajo de unos 880 píxeles de ancho útil las fichas se apilan, porque dos no caben legibles.
- **El calendario de un tramo «sin decidir» abre en 1970** — es el suelo del selector; teclea el año directamente.

66. Configuración SQX — los ajustes con que nacen los proyectos nuevos

La zona **Configuración SQX** de la ventana, en BIBLIOTECA. Reúne en un sitio todos los ajustes con los que se construye y se prueba un proyecto nuevo de SQX —precisión, motor, tipos de orden, salidas, money management, crosschecks, cross-market, CrossTF, MC Retest, WFC, SPP y WFM—, cada test en su sección, y deja cambiarlos con un desplegable allí donde los valores posibles son fijos.

Qué pregunta responde

- **¿Con qué ajustes se va a construir y a probar el próximo proyecto?** Cada valor que `sqx/projects/` escribe en las tareas, leído de `assets/_build.yaml`, con la frase que el propio fichero pone encima para explicarlo.
- **¿Qué timeframes lee el CrossTF, con qué precisión corre la WFM, cuántas simulaciones lleva cada MC Retest?** Sin abrir el YAML: un desplegable por timeframe, uno por precisión, una caja por número.
- **¿Qué es común a todos los activos?** Los dos esquemas de coste (`_classes.yaml`), lo global de `_policy.yaml` —qué es cada tramo, el swap, los rangos del MC Retest— y el universo de retest de `_markets.yaml`, que aquí sólo se consulta. Antes vivían en Activos.

Cuándo lo usas, y cuándo no

Lo usas antes de crear un proyecto, para mirar o cambiar cómo va a salir: por ejemplo, para añadir un timeframe al CrossTF, bajar las simulaciones de una tarea del MC Retest cara o cambiar la precisión de un test.

No lo usas para cambiar un proyecto que ya existe. **Lo que se cambia aquí se aplica a los proyectos que se creen a partir de ahora; los existentes no cambian** — la zona lo dice arriba, en amarillo. Un `project.cfx` ya creado lleva sus ajustes dentro y esta zona no lo toca. Tampoco es el sitio de los costes, las fechas de los tramos o los mercados de un activo concreto: eso es **Activos** (capítulo 38). Y la zona **no habla con SQX**: sólo lee y escribe ficheros de `assets/`.

Antes de empezar

- La ventana y su demonio (capítulo 35). No hace falta SQX ni ningún worker.
- Para verla sin la ventana entera (o sacar sus capturas sin pantalla), la zona también se abre sola, con el demonio ya levantado (`python3 -m ui.daemon.serve`, o la ventana abierta con `bin/algoui`):

```
python3 -m ui.desktop.sqxconfig.preview
```

Cómo se ejecuta

En la ventana: **BIBLIOTECA** → **Configuración SQX** (o `bin/algoui --zone "Configuración SQX"`). O suelta, con el comando de arriba.

flag de preview	obligatorio	qué hace
<code>--shot CARPETA</code>	no	guarda cuatro capturas (todo plegado, CrossTF, MC Retest, WFM) en esa carpeta y sale

Abrir la zona tarda menos de un segundo: lee cuatro ficheros YAML. No toca SQX, así que se puede usar con la GUI del maestro abierta y con el custodio en mitad de una corrida.

Para cambiar un valor:

1. Pulsa en el índice de la izquierda (o en la cabecera ►) la sección que quieres: se despliega.
2. Donde hay un desplegable, elige; se escribe al instante. Donde hay una caja, escribe el número y sal de la caja (tabulador o clic fuera); se escribe al salir.
3. En las listas ordenadas —los tipos de orden, los timeframes del CrossTF, los crosschecks— hay un desplegable por elemento, + añade al final la primera opción que la lista aún no lleva y – quita el último. Una lista no puede repetir un elemento.
4. La línea bajo el título dice, en verde, qué se ha escrito; o, en rojo, por qué no.

Qué produce

Cada cambio escribe **una línea** en el fichero que lo guarda — `assets/_build.yaml` , `assets/_classes.yaml` o `assets/_policy.yaml` — a través de `core.assetwrite` , el único que escribe en `assets/` . Los comentarios, las comillas y las listas en una línea (`[H4, H12]`) se quedan como estaban. Como `assets/` está en git, el cambio aparece en `git diff` ; se deshace desde la propia ventana, volviendo a elegir el valor de antes.

Cómo se lee el resultado

BIBLIOTECA · CONFIGURACIÓN SQX

Configuración SQX

Se aplica a los proyectos que se creen a partir de ahora; los existentes no cambian. Cada cambio se escribe en assets/ conservando los comentarios, y git lo ve.

21 secciones leídas de assets/

CONSTRUCCIÓN Y TESTS · _BUILD.YAML

Complejidad de las reglas

Tipos de orden

Salidas

Money management

Opciones de trading

Motor de backtest

Databank

Precisión

Crosschecks

Cross-market

Cross-timeframe (CrossTF)

MC Retest

WFC y CSCV

SPP

Walk Forward Matrix (WFM)

ESQUEMAS DE COSTE · _CLASSES.YAML

Clase forex

Clase no forex

POLÍTICA COMÚN · _POLICY.YAML

Tramos: qué es cada uno

Swap

MC Retest

UNIVERSO DE RETEST · _MARKETS.YAML

Mercados de retest

CONSTRUCCIÓN Y TESTS · _BUILD.YAML

Complejidad de las reglas · 3 valores · assets/_build.yaml

Tipos de orden · 1 valor · assets/_build.yaml

Salidas · 8 valores · assets/_build.yaml

Money management · 9 valores · assets/_build.yaml

Opciones de trading · 2 valores · assets/_build.yaml

Motor de backtest · 1 valor · assets/_build.yaml

Databank · 2 valores · assets/_build.yaml

Precisión · 2 valores · assets/_build.yaml

Crosschecks · 3 valores · assets/_build.yaml

Cross-market · 3 valores, 1 bloqueados · assets/_build.yaml

Cross-timeframe (CrossTF) · 5 valores, 1 bloqueados · assets/_build.yaml

MC Retest · 72 valores, 8 bloqueados · assets/_build.yaml

WFC y CSCV · 14 valores, 8 bloqueados · assets/_build.yaml

SPP · 9 valores, 2 bloqueados · assets/_build.yaml

Walk Forward Matrix (WFM) · 59 valores, 3 bloqueados · assets/_build.yaml

ESQUEMAS DE COSTE · _CLASSES.YAML

Clase forex · 17 valores, 8 bloqueados · assets/_classes.yaml

Clase no forex · 19 valores, 8 bloqueados · assets/_classes.yaml

POLÍTICA COMÚN · _POLICY.YAML

Tramos: qué es cada uno · 7 valores · assets/_policy.yaml

Swap · 2 valores · assets/_policy.yaml

MC Retest · 14 valores, 10 bloqueados · assets/_policy.yaml

UNIVERSO DE RETEST · _MARKETS.YAML

Mercados de retest · 4 valores, 4 bloqueados · assets/_markets.yaml

recargar

A la izquierda, el índice: primero **Construcción y tests** (`_build.yaml` , una sección por test), luego **Esquemas de coste** (`_classes.yaml`), **Política común** (lo global de `_policy.yaml`) y **Universo de retest** (`_markets.yaml`). Cada cabecera dice cuántos valores tiene la sección y cuántos están **bloqueados**.

BIBLIOTECA · CONFIGURACION SQX

Configuración SQX

Se aplica a los proyectos que se creen a partir de ahora; los existentes no cambian. Cada cambio se escribe en assets/ conservando los comentarios, y git lo ve.

21 secciones leídas de assets/

CONSTRUCCIÓN Y TESTS · _BUILD.YAML

Complejidad de las reglas

Tipos de orden

Salidas

Money management

Opciones de trading

Motor de backtest

Databank

Precisión

Crosschecks

Cross-market

Cross-timeframe (Crosstf)

MC Retest

WFC y CSCV

SPP

Walk Forward Matrix (WFM)

ESQUEMAS DE COSTE · _CLASSES.YAML

Clase forex

Clase no forex

POLITICA COMÚN · _POLICY.YAML

Tramos: qué es cada uno

Swap

MC Retest

UNIVERSO DE RETEST · _MARKETS.YAML

Mercados de retest

► Databank · 2 valores · assets/_build.yaml

► Precisión · 2 valores · assets/_build.yaml

► Crosschecks · 3 valores · assets/_build.yaml

► Cross-market · 3 valores, 1 bloqueados · assets/_build.yaml

▼ Cross-timeframe (Crosstf) · 5 valores, 1 bloqueados · assets/_build.yaml

Tramo

Precisión

Condiciones (vacío) · bloqueado

Timeframes extra

Los timeframes extra POR DEFECTO según el timeframe en que se construyó la estrategia, en el orden en que se vuelven los bloques 1, 2, ... El bloque 0 es siempre el test principal. '--timeframes' en la línea de comandos manda sobre esto. Dueño, 2026-09-26: desde M30, H1 y H4; desde H1, H4 y H12 (H12 es un timeframe propio de SQX, "custom"). D1 no es alcanzable escalando desde H1: medido 2026-09-23, mueve los _

M30

aviso: un run ya hecho se relea con este valor

H1

aviso: un run ya hecho se relea con este valor

► MC Retest · 72 valores, 8 bloqueados · assets/_build.yaml

► WFC y CSCV · 14 valores, 8 bloqueados · assets/_build.yaml

► SPP · 9 valores, 2 bloqueados · assets/_build.yaml

► Walk Forward Matrix (WFM) · 59 valores, 3 bloqueados · assets/_build.yaml

ESQUEMAS DE COSTE · _CLASSES.YAML

► Clase forex · 17 valores, 8 bloqueados · assets/_classes.yaml

► Clase no forex · 19 valores, 8 bloqueados · assets/_classes.yaml

POLITICA COMÚN · _POLICY.YAML

► Tramos: qué es cada uno · 7 valores · assets/_policy.yaml

► Swap · 2 valores · assets/_policy.yaml

► MC Retest · 14 valores, 10 bloqueados · assets/_policy.yaml

UNIVERSO DE RETEST · _MARKETS.YAML

► Mercados de retest · 4 valores, 4 bloqueados · assets/_markets.yaml

Cada fila es un valor: su nombre, su editor y, si el fichero le pone un comentario, ese comentario a la derecha. Los comentarios largos se cortan y **seguir leyendo** los abre enteros. Un valor **bloqueado** se ve pero no se cambia desde aquí, y al pasar el ratón dice por qué:

- **Títulos y nombres de databank** (MCR 1 Bar, WFC_Build...): los busca por ese nombre exacto el análisis de Python; cambiarlos aquí lo rompería en silencio.
- **Tipo de periodo y tipo de optimización de la WFM**: fijos por decisión tuya del 24-09.
- **Condiciones de aceptación** del cross-market, el CrossTF y el WFC: son estructuras, no valores, y vacías significan «ninguna», por decisión tuya.
- **Los campos fields, field y unit de los esquemas de coste**: core/assetdata.py, core/assets.py y core/assetwrite.py los leen por su nombre y cada fichero de activo lleva esos campos; cambiarlos aquí rompería todos los activos.
- **Nombres y valores de fábrica de SQX** en los rangos del MC Retest, y **todo el universo de retest**, que se decide en Activos.

BIBLIOTECA · CONFIGURACIÓN SQX

Configuración SQX

Se aplica a los proyectos que se creen a partir de ahora; los existentes no cambian. Cada cambio se escribe en assets/ conservando los comentarios, y git lo ve.

21 secciones leídas de assets/

CONSTRUCCIÓN Y TESTS · _BUILD.YAML

Complejidad de las reglas
Tipos de orden
Salidas
Money management
Opciones de trading
Motor de backtest
Databank
Precisión
Crosschecks
Cross-market
Cross-timeframe (CrossTF)
MC Retest
WFC y CSCV
SPP
Walk Forward Matrix (WFM)

ESQUEMAS DE COSTE · _CLASSES.YAML

Clase forex
Clase no forex

POLITICA COMÚN · _POLICY.YAML

Tramos: qué es cada uno
Swap
MC Retest

UNIVERSO DE RETEST · _MARKETS.YAML

Mercados de retest

▼ MC Retest · 72 valores, 8 bloqueados · assets/_build.yaml

MC Retest – Paso 13 del workflow. Ocho tareas, cada una perturbando UNA cosa y volviendo a correr el backtest entero N veces, más una octava que perturba las seis a la vez. Aislar es todo el diseño: una tarea que mueve dos cosas no puede atribuir el daño a ninguna. Δ Los títulos son un CONTRATO con 'studies/breakage/mcRetest/inputs/tasks.py', que busca los databanks por ese nombre exacto y verifica qué métodos _

seguir leyendo

Simulaciones1000≥ 1

Tareas · MCR 1 Bar

TítuloMCR 1 Bar · bloqueado

Tramobuild

Simulaciones1000≥ 1

Precisión1 · sólo el timeframe elegido (el más rápido)

Muestra completa

Tareas · MCR 1 Bar · Métodos · Randomize Starting Bar

Max Change500≥ 0

Tareas · MCR 2 Spread

TítuloMCR 2 Spread · bloqueado

Tramobuild

Simulaciones1000≥ 1

Precisión2 · un minuto (lento)

Muestra completa

Tareas · MCR 3 Slippage

TítuloMCR 3 Slippage · bloqueado

Tramobuild

Simulaciones1000≥ 1

Precisión2 · un minuto (lento)

Muestra completa

Tareas · MCR 4 MinDist

TítuloMCR 4 MinDist · bloqueado

Tramo

Simulaciones1000≥ 1

Precisión2 · un minuto (lento)

Muestra completa

Sólo tiene sentido si la población lleva órdenes stop o limit: a mercado no hay ninguna orden pendiente cuya distancia al precio el broker pueda rechazar. Regla del dueño, 2026-09-23: 'sqx/projects/orders.py' _

seguir leyendo

Dentro de una sección, los subtítulos violetas siguen la forma del fichero: **Tareas › MCR 1 Bar › Métodos › Randomize Starting Bar**. Cada tarea se nombra por su título.

BIBLIOTECA · CONFIGURACIÓN SQX

Configuración SQX

Se aplica a los proyectos que se creen a partir de ahora; los existentes no cambian. Cada cambio se escribe en assets/ conservando los comentarios, y git lo ve.

21 secciones leídas de assets/

CONSTRUCCIÓN Y TESTS · _BUILD.YAML

Complejidad de las reglas
Tipos de orden
Salidas
Money management
Opciones de trading
Motor de backtest
Databank
Precisión
Crosschecks
Cross-market
Cross-timeframe (CrossTF)
MC Retest
WFC y CSCV
SPP
Walk Forward Matrix (WFM)

ESQUEMAS DE COSTE · _CLASSES.YAML

Clase forex
Clase no forex

POLITICA COMÚN · _POLICY.YAML

Tramos: qué es cada uno
Swap
MC Retest

UNIVERSO DE RETEST · _MARKETS.YAML

Mercados de retest

▼ Walk Forward Matrix (WFM) · 59 valores, 3 bloqueados · assets/_build.yaml

Walk Forward Matrix – Paso 19, y el ÚLTIMO que mira datos. Reoptimiza la estrategia sobre una ventana deslizante y corre cada elección sobre el tramo siguiente, para 30 combinaciones de cuántas pasadas y cuánto de cada pasada va fuera de muestra. Lo lee 'studies/optimisation/wfm/'. Δ Esta tarea NO se cobra al último tramo, que es la convención del resto de spans. Decisión del dueño, 2026-09-24: 'costs_segment: _

seguir leyendo

TítuloWFM · bloqueado

Tramobuild..oos2

Tramo de costesoos1

Precisión2 · un minuto (lento)

Máx. tests5000≥ 1

Tipo de periodopercent · bloqueado

Tipo de optimizaciónfloating · bloqueado

Tipo de walk-forwardexact_is_exact_oos · Exact IS, Exact OOS (slow)

Recorrido ± %35entre 1 y 100

Paso %4entre 1 y 100

OOS %

Param1: cuánto de cada pasada va fuera de muestra

Desde20entre 1 y 99

Hasta40entre 1 y 99

Paso5entre 1 y 99

Pasadas

Param2: cuántas pasadas

Desde6≥ 1

Hasta16≥ 1

Paso2≥ 1

a qué precio se cobra la ventana entera, no al del último tramo

Por PASO y por celda. El donante trae 500 y aquí hubo 15.000 un rato; el dueño lo bajó a 5.000 el 2026-09-24 al poner 'exact_is_exact_oos', y el reparto es deliberado: 'MaxTests' compra una elección mejor por _

seguir leyendo

F110: los ejes son nº de pasadas y % de OOS, no días ni barras

F110: la ventana de optimización se desliza, no se ancla

Con qué fidelidad se calcula cada pasada. Es el atributo 'type' del elemento «WalkForward» y SOX lo llama "Walk-Forward type": simulated.is.simulated_oos = 0 "Simulated IS, Simulated OOS (fastest)" _

seguir leyendo

Cuanto se mueve cada parámetro al reoptimizar, igual que en el SPP: ±35 % del valor original, y un paso mueve un 4 %. SOX no acepta el paso en %, acepta 'maxSteps', así que 'sqx/projects/wfm.py' lo deriva: _

seguir leyendo

Los desplegables sólo ofrecen lo que se puede elegir:

valor	opciones
precisión de un test	las cuatro de SQX: 1 timeframe elegido, 2 un minuto, 3 y 4 tick real
precisión de una tarea del MC Retest	1 y 2, las dos que documenta <code>_build.yaml</code>
motor	los ocho motores de SQX (hoy se usa <code>MetaTrader5 (hedged)</code>)
tipos de orden	<code>EnterAtMarket</code> , <code>EnterAtStop</code> , <code>EnterAtLimit</code> , <code>EnterReverseAtMarket</code>
timeframes del CrossTF	M1, M5, M15, M30, H1, H4, H12, D1 — los que sabe reescalar la fábrica de hermanas —, menos el de origen (desde H1 no se ofrece H1)
método de money management	los seis que llevan las tareas del donante
condición de parada del databank	<code>databank-full</code> , <code>passed-count</code> , <code>time-limit</code> , <code>never</code>
métrica de una condición de la WFM	las cuatro columnas <code>special</code> si la condición lee <code>special</code> ; si no, las que ya usan las demás (calibradas sobre las 150 casillas del maestro)
tramo de una tarea	<code>build</code> , <code>oos1</code> y <code>build..oos1</code> ; <code>oos2</code> y sus rangos sólo en el WFC y la WFM, que son a los que <code>_policy.yaml</code> se lo reserva
tipo de walk-forward	los tres de SQX, con su nombre en SQX
sí / no	«Sí · true» y «no · false»

△ Las listas de SQX —las cuatro precisiones, los ocho motores, las condiciones de parada— **se copiaron a mano el 27-09-2026** de la página de ajustes del propio SQX (`SQX/internal/web/RESULTS2/result2.js`). La ventana no lee ese fichero: si una versión nueva de SQX añade un motor o una precisión, no aparecerá aquí hasta que alguien edite `ui/daemon/sqxconfig/lists.py`.

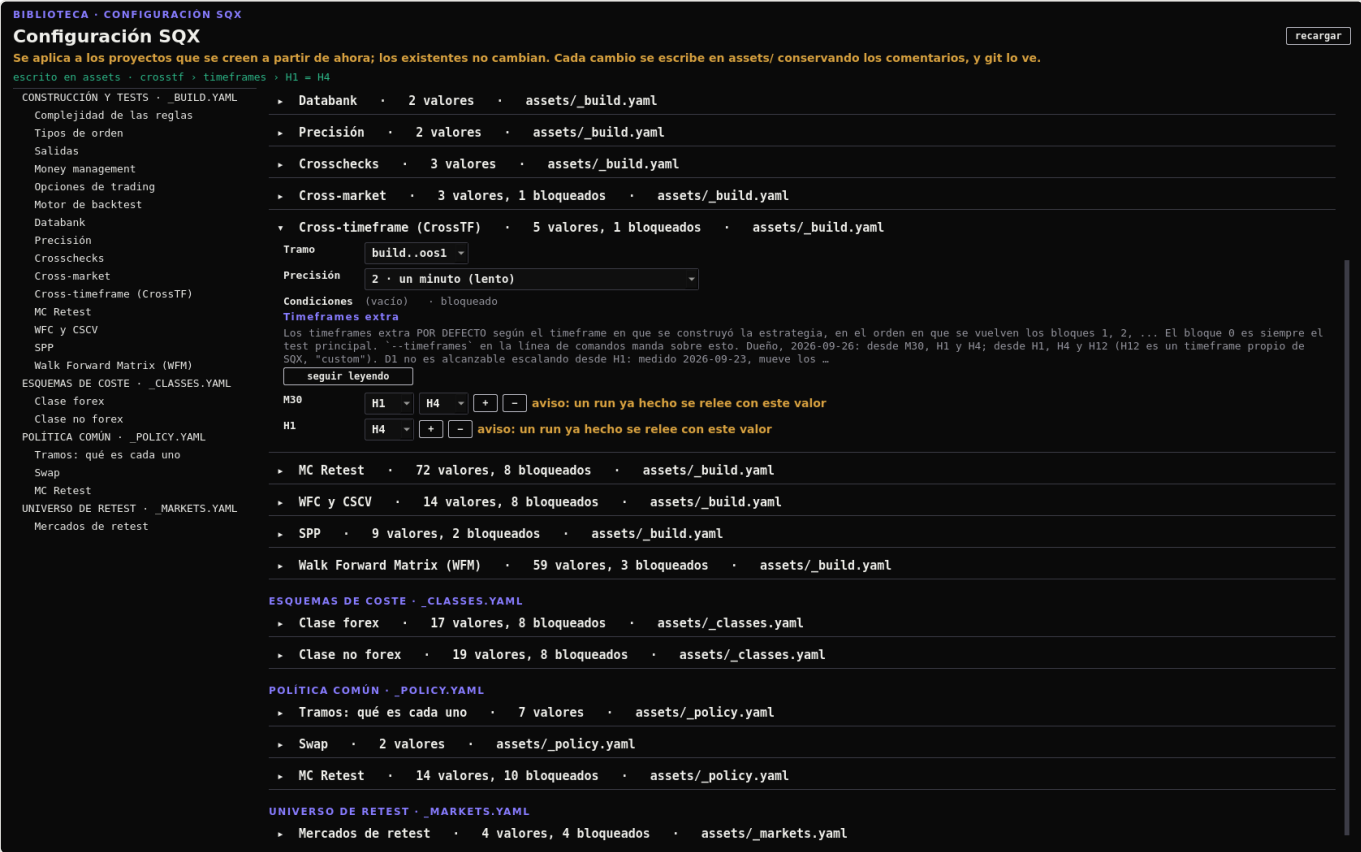
Los números también tienen su rango, cuando el fichero o la cantidad lo fijan: al lado de la caja se lee «≥ 1» o «entre 1 y 100». Un porcentaje no pasa de 100, el rectángulo de la WFM no pasa de 5 casillas (la matriz sólo tiene 5 columnas), y las horas mínimas de la salida por barras no pueden quedar por encima de las máximas.

El aviso amarillo de fila. Dos valores llevan al lado «*aviso: un run ya hecho se relee con este valor*»: los **timeframes extra del CrossTF** y el **tramo de cada tarea del WFC**. Los dos estudios que los usan (`studies/transfer/crossTF` y `studies/readings/structure`) vuelven a leer `_build.yaml` cuando analizan un run, y no guardan el valor con el que corrió. Si cambias el orden de los timeframes después de un CrossTF, al analizarlo cada celda se leerá sobre las barras de otro timeframe, sin que nada avise. Cámbialos **entre runs, no después de uno**; la propuesta de que cada run guarde los suyos está en `OPEN.md` §80.

Un ejemplo completo

Quitar el H12 de los timeframes extra de las estrategias construidas en H1, y volver a ponerlo (prueba del 28-09-2026):

1. Índice → **Cross-timeframe (CrossTF)**. Se despliega; en **Timeframes extra** > **H1** hay dos desplegables, **H4** y **H12**, y al lado el aviso amarillo.
2. Pulsa – en la fila de H1. El segundo desplegable desaparece y la línea de estado se pone verde:



escrito en assets · crosstf > timeframes > H1 = H4

3. En la terminal, `git diff assets/_build.yaml` enseña una sola línea cambiada y los comentarios intactos:

```
@@ -143,7 +143,7 @@ crosstf:
    # equivocarlo lee cada celda sobre las barras de otro timeframe, en silencio.
    timeframes:
        M30: [H1, H4]
-    H1: [H4, H12]
+    H1: [H4]
```

4. Para volver a ponerlo: + en la fila de H1 añade un desplegable al final (con la primera opción que la lista no lleva, M1); elige en él **H12**. El fichero vuelve a decir H1: [H4, H12] y `git diff` ya no enseña nada.

La ventana también dice que no. Tres intentos de la prueba, y la respuesta de cada uno — en los tres el fichero quedó intacto:

intento	respuesta en rojo
escribir sobre <code>exits</code> , una sección entera	«no es un valor de este fichero: o no existe, o es una sección entera o un elemento suelto de una lista»
escribir en <code>crosstf > nope</code> , que no existe	la misma frase
motor <code>MetaTrader9</code>	««MetaTrader9» no es una de las opciones: [MetaTrader4, ...]»

Qué NO te dice

- **No cambia ningún proyecto existente.** Si quieres que un proyecto ya creado corra con otro ajuste, hay que crear uno nuevo (capítulo 28) o reconfigurar esa tarea con su comando.
- **No valida que la combinación tenga sentido.** El desplegable garantiza que el valor existe en SQX, no que sea buena idea: D1 desde H1, por ejemplo, está en la lista y el fichero explica por qué no conviene. Lee el comentario de la fila antes de cambiar nada.
- **No sabe qué se ha construido con el valor anterior.** Git sí: `git log -p assets/_build.yaml`.
- **No enseña los ajustes por activo** (costes, tramos, mercados): eso es Activos.

Si algo falla

- **«El demonio no contesta»** en rojo bajo el título: el demonio no está levantado. Abre la ventana con `bin/algoui` o `python3 -m ui.daemon.serve`, y pulsa **recargar**.
- **«no escrito — ... no es un número»:** la caja de un número recibió texto. El valor del fichero no se ha tocado y la caja vuelve a enseñarlo.
- **«no escrito — ... no es una de las opciones»:** alguien cambió el fichero a mano con un valor que la zona no ofrece; el desplegable lo enseña como «(el del fichero, fuera de la lista)».
- **«no escrito — ... repite un elemento»** o **«... está por encima del máximo»:** la lista llevaría dos veces el mismo timeframe, o el número se sale del rango que la caja dice al lado.
- **«no escrito — Contrato con ...»** al tocar un valor bloqueado por otra vía: los títulos, los databanks y los campos `fields`, `field` y `unit` de los esquemas de coste los lee el código por su nombre, y sólo se cambian en el YAML y en el código a la vez.
- **Otro cambió el fichero mientras tenías la zona abierta:** pulsa **recargar** para volver a leer `assets/`.

67. Datos — AlgoData, velas, spread real y calidad del feed

La zona **Datos** de la ventana, en BIBLIOTECA. Enseña lo que guarda `~/Desktop/AlgoData` sin abrir ni un fichero a mano: qué carpetas hay, cuánto ocupan y de cuándo son; las velas de cada activo; y los dos estudios del paso 4 que describen el activo antes de construir nada — el spread real de Darwinex y la calidad del feed M1.

Qué pregunta responde

- **¿Qué tengo en AlgoData, cuánto ocupa y qué está viejo?** Cada carpeta hasta tres niveles, con su tamaño, sus ficheros, la fecha de lo último que se escribió y cuántos exports firmados (`manifest.json`) cuelgan de ella.
- **¿Qué pinta tiene el precio de este activo?** Sus velas a D1, H4 o H1, del último año (lo que se abre por defecto), de tres meses, de tres o diez años o de toda la historia.
- **¿Cuánto cuesta de verdad el spread aquí, y es constante?** El informe del spread real, tal y como lo escribió `studies.data.spread.scan`, y su banda para el MC Retest (`studies.data.spread.bands`).
- **¿Es fiable el feed M1 con el que construyo?** El informe de calidad del feed, tal y como lo escribió `studies.data.feedQuality.scan`: picos, congelados y huecos por año, y desde qué año es estable.

Cuándo lo usas, y cuándo no

Lo usas antes del paso 1 y en el paso 4: para ver si un activo tiene barras, ticks de Darwinex y calidad escaneada antes de montar un proyecto; para mirar qué exports hay y de qué fecha antes de pedir uno nuevo; y para saber qué carpeta está creciendo.

No lo usas para calcular nada. La zona **no corre ningún estudio, no exporta velas y no toca SQX**: lee lo que ya está en disco. Si un activo no tiene su informe de spread o de calidad, la zona dice qué comando lo escribe, y ese comando se corre aparte. Tampoco sustituye a la preflight (`python3 -m core.assets <SÍMBOLO>`), que es la que para el trabajo cuando algo falta.

Antes de empezar

- La ventana y su demonio (capítulo 35). No hace falta SQX ni ningún worker.
- Las velas salen de la librería de barras (`AlgoData/bars/<feed>/M1.parquet` , capítulo 13). Si un activo no está en ella, la pestaña Velas lo dice.
- El spread y la calidad salen de sus carpetas: `AlgoData/spread/<feed de ticks>/` y `AlgoData/feedQuality/<feed>/`. Los escriben los capítulos 59 y 57.

Cómo se ejecuta

```
algoui
```

En la barra de la izquierda, **BIBLIOTECA > Datos** (o `algoui --zone Datos`). Para verla sin la ventana entera, o sacar sus capturas sin pantalla, la zona se abre sola con el demonio ya arrancado (`python3 -m ui.daemon.serve`):

```
python3 -m ui.desktop.datazone.preview --asset XAUUSD --tab Velas
```

flag	obligatorio	qué hace
--asset	no	el activo que se abre; por defecto XAUUSD
--tab	no	la pestaña que se abre: Catálogo , Velas , Spread real , Banda del spread , Calidad del feed
--shot DIR	no	guarda una captura PNG de cada pestaña en esa carpeta y sale (para comprobar la zona sin pantalla, con QT_QPA_PLATFORM=offscreen)

Abrir la zona tarda menos de un segundo. La primera vez que pides un timeframe de un activo, el demonio lo remuestrea del M1 (~1 s) y lo guarda en `barsDerived/` ; las siguientes lo lee de ahí. El H1 de toda la historia de XAUUSD son 140 000 velas: tarda medio segundo en llegar.

Qué produce

Nada propio. Lo único que se escribe es la caché de la librería de barras: un timeframe pedido por primera vez queda en `AlgoData/barsDerived/<feed>/<TF>-<huella>.parquet` , como cuando lo pide cualquier estudio. Esa carpeta se puede borrar entera: se regenera al leer.

El demonio sirve cinco rutas, por si quieres miraras con `curl` :

ruta	qué devuelve
<code>/api/data/catalogue</code>	cada carpeta de AlgoData con tamaño, ficheros, formatos, última escritura y exports firmados
<code>/api/data/assets</code>	cada activo con su feed de barras, de ticks y de calidad
<code>/api/data/bars?feed=...&tf=D1&since=AAAA-MM-DD</code>	las velas OHLC de un feed a D1, H4 o H1
<code>/api/data/spread?feed=...&part=spread (o band)</code>	el informe del spread real, o su banda
<code>/api/data/feedquality?feed=...</code>	el informe de calidad del feed

Cómo se lee el resultado

Arriba, siempre: `AlgoData/` , cuánto ocupa entera y cuántos ficheros tiene, y el umbral de «rancio» (90 días sin escribir, de `perf/config.yaml`). Debajo, el selector de activo y una línea con lo que hay de él: desde cuándo hasta cuándo tiene M1 y cuántas velas, y qué feed usa para el spread y para la calidad («—» si no tiene).

Catálogo. Una fila por carpeta, anidadas; se despliegan con la flecha. «Última escritura» en ámbar quiere decir que nadie ha escrito ahí en más de 90 días. «Exports firmados» cuenta los `manifest.json` de debajo: una carpeta de datos sin ninguno no se puede reproducir. Al pasar el ratón por el nombre sale la ruta y, si la carpeta tiene su propio manifest, el comando que la produjo. Las carpetas del archivo de estrategias van sin desglosar: su nombre es la identidad de la estrategia, y la ventana nunca la enseña.

BIBLIOTECA · DATOS

Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

ActivoXAUUSD M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

CatálogoVelasSpread realBanda del spreadCalidad del feed

Árbol	Tamaño	Ficheros	Formatos	Última escritura	Exports firmados	Última firma
bars	1.7 GB	14	.parquet x13, .json x1	2026-09-24	1	2026-09-24
spread	1.5 GB	113	.json x46, .parquet x39, .md x14	2026-09-27	—	—
raw	356 MB	273	.parquet x238, .json x20, .csv x9	2026-09-27	20	2026-09-27
barsDerived	155 MB	35	.parquet x35	2026-09-27	—	—
reports	114 MB	1 152	.html x584, .json x475, .csv x44	2026-09-27	31	2026-09-27
structural	52 MB	62	.sqx x48, .json x5, .csv x4	2026-09-27	1	2026-09-27
pipeline	35 MB	2 025	.sqx x2000, .json x18, .parquet x3	2026-09-24	—	—
strategyPermutations	21 MB	202	.sqx x161, .json x17, .parquet x7	2026-09-27	—	—
archive	18 MB	111	.json x54, .csv x27, .parquet x24	2026-09-27	20	2026-09-27
manual-fuentes	16 MB	186	.png x113, .md x73	2026-09-27	—	—
metrics	15 MB	16	.csv x8, .json x8	2026-09-27	8	2026-09-27
logs	14 MB	1 484	.gz x1426, .log x48, .lock x6	2026-09-27	3	2026-09-27
harvest	8.5 MB	5	.parquet x3, .csv x1, .json x1	2026-09-27	1	2026-09-27
feedQuality	3.9 MB	69	.json x28, .parquet x14, .md x13	2026-09-26	—	—
crosstf	2.8 MB	32	.sqx x30, .parquet x2	2026-09-27	—	—
backups	2.6 MB	5	.json x4, .log x1	2026-09-26	—	—
projects	2 MB	23	.gz x21, .csv x1, .txt x1	2026-09-27	—	—
profiling	1.1 MB	14	.csv x9, .log x2, .txt x2	2026-09-27	—	—
templates	909 KB	36	.json x12, .xml x9, .md x7	2026-09-27	5	—
cache	747 KB	1	.json x1	2026-09-18	—	—
projectsBackup	611 KB	17	.config x4, .sqx x3, .txt x2	2026-09-22	—	—
notes	592 KB	17	.md x14, .png x3	2026-09-27	—	—
atrCalculator	124 KB	11	.sqx x6, .csv x3, .parquet x1	2026-09-27	—	—
ledger	74 KB	9	.jsonl x9	2026-09-27	—	—

Velas. Se abre en «1 año», donde a D1 caben las velas. Cuando cada vela cabe en al menos tres píxeles se pintan velas: azul la que sube, naranja la que baja (verde y rojo están reservados a los veredictos). Cuando no caben — toda la historia a D1 son 6 088 velas — cada columna de la gráfica resume las suyas: la línea gris es su rango de máximo a mínimo y la blanca el cierre de la última. La leyenda de debajo dice cuál de los dos dibujos estás viendo. Al pasar el ratón sale la vela, o el tramo de velas de esa columna, con su apertura, máximo, mínimo y cierre.

Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo **XAUUSD** M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo **Velas** Spread real Banda del spread Calidad del feed

Timeframe **D1** Período **1 año**

XAUUSD_DukasM1_Infinox · D1 · 259 velas · 2025-09-22 00:00 → 2026-09-22 00:00 · remuestreadas del M1 de la librería, idénticas al export de SQX



Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo **XAUUSD** M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo **Velas** Spread real Banda del spread Calidad del feed

Timeframe **D1** Período **toda la historia**

XAUUSD_DukasM1_Infinox · D1 · 6 088 velas · 2003-05-05 00:00 → 2026-09-22 00:00 · remuestreadas del M1 de la librería, idénticas al export de SQX



Spread real, Banda del spread, Calidad del feed. El informe entero del estudio, con su veredicto arriba, sus pestañas, sus avisos y su glosario, pintado igual que un estudio en la ficha de Estrategia (capítulo 58). La línea de encima dice de qué carpeta se ha leído. Cómo se lee cada informe está en su capítulo: 59 (spread real y banda) y 57 (calidad del feed).

BIBLIOTECA · DATOS

Datos

AlgoData/ · 4 GB en 5 912 Ficheros · rancio a partir de 90 días sin escribir

Activo

XAUUSD

 · M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo · Velas · Spread real · Banda del spread · Calidad del feed

XAUUSD_DarwTick_Infinox · leído de AlgoData/spread/XAUUSD_DarwTick_Infinox/, tal y como lo escribió el estudio

relativo NO constante → modelo volatilidad

vigilar

 nota 0.1877

Peor año del spread relativo a 1.96 × la mediana de los años (tolerancia ±20%); en puntos 1.91 ×. Propuesta con factor 1.25: comisión build 0.0131 %, oos1 0.0138 %, oos2 0.0118 %. Hoy assets/ declara spread_is 5.0 puntos, spread_oos 10.0 puntos, comisión 0.001 (pct_of_notional). MC Retest de spread: 0.53x-1.95x el spread de la tarea (cuantiles 2,5-97,5 % del día medido ÷ la media del modelo).

población · calculado 2026-09-27T11:28:56 · config 91570e0e61c04bbb · 0.06 s

Propuesta para SQX · Por año · Por hora · Modelo · Cola

Propone, no escribe: assets/ es del dueño (core.assetwrite).

Por tramo

Los días sin Darwinex (antes de octubre de 2017) llevan el spread del modelo: «% días medidos» dice cuánto del tramo es medición.

Tramo	Desde	Hasta	% días medidos	Spread medio (pb)	Comisión % propuesta	Precio mediano	Puntos equivalentes
build	2008-01-01	2017-12-29	2.448	1.051	0.01313	1 251	16.44
oos1	2018-01-02	2022-12-30	100	1.104	0.0138	1 705	23.52
oos2	2023-01-03	2026-08-28	99.79	0.9431	0.01179	2 641	31.14

GLOSARIO

Spread de apertura — El del primer tick de Darwinex de cada minuto: lo que paga una orden a mercado en la apertura de esa vela con precisión DATATICK.

pb — Puntos básicos del precio: 1 pb = 0,01 %. El spread relativo al precio.

Puntos — El spread dividido por el tick de assets/ — la unidad del spread de SQX (oro 0,01: USDJPY 0,01).

Constante — Cada año completo con su media a ± la tolerancia del dueño de la mediana de los años (ledger spread.constancy.tolerance).

Error del modelo — Media predicha del año ÷ media medida − 1, ajustando en un lado del año de corte y prediciendo el otro. «Hacia atrás» es la dirección que necesitan los años de Dukascopy.

Comisión % propuesta — Spread medio del tramo × el factor del dueño, en % del notional. SQX la cobra una vez por operación sobre el precio de apertura (OPEN.md #26), y una operación paga un spread entero ida y vuelta.

Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo XAUUSD M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo Velas Spread real **Banda del spread** Calidad del feed

XAUUSD_DarwTick_Infinox · leído de AlgoData/spread/XAUUSD_DarwTick_Infinox/, tal y como lo escribió el estudio

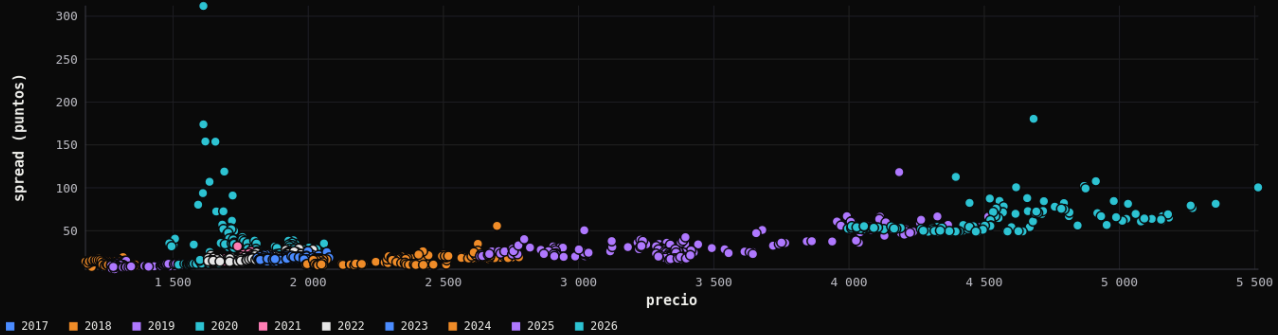
MC Retest de spread hoy: 17.8 – 83.4 puntos [informa](#)

Al último precio (4,274.5) la curva media da 49.2 puntos y la banda 0.025-0.975, 17.8 a 83.4. Exponente de la media $b = 1.18$ (1 = proporcional al precio, 0 = fijo en puntos). Peor año fuera de la banda por un lado: 18 % de sus días (lo esperado, 2,5 %).

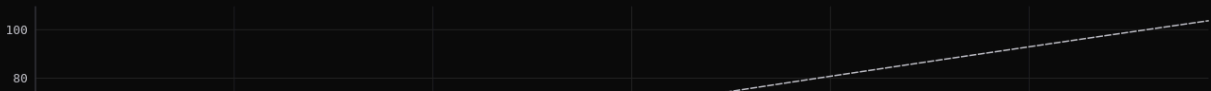
población · calculado 2026-09-27T10:48:00 · config f2bb399635cae4ba · 0.01 s

Curva Rangos para el MC Retest Calibración

Spread medio de cada día contra el precio



Curvas ajustadas: media y banda



Datos

AlgoData/ · 4 GB en 5 912 ficheros · rancio a partir de 90 días sin escribir

Activo XAUUSD M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox

Catálogo Velas Spread real Banda del spread **Calidad del feed**

XAUUSD_DukasM1_Infinox · leído de AlgoData/feedQuality/XAUUSD_DukasM1_Infinox/, tal y como lo escribió el estudio

estable desde 2014 [pasa](#) [nota](#) 16

$K = 25$; 16 anomalías marcadas por año (mediana desde 2013); huecos propios 11 al año (sesión bien deducida); 10 meses de episodio en congelados o huecos (0 desde el año estable) y 21 en el rollover, que sólo se cuentan.

población · calculado 2026-09-26T19:41:23 · config 6fd8fe2dc54145b · 0.08 s

Por año Por mes Los más extremos Calibración

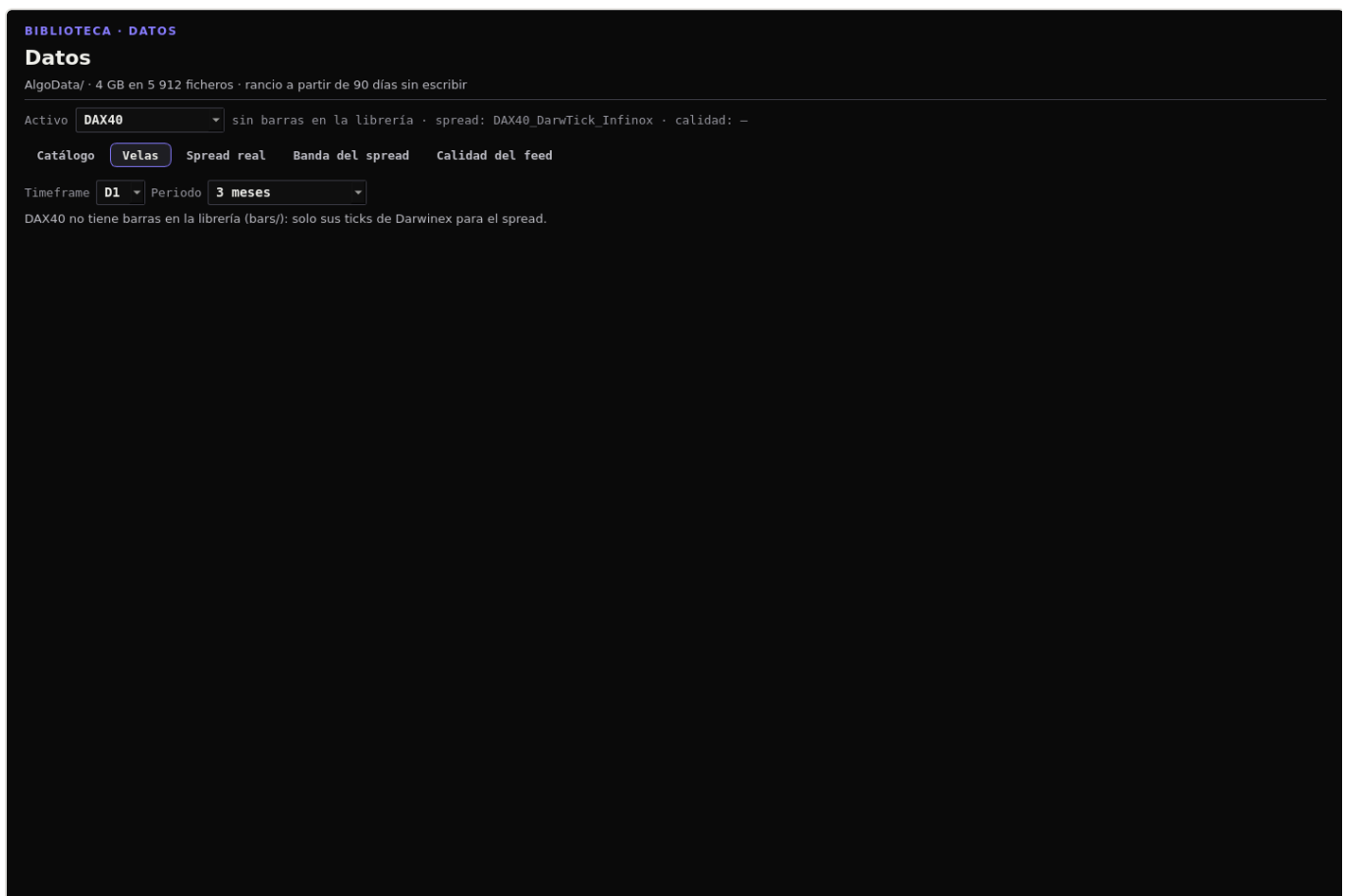
Recuento por año

«marcadas» suma lo que la atribución del paso 8 lee.

Año	Cierre	Mecha	Cierre y mecha	Cierre y vuelta	Mecha y vuelta	Vuelta m=1	Vuelta m=5	Movimiento extremo
2003	0	0	0	0	0	0	0	
2004	0	0	0	0	0	0	0	
2005	0	0	0	0	0	0	0	
2006	0	0	0	0	0	0	0	
2007	0	0	0	0	0	0	0	
2008	26	4	0	2	2	1	1	
2009	0	0	0	0	0	0	0	
2010	0	0	0	0	0	0	0	
2011	27	0	0	2	0	0	6	
2012	8	2	1	0	0	0	0	
2013	172	18	5	16	2	11	18	
2014	37	7	2	0	0	0	0	
2015	36	5	2	1	3	0	2	
2016	57	10	2	7	2	4	8	
2017	10	5	1	2	1	1	0	
2018	4	1	0	0	1	0	0	

Un ejemplo completo

1. `algoui` → **Datos**. Arriba: «AlgoData/ · 4 GB en 5 909 ficheros». En el Catálogo, `bars` (1.7 GB) y `spread` (1.5 GB) son lo que más pesa; `spread` no tiene ningún export firmado.
2. Activo **XAUUSD**: «M1 2003-05-05 → 2026-09-22 · 7.9 M velas · spread: XAUUSD_DarwTick_Infinox · calidad: XAUUSD_DukasM1_Infinox». Tiene las tres cosas.
3. **Velas**, D1, «1 año» (lo que abre): 259 velas del 2025-09-22 al 2026-09-22; el oro sube de 3 750 a 5 400 a finales de enero y vuelve a 4 370.
4. **Spread real**: «relativo NO constante → modelo volatilidad», en ámbar (vigilar). La pestaña «Propuesta para SQX» da la comisión por tramo: build 0.01313 %, oos1 0.0138 %, oos2 0.01179 %.
5. **Calidad del feed**: «estable desde 2014», en verde.
6. Cambio el activo a **DAX40**: «sin barras en la librería · spread: DAX40_DarwTick_Infinox · calidad: —». Velas dice que no hay barras; el spread sí se puede leer.



Qué NO te dice

- **Si los datos están al día con SQX.** Las velas son las de la última sincronización de la librería (`bars/manifest.json`); si SQX ha actualizado su historia después, aquí no se ve. Eso lo hace el capítulo 25.
- **Si un informe de spread o de calidad está caducado.** Se enseña el último que se escribió, con su fecha y su firma de configuración en la línea gris del veredicto; si se cambió la configuración del estudio después, hay que volver a correrlo.
- **Qué se puede borrar.** Una carpeta grande o rancia no es una carpeta sobrante. Para eso está `python3 -m perf.disk.report` (capítulo 12), que además tampoco borra nada.
- **Un «activo» es el nombre del feed hasta el primer guion bajo.** Si algún día hubiera dos feeds de barras del mismo símbolo, la zona usaría el primero en orden alfabético.

Si algo falla

- «El demonio no respondió a `/api/data/...`»: el demonio no está arrancado. `algoui` lo arranca solo; con `preview`, arráncalo antes con `python3 -m ui.daemon.serve`.
- «`XAUUSD_...` no tiene `spread.json` todavía. Lo escribe: `python3 -m studies.data.spread.scan --symbol XAUUSD`»: ese estudio no se ha corrido para ese activo. Corre el comando que dice.
- «No pude leer las velas de ... »: la librería de barras no tiene ese feed o su M1 está roto. Mira el capítulo 13.

60. Cómo se lee la ventana — etiquetas, números, identidad y modo lectura

Qué pregunta responde

- **¿Qué significa esta columna?** Toda etiqueta que la ventana enseña sale de un único glosario: «Beneficio neto», «Murió en», «Nulo · corridas», siempre con mayúscula inicial. Si un estudio nuevo trae una clave que el glosario aún no tiene, la ventana la humaniza («null.chunk3_traits» se lee «Nulo chunk3 traits») en vez de enseñarla cruda.
- **¿Cuánto es este número?** Ninguna cifra sale en notación científica. Se escribe entera, con los miles separados por un espacio («12 345»), y desde 100 000 con «K» o «M» («123.5 K», «2.4 M»). Un p-valor se lee «0.0123», y por debajo de 0.0001, «< 0.0001».
- **¿Esta máquina puede lanzar algo en SQX?** Si no hay ningún install de SQX, arriba de todo sale en ámbar «Esta máquina no tiene SQX: modo lectura», y el  de la barra de carga queda desactivado diciendo por qué en su propio texto.

Cuándo lo usas, y cuándo no

No es un comando que se corra: es cómo se ve la ventana desde el 27-09-2026. Lo usas para saber qué esperar al leerla. Lo único que se «ejecuta» es la captura (`--shot`), que sirve para ver la ventana sin abrirla, por ejemplo desde una sesión sin pantalla.

No sirve para Windows: la ventana solo se usa en el servidor Linux (decisión del dueño, 27-09-2026). El modo lectura es un guarda por si falta SQX, no una versión para otro sistema.

Antes de empezar

Nada. La ventana se abre como siempre, con `bin/algoui` o el icono del escritorio.

Cómo se ejecuta

La ventana, como siempre:

```
bin/algoui
```

Una captura de la ventana sin abrirla (guarda un PNG y se cierra sola):

```
QT_QPA_PLATFORM=offscreen python3 -m ui.desktop.launch --zone "Registro de búsquedas" --port 8756 --shot /tmp/capturas
```

flag	obligatorio	qué hace
<code>--zone</code>	no	la zona que se abre, por su nombre en la barra lateral; sin él, «Cobertura»
<code>--shot</code>	no	carpeta donde guardar la captura <code><zona>.png</code> ; la ventana se cierra a los 2,5 s
<code>--port</code>	no	el puerto del demonio; sin él, <code>ui_port</code> de <code>config/machine.yaml</code> (8765)

Tarda unos 5 segundos. No toca SQX: solo lee del demonio. Si el demonio no está levantado, lo arranca y lo para al terminar.

⚠ **Para capturas y pruebas, pasa siempre otro `--port`** (por ejemplo `--port 8756`). El 8765 es el demonio de tu ventana: si el que está ahí corre código más viejo y está ocioso, la ventana nueva lo sustituye, y una captura sin `--port` te lo cambiaría por debajo.

Para saber qué installs ve el demonio:

```
curl -s 127.0.0.1:8765/api/health
```

Qué produce

- La captura: `<carpeta>/<zona>.png` (los espacios del nombre pasan a `_`: el Registro de búsquedas se guarda como `Registro_de_búsquedas.png`). Sobrescribe una captura anterior de la misma zona en la misma carpeta.
- `/api/health` devuelve, además de lo de siempre, el bloque `sqx : {"installs": {"conductor": true, "custodian": true, "master": true}}` — cada install de `config/machine.yaml` y si su carpeta existe en esta máquina.

Cómo se lee el resultado

Las **pestañas activas** son un recuadro redondeado con el borde violeta, no una raya debajo. El violeta es el color de acento, y no significa nada: verde, ámbar y rojo siguen siendo pasa, vigilar y muere.

AlgoProject

BIBLIOTECA

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Configuración SQX

Datos

PROYECTO

Proyectos

Proyecto

Estrategia

OPERACIÓN

En marcha

Registro de búsquedas

PORTFOLIOS

Portfolios

Recargar

Proyectos > elige proyecto > elige estrategia

Registro de búsquedas **ESTUDIO** USDJPY_M30_donchianUpperCrossUp

Una línea por búsqueda que miró datos y redujo una población, en todo el estudio (activo + timeframe + familia). Sólo lectura: el ledger no se edita. Tres supervivientes de 10.000 no valen lo que tres de 50.

Por qué importa: cada mirada a los datos es una prueba más, y cuantas más pruebas, más fácil es que el mejor resultado sea suerte. Este registro las cuenta todas – las de los estudios, las del SQX y los filtros que aplicas en la ventana («lanzado por: ventana») – para que el Sharpe desinflado y la puerta ciega del paso 20 sepan cuánto se ha buscado.

1 de 6 búsquedas de este estudio son filtros aplicados o borrados a mano en la ventana; su criterio sale en violeta.

PUERTA CIEGA DEL PASO 20 · 17 pendiente · 18 pendiente · 19 pendiente
ledger: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19]. Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala

PROBADO EN TOTAL · N = 200 candidatos en 1 búsqueda(s) · $\sigma = 0.0223$ (per_period) – la σ que necesita el Sharpe desinflado

Embudo

Historia gastada

Búsquedas

Cuándo	Paso	Tramo	Entran	Salen	Conserva	Criterio
2026-09-27 20:30	18.5	build	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-27 20:30	18.5	oos1	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-27 20:37	18.5	build	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-27 20:37	18.5	oos1	150	150	100 %	marketSurfaces: 10 mercados (principal + 9 de 9 declarados) x 150 variantes, NetProfit; 0 pasan
2026-09-28 07:18	8	oos1	200	183	91.5 %	Net profit (OOS) > 0.0 AND spread.operaciones [OOS] ≥ 30.0
2026-09-28 07:24	8	oos1	200	200	100 %	snoopingScreen/stepm

registry.csv · runs.csv · library/ — 7 filas x 6 timeframes · daemon en 127.0.0.1

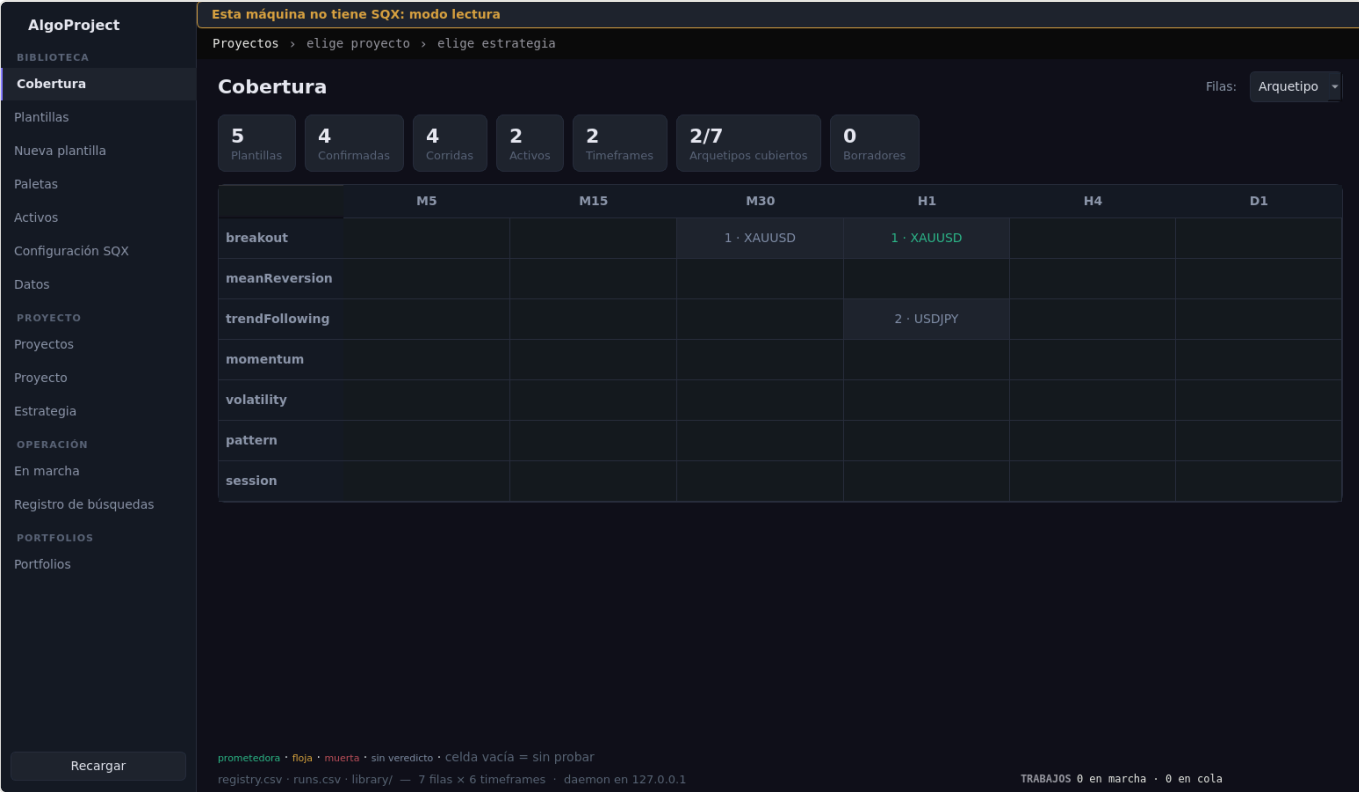
TRABAJOS 0 en marcha · 0 en cola

Las **etiquetas** salen del glosario. En el panel de databanks de Proyecto las columnas dicen «Estrategia», «Beneficio neto IS», «Operaciones IS», «Factor de beneficio IS», «% ganadoras IS»; en la ficha de una estrategia, las métricas de SQX salen en castellano y el nombre original de SQX aparece al pasar el ratón.



La **identidad** (la huella SHA-256 de la estrategia) no se enseña en ningún texto de la ventana: ni en la barra de contexto, ni en la cabecera de un estudio, ni en las tablas. Sigue ahí por debajo — con ella se emparejan los resultados — y se ve al pasar el ratón sobre el nombre de la estrategia en la barra de contexto.

El **modo lectura** es la franja ámbar de arriba. Todo se sigue leyendo; el  de la barra de carga (que reintenta cargar operaciones en el conductor) queda desactivado, con el texto « sin SQX: modo lectura». Ninguna zona se pone gris. El otro botón que llega a SQX, «Continuar workflow», no depende de la franja: su propia comprobación (capítulo 69) se niega si el proyecto no vive en un worker de esta máquina.



Un ejemplo completo

El 27-09-2026 se probó el modo lectura con un `config/machine.yaml` de prueba que apuntaba los tres installs a carpetas que no existen (y el demonio a otro puerto, para no molestar al de siempre). El demonio respondió:

```
{ "ok": true, "module": "templates", "code": "0cec582189c36056", "pid": 2236596, "busy": 0,
  "sqx": { "installs": { "conductor": false, "custodian": false, "master": false } } }
```

Con el `machine.yaml` real:

```
{"ok":true,"module":"templates","code":"0cec582189c36056","pid":2236895,"busy":0,
"sqx":{"installs":{"conductor":true,"custodian":true,"master":true}}}
```

y la franja no aparece. La captura del modo lectura de arriba se rehízo el 28-09-2026 sobre la ventana definitiva, pasándole a la ventana esa misma respuesta —los tres installs en `false` — sin tocar el `machine.yaml` de verdad.

Qué NO te dice

- **Que un install esté vivo.** «true» solo dice que su carpeta existe. Un worker parado sale igual de «true»; saber si responde es cosa de la zona **En marcha** (capítulo 70).
- **Que todas las claves estén ya en el glosario.** Una clave que un estudio nuevo trae y el glosario aún no tiene sale humanizada («Nulo chunk3 traits»), no traducida; se arregla añadiéndola al glosario (`ui/text/glossary.py`).

Si algo falla

- **La captura sale en negro o a medio pintar:** alguna zona tardó más de 2,5 s en leer del demonio. Repite la captura; la segunda vez el demonio ya tiene la caché caliente.
- **«el demonio no respondió en el puerto 8765»:** arráncalo a mano con `python3 -m ui.daemon.serve` para ver el error.